



UNIVERSITÉ VIRTUELLE DE CÔTE D'IVOIRE

UFR INFORMATIQUE
ET SCIENCES DU NUMÉRIQUE

N° d'ordre :

Mémoire Pour l'obtention du :
Diplôme de Master

Domaine : Sciences et Technologies

Mention : Informatique et Sciences du Numérique

Spécialité : Cybersécurité et Internet des Objets (CIO)

Présenté par

ACHI JEAN MARTIAL ZEPHIRIN

Sujet :

SYSTÈME DE GESTION DES EMBOUTEILLAGES AVEC DES FEUX TRICOLORES CONNECTÉS BASÉ SUR LA TECHNOLOGIE SDN

Soutenu le 25 septembre 2026 devant le jury composé de :

M. BABRI Michel

Professeur Titulaire

Université Virtuelle de Côte d'Ivoire

Président

M. ATTA Amanvon Ferdinand

Maître Assistant

Université Virtuelle de Côte d'Ivoire

Examineur

M. ANOH Nogbou Georges

Maître de Conférences

Université Félix Houphouët-Boigny

Encadreur

AVANT-PROPOS

L'urbanisation rapide et soutenue que connaissent les grandes métropoles africaines soulève aujourd'hui des défis considérables en matière de mobilité urbaine. En Côte d'Ivoire, le district autonome d'Abidjan, avec ses plus de six millions d'habitants concentrés sur un territoire exigu, illustre à lui seul l'ampleur de la crise de la circulation : embouteillages chroniques aux heures de pointe, feux tricolores à cycles rigides inadaptés aux flux réels, pertes économiques importantes et dégradation de la qualité de vie des usagers de la route.

Face à cette réalité, les systèmes classiques de régulation du trafic, fondés sur des temporisations fixes et une logique décentralisée, montrent leurs limites. Chaque carrefour fonctionne de manière autonome, sans connaissance des flux globaux, ce qui génère des inefficacités notables : files d'attente disproportionnées, intersections inutilement bloquées, incapacité à s'adapter aux variations dynamiques de la circulation.

C'est dans ce contexte que les technologies de **Software-Defined Networking (SDN)** offrent des perspectives prometteuses. En dissociant le plan de contrôle du plan de données, le SDN permet une gestion centralisée, programmable et dynamique des équipements du réseau. Appliqué à la gestion de la circulation urbaine, ce paradigme ouvre la voie à une coordination intelligente des feux tricolores connectés, capables de s'adapter en temps réel aux conditions de trafic grâce aux données issues de capteurs IoT et aux communications V2X.

Ce mémoire s'inscrit dans cette démarche. Il explore la conception, l'implémentation et l'évaluation d'un **Système de Gestion des Embouteillages avec des Feux Tricolores Connectés basé sur SDN**, dans un environnement de simulation reproduisant les contraintes d'une ville africaine. À travers l'utilisation d'outils open source tels que SUMO, Mininet-WiFi et le contrôleur Ryu, ce travail cherche à démontrer la faisabilité technique et l'efficacité opérationnelle d'une telle solution, tout en proposant des pistes concrètes pour son déploiement dans des contextes à ressources limitées.

Ce travail se veut une contribution modeste mais sincère à la réflexion sur la mobilité durable en Afrique, et à l'appropriation locale des technologies réseau avancées au service du bien commun.

DÉDICACE

À mes parents

À mon épouse **ESMEL Nome Rita Marlene épouse ACHI** et à mes enfants

À mes frères et sœurs, particulièrement à **Feu ACHI Judicaël Rodrigue**.

REMERCIEMENTS

Avant tout, je rends grâce à mon Dieu, **Jésus-Christ**, qui m'a accordé l'orientation spirituelle, la santé, le courage, la volonté et la patience nécessaires pour mener à bien ce projet de fin d'études. C'est par Sa grâce que j'ai pu franchir chaque étape de ce parcours exigeant.

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce mémoire portant sur le **Système de Gestion des Embouteillages avec des Feux Tricolores Connectés basé sur SDN**. Leurs soutiens, expertises et encouragements ont constitué un appui d'une valeur inestimable tout au long de cette aventure académique.

Mes remerciements les plus sincères vont en premier lieu à mon encadreur, **M. ANOH Nogbou Georges**, Maître de Conférences à l'Université Félix Houphouët-Boigny, pour la qualité de son encadrement, la pertinence de ses conseils et la constance de son soutien tout au long de ce travail de recherche. Sa vision novatrice, ses orientations éclairées et ses suggestions précieuses ont considérablement enrichi ce mémoire et forgé ma rigueur intellectuelle.

J'adresse également ma reconnaissance aux collègues de la **Direction des Bases de Données et des Statistiques Économiques**, dont les encouragements répétés et la bienveillance ont su me redonner de l'élan dans les moments de doute.

Je souhaite exprimer une gratitude toute particulière à mes amis et camarades de classe, **M. CURTIS DOGBO** et **M. IRIE SAIN ANGE**, qui ont été bien plus que de simples compagnons d'études. Leur aide technique véritable, leur disponibilité sans faille et leur solidarité à chaque étape de ce projet ont été déterminantes dans la concrétisation de ce travail. Leur engagement à mes côtés restera gravé dans ma mémoire.

Je remercie également les membres du jury, le **Professeur BABRI Michel**, Président du jury, et ses consorts, pour avoir accepté d'évaluer ce mémoire avec attention et rigueur. Leurs remarques constructives et leur regard critique ont indubitablement contribué à l'amélioration de la qualité de ce travail.

Enfin, que toutes les personnes qui ont, d'une manière ou d'une autre, apporté leur pierre à l'édifice de ce mémoire trouvent ici l'expression de ma sincère et profonde reconnaissance. Chaque geste de soutien, aussi discret soit-il, a compté et a eu sa part dans l'accomplissement de ce projet.

RÉSUMÉ

L'urbanisation accélérée des métropoles africaines génère une congestion chronique des réseaux routiers, à laquelle les systèmes de régulation à feux tricolores à cycles fixes ne peuvent répondre efficacement. En Côte d'Ivoire, le district autonome d'Abidjan, avec 6,3 millions d'habitants en 2021 (INS, 2021) et 1,4 million de véhicules en 2023 (ministère des Transports, 2023), illustre l'ampleur de ce défi : un automobiliste y perd en moyenne 1h45 par jour dans les embouteillages (Direction Générale des Transports Terrestres, 2022). Ce travail propose, conçoit et évalue un système de gestion des embouteillages à feux tricolores connectés fondé sur l'architecture Software-Defined Networking (SDN). La démarche expérimentale s'articule en deux phases complémentaires. La Phase A exploite le simulateur de trafic SUMO et l'interface TraCI pour évaluer un orchestrateur adaptatif sur un carrefour à profil asymétrique (1 300 véhicules, 65 % sur l'axe Nord-Sud). La Phase B recourt à Mininet-WiFi et au contrôleur SDN Ryu (OpenFlow 1.3) pour mesurer l'apport de la qualité de service (QoS) sur les communications de contrôle soumises à une congestion réseau délibérée de 16 Mbit/s sur un lien de 5 Mbit/s. Les résultats montrent que le contrôle adaptatif assure l'écoulement de la totalité des véhicules simulés (100 %), contre 74,9 % pour le système à cycles fixes (974/1 300) ; mesuré sur trente graines aléatoires indépendantes, ce résultat reste stable (25,11 % de véhicules bloqués en moyenne, IC 95 % [24,97 ; 25,25]) et se maintient face à un plan fixe dimensionné par la formule de Webster, qui n'écoule que 76,5 % du trafic. Parallèlement, l'activation des files de priorité HTB via SDN réduit significativement le jitter et le taux de perte du flux de contrôle en environnement congestionné. Ces résultats, mesurés séparément sur chacune des deux phases, illustrent la complémentarité de ces deux niveaux d'architecture, à savoir le contrôle applicatif adaptatif et le réseau SDN priorisé, pour une gestion intelligente et scalable du trafic urbain dans les contextes à ressources limitées.

Mots-clés : Software-Defined Networking ; feux tricolores connectés ; gestion adaptative du trafic ; SUMO ; Mininet-WiFi ; qualité de service ; mobilité urbaine

ABSTRACT

The rapid urbanization of African metropolitan areas generates chronic road congestion that fixed-cycle traffic signal systems are structurally unable to address. In Côte d'Ivoire, the Abidjan district, with 6.3 million inhabitants in 2021 (INS, 2021) and 1.4 million vehicles in 2023 (Ministry of Transport, 2023), exemplifies this challenge: commuters lose an average of 1 hour and 45 minutes daily to traffic congestion (Direction Générale des Transports Terrestres, 2022). This paper proposes, implements, and evaluates a traffic congestion management system based on Software-Defined Networking (SDN)-controlled connected traffic lights. The experimental approach is structured into two complementary phases. Phase A uses the SUMO traffic simulator and the TraCI interface to assess an adaptive orchestrator on an asymmetric-load intersection (1,300 vehicles, 65% on the North-South axis). Phase B employs Mininet-WiFi and the Ryu SDN controller (OpenFlow 1.3) to measure the impact of Quality of Service (QoS) mechanisms on control communications under deliberate network congestion of 16 Mbit/s over a 5 Mbit/s link. Results indicate that the adaptive controller achieves full traffic throughput (100% of simulated vehicles), compared to 74.9% for the fixed-cycle baseline system (974/1,300 vehicles); measured over thirty independent random seeds, this result remains stable (25.11% of vehicles blocked on average, 95% CI [24.97; 25.25]) and holds against a fixed plan dimensioned with Webster's formula, which clears only 76.5% of the traffic. Furthermore, activating Hierarchical Token Bucket (HTB) priority queues through SDN significantly reduces jitter and packet loss on control flows under congested network conditions. These findings, measured separately on each phase, illustrate the complementary value of the two architectural layers, namely adaptive application-level control and SDN-prioritized networking, as a technically feasible and scalable solution for intelligent urban traffic management in resource-constrained environments.

Keywords: Software-Defined Networking; connected traffic lights; adaptive traffic control; SUMO; Mininet-WiFi; quality of service; urban mobility

Sommaire

AVANT-PROPOS	I
DÉDICACE	II
REMERCIEMENTS.....	III
RÉSUMÉ	IV
ABSTRACT.....	V
Liste des Tableaux.....	VII
Liste des Figures	VIII
Liste des Abréviations.....	IX
Introduction générale	1
CHAPITRE 1 : GÉNÉRALITÉS SUR LES SYSTÈMES DE GESTION DU TRAFIC ET SDN.....	6
1. Généralités sur les systèmes de gestion du trafic.....	6
2. Généralités sur les SDN	17
CHAPITRE 2 : ÉTAT DE L'ART	26
Introduction.....	26
1. Systèmes Matériels et Architectures IoT pour la Gestion du Trafic	26
2. Vision par Ordinateur, Apprentissage Automatique et Contrôle Adaptatif.....	29
3. Revues Systématiques, Technologies Émergentes et Dimensions Transversales	31
Conclusion et perspectives.....	33
CHAPITRE 3 : CONCEPTION, IMPLÉMENTATION ET ÉVALUATION DES PERFORMANCES.....	35
1. Introduction.....	35
2. Environnement technique	36
3. Modélisation du carrefour et scénario de trafic	40
4. Phase A : Contrôle adaptatif des feux tricolores	42
5. Phase B : Réseau véhiculaire SDN avec QoS.....	48
6. Phase C : Rapport de performance consolidé	55
7. Proposition de déploiement : contrôleurs par secteur et redondance.....	58
8. Conclusion	60
CONCLUSION GÉNÉRALE.....	61
Annexe : Accès aux ressources du projet.....	62
BIBLIOGRAPHIE	64

Liste des Tableaux

Tableau 1.1 : Comparaison des systèmes classiques/intelligents et des approches SDN	25
Tableau 3.1 : Outils de l'environnement expérimental	36
Tableau 3.2 : Phases du feu tricolore du carrefour J1	40
Tableau 3.3 : Répartition asymétrique du trafic	41
Tableau 3.4 : Comparaison Baseline vs. Adaptatif (Phase A).....	47
Tableau 3.5 : Comparaison Sans QoS (B1) vs. Avec QoS SDN (B2).....	55
Tableau 3.6 : Répartition proposée des contrôleurs SDN par secteur.....	59

Liste des Figures

Figure 1.1 : Illustration historique des premiers feux de signalisation	6
Figure 1.2 : Feux à électrolampe à trois étapes.....	7
Figure 1.3 : Schéma de composant d'un feu	8
Figure 1.4 : Schéma de l'architecture SDN	17
Figure 1.5 : Architecture générale d'un réseau SDN pour la gestion de trafic urbain	19
Figure 3.1 : Contenu du dossier Mininet cloné.....	37
Figure 3.2 : Interface graphique SUMO-GUI.....	39
Figure 3.3 : Structure du répertoire experiments/	39
Figure 3.4 : Ensemble des scripts du projet	39
Figure 3.5 : Arborescence complète du projet	40
Figure 3.6 : Représentation du carrefour J1 dans SUMO	41
Figure 3.7 : Visualisation du scénario de trafic asymétrique	42
Figure 3.8 : Exécution du test Baseline	43
Figure 3.9 : Résultat, 974 véhicules traités sur 1 300	43
Figure 3.10 : Exécution du test Adaptatif (lancement de l'orchestrateur)	44
Figure 3.11 : Exécution du test Adaptatif, suivi en temps réel des commutations de phase.....	45
Figure 3.12 : Comptage des commutations NS et EW dans le log	45
Figure 3.13 : Exécution du script de comparaison Baseline vs. Adaptatif	46
Figure 3.14 : Rapport comparatif complet généré par le script	46
Figure 3.15 : Lancement du contrôleur Ryu (Terminal 1)	49
Figure 3.16 : Lancement de Mininet-WiFi (Terminal 2)	49
Figure 3.17 : Test de connectivité pingall (B1).....	50
Figure 3.18 : Résultat du pingall, connectivité totale confirmée	50
Figure 3.19 : Lancement des serveurs iperf en arrière-plan.....	51
Figure 3.20 : Lancement du trafic de fond.....	51
Figure 3.21 : Flux de contrôle car1, mesure du jitter sans QoS.....	51
Figure 3.22 : Vérification API Ryu, QoS désactivée (priority_enabled: false)	52
Figure 3.23 : Files HTB OVS confirmées (linux-htb)	53
Figure 3.24 : Activation manuelle de la QoS via force_qos_on.sh.....	53
Figure 3.25 : QoS activée.....	54
Figure 3.26 : Règles OpenFlow confirmant la priorisation UDP:9999	54
Figure 3.27 : Génération du rapport consolidé (merge_report.py)	56
Figure 3.28 : Rapport final, métriques croisées Phase A et Phase B	56
Figure 3.29 : Synthèse graphique des performances comparées	56

Liste des Abréviations

Abréviation	Signification
DRL	Deep Reinforcement Learning
DGTT	Direction Générale des Transports Terrestres (Côte d'Ivoire)
EW	Est-Ouest (direction de trafic)
HTB	Hierarchical Token Bucket (algorithme de mise en file QoS)
INS	Institut National de la Statistique (Côte d'Ivoire)
IoT	Internet of Things (Internet des Objets)
IoV	Internet of Vehicles (Internet des Véhicules)
LiFi	Light Fidelity (communication par lumière visible)
ML	Machine Learning (Apprentissage Automatique)
Min.	Ministère (des Transports, Côte d'Ivoire)
NS	Nord-Sud (direction de trafic)
OpenCV	Open Source Computer Vision Library
OpenFlow	Protocole de communication SDN entre contrôleur et commutateur
OVS	Open vSwitch (commutateur virtuel open-source)
QoS	Quality of Service (Qualité de Service)
RFID	Radio Frequency Identification
SDN	Software-Defined Networking (Réseau Défini par Logiciel)
SORT	Simple Online and Realtime Tracking
STS	Smart Traffic Signal (contrôleur de signalisation intelligente)
SUMO	Simulation of Urban MObility (simulateur de trafic urbain)
TraCI	Traffic Control Interface (API de contrôle SUMO)
UDP	User Datagram Protocol
V2I	Vehicle-to-Infrastructure (communication véhicule-infrastructure)
V2X	Vehicle-to-Everything (communication véhicule-tout)
VANET	Vehicular Ad-hoc Network (réseau ad hoc véhiculaire)
VLC	Visible Light Communication (communication par lumière visible)
YOLO	You Only Look Once (algorithme de détection d'objets en temps réel)
ADAS	Advanced Driver-Assistance Systems (systèmes d'aide à la conduite)
C-V2X	Cellular Vehicle-to-Everything (communication véhicule-tout cellulaire)
DSRC	Dedicated Short-Range Communications (communications dédiées à courte portée)
GSMA	GSM Association (association mondiale des opérateurs mobiles)
ITS / STI	Intelligent Transportation Systems / Systèmes de Transport Intelligents

Abréviation	Signification
ITS-G5	Norme européenne de communication V2X à 5,9 GHz (basée sur IEEE 802.11p)
LoRa	Long Range (technologie radio longue portée à faible consommation)
MOVA	Microprocessor Optimised Vehicle Actuation (système adaptatif de commande de feux)
NETCONF	Network Configuration Protocol (protocole de configuration réseau)
PMV	Panneau à Messages Variables
REST	Representational State Transfer (style d'architecture pour API web)
SCATS	Sydney Coordinated Adaptive Traffic System (système adaptatif de gestion du trafic)
SCOOT	Split, Cycle and Offset Optimisation Technique (système adaptatif de gestion du trafic)
UBR	Unité Bord de Route (équipement V2X installé en bordure de voirie)

Introduction générale

1. Contexte et problématique

L'urbanisation galopante que connaissent les grandes métropoles africaines, notamment en Afrique subsaharienne, engendre une pression croissante sur les infrastructures urbaines, en particulier les réseaux routiers. En Côte d'Ivoire, le district autonome d'Abidjan concentre plus de 20 % de la population nationale sur moins de 0,1 % du territoire, avec une population estimée à plus de 6,3 millions d'habitants en 2021 (INS, 2021). Cette concentration urbaine s'accompagne d'une explosion du nombre de véhicules : le parc automobile ivoirien est passé de 600 000 véhicules en 2010 à plus de 1,4 million en 2023 (ministère des Transports, 2023), sans que les infrastructures routières n'aient été redimensionnées en conséquence. La régulation de ce trafic repose sur un parc de feux tricolores dont l'entretien, la maintenance et l'extension relèvent de l'Agence de gestion des routes (AGEROUTE). Selon l'agence, le nombre de carrefours équipés de feux tricolores est passé de 156 en 1988 à 247 en 1999, pour dépasser 457 sur l'ensemble du territoire national sur la période 2021-2023 [1]. Le district d'Abidjan en concentre environ 350, d'après le chargé des opérations et de la signalisation de l'AGEROUTE [2], soit environ les trois quarts du parc national. Ce parc reste pourtant insuffisant et peu coordonné : en 2021, l'Autorité de la mobilité urbaine dans le Grand Abidjan (AMUGA) déployait à Cocody des brigades de régulation sur des carrefours retenus pour leurs bouchons fréquents et leur absence de feux tricolores [3], et elle prévoit désormais d'équiper de feux intelligents les 300 grands carrefours de l'agglomération, en commençant par 20 carrefours jugés problématiques [4].

L'une des manifestations les plus visibles de cette inadéquation est l'augmentation spectaculaire des embouteillages, particulièrement aux heures de pointe. Ces congestions routières ont des impacts multiples : pertes économiques (temps de transport, consommation de carburant), pollution atmosphérique (CO₂, particules fines), stress psychologique pour les conducteurs et inefficacité généralisée du système de transport. À Abidjan, selon une étude menée par la Direction Générale des Transports Terrestres (2022), un automobiliste passe en moyenne 1h45 par jour dans les embouteillages, ce qui représente l'équivalent de 21 jours perdus par an.

Face à cette situation, les systèmes classiques de gestion du trafic, notamment les feux tricolores à cycle fixe, apparaissent largement dépassés. Ces dispositifs, conçus à une époque où le volume de véhicules était bien plus faible, ne s'adaptent pas aux variations dynamiques du trafic. Ils fonctionnent de manière rigide, indépendamment des flux réels observés à

chaque intersection. Il en résulte des files d'attente disproportionnées, des intersections désertes où le feu reste inutilement rouge, et un gaspillage énergétique conséquent. Cette inefficacité est d'autant plus marquée dans les zones périurbaines où les flux sont déséquilibrés et non symétriques.

Dans un contexte où la mobilité urbaine est un enjeu majeur de développement durable (ODD 11), il devient impératif de repenser les mécanismes de régulation de la circulation, en s'appuyant sur des technologies intelligentes et flexibles. C'est dans cette logique que s'inscrit la présente étude, qui propose d'explorer une solution de **gestion dynamique des feux tricolores connectés**, basée sur les principes du **Software-Defined Networking (SDN)**, combinés aux technologies de l'Internet des Objets (IoT) et de la communication V2X.

2. Justification du sujet

La gestion intelligente du trafic fait partie des piliers des « villes intelligentes » (smart cities), telles que promues par les politiques publiques ivoiriennes (Plan National de Développement 2021-2025) et les stratégies africaines de transition numérique. De nombreux projets pilotes ont vu le jour sur le continent : feux intelligents à Nairobi (Kenya), régulation par caméra et IA à Kigali (Rwanda), corridor vert à Johannesburg (Afrique du Sud). Cependant, très peu d'études ont été menées en Côte d'Ivoire sur la faisabilité d'un **pilotage centralisé** des feux de circulation via des technologies logicielles ouvertes comme le SDN.

Le SDN constitue une rupture dans l'architecture réseau traditionnelle en dissociant le plan de contrôle (décision) du plan de données (transfert). Dans le cadre de la gestion de la circulation, cette approche permettrait de centraliser les décisions de changement de feux, tout en collectant en temps réel les données issues des capteurs routiers (caméras, détecteurs à boucle, radars, GPS des véhicules). Contrairement à une architecture distribuée où chaque feu prend ses propres décisions (souvent sans coordination), le SDN offre une vue globale du réseau de circulation, permettant ainsi de mettre en œuvre des stratégies de régulation coordonnées, telles que les « vagues vertes » ou la priorité aux véhicules d'urgence.

Ce choix technologique s'inscrit également dans une logique de **flexibilité, d'évolutivité et d'optimisation des ressources**. Il est possible de modifier les algorithmes de gestion sans changer les équipements physiques ; le système est programmable, interopérable et peut être mis à l'échelle selon les besoins. En intégrant des outils open source comme SUMO

(simulateur de trafic), Mininet-WiFi (émulateur de réseau SDN) et Ryu (contrôleur SDN), cette solution devient non seulement théoriquement accessible, mais aussi **réalisable avec des moyens raisonnables dans un contexte africain**.

3. Objectifs du mémoire

Ce mémoire vise à évaluer la faisabilité et l'efficacité d'une solution de **gestion intelligente des feux tricolores connectés** reposant sur une architecture SDN dans un contexte urbain africain. Il s'agit d'un travail de **recherche appliquée**, combinant modélisation, simulation et analyse critique.

L'objectif général peut être formulé comme suit :

Proposer et évaluer une architecture de régulation du trafic urbain basée sur des feux tricolores connectés pilotés par un contrôleur SDN, dans le but d'améliorer la fluidité de la circulation et de réduire les embouteillages.

De cet objectif général découlent plusieurs objectifs spécifiques :

- a. Analyser les limites des systèmes de feux tricolores classiques dans les villes africaines.
- b. Étudier l'apport du SDN, de l'IoT et du V2X dans la régulation du trafic.
- c. Concevoir une architecture technique intégrant un contrôleur SDN, des capteurs IoT et des feux connectés.
- d. Implémenter cette architecture dans un environnement simulé à l'aide de SUMO, Mininet-WiFi et Ryu.
- e. Évaluer les performances du système proposé selon des métriques clés : débit, temps de parcours, longueur des files, latence de décision.
- f. Comparer ces performances à celles d'un système traditionnel à cycle fixe.

4. Méthodologie

La méthodologie adoptée repose sur une **approche expérimentale par simulation**. Le projet combine des aspects d'ingénierie des réseaux, de modélisation du trafic urbain, et de développement logiciel. La démarche suivie peut être résumée en cinq étapes :

a. Étude documentaire

Une revue systématique de la littérature est menée pour recenser les travaux sur les ITS (Intelligent Transport Systems), les feux adaptatifs, les architectures SDN dans les VANETs (Vehicular Ad hoc Networks) et les expérimentations menées en Afrique. Des sources scientifiques (IEEE, Springer, ScienceDirect) et institutionnelles (UEMOA, Banque mondiale, ONU-Habitat) sont mobilisées.

b. Conception de l'architecture

Une architecture centralisée est définie, comportant :

- des feux connectés simulés comme objets IoT,
- des capteurs virtuels de densité de trafic,
- un contrôleur SDN (Ryu) assurant la qualité de service des communications de contrôle,
- un réseau simulé avec Mininet-WiFi,
- une couche de visualisation du trafic avec SUMO.

c. Implémentation technique

Des scripts Python sont développés pour relier SUMO (via TraCI) et Ryu. À chaque étape temporelle, les données de trafic sont collectées, analysées, et les phases des feux sont pilotées par TraCI, tandis que les règles OpenFlow de priorisation du trafic de contrôle sont ajustées dynamiquement. Des tests unitaires sont réalisés pour vérifier la cohérence des échanges.

d. Scénarios de simulation

Le scénario retenu est un carrefour à profil asymétrique, 1 300 véhicules répartis à 65 % sur l'axe Nord-Sud et 35 % sur l'axe Est-Ouest, rejoué sur trente graines aléatoires indépendantes. Pour chaque scénario, on compare :

- un système à **cycle fixe classique** (référence),
- un système à **régulation SDN dynamique**.

e. Évaluation

Les performances sont mesurées à travers des indicateurs clés :

- **taux de complétion des trajets, soit le nombre de véhicules écoulés rapporté au nombre de véhicules injectés,**
- **temps moyen de parcours et temps perdu moyen par véhicule,**
- **gigue et taux de perte du flux de contrôle sur le réseau SDN,**
- **nombre de commutations de phase effectuées par l'orchestrateur.**

Les résultats sont présentés sous forme de **graphes, tableaux comparatifs** et courbes de performance, analysés en termes de gains, limites, faisabilité et perspectives.

5. Plan du document

Le mémoire est structuré selon une progression logique en cinq grandes sections :

- **Introduction** : elle pose le cadre du problème, la justification du sujet, les objectifs poursuivis et la méthodologie retenue.
- **Chapitre 1** : Généralités. Ce chapitre présente les bases du trafic urbain, le fonctionnement des feux tricolores classiques, et introduit les technologies clés : Internet des objets appliqué à la mobilité, communications V2X, principes du SDN et son intérêt dans un contexte urbain intelligent.
- **Chapitre 2** : État de l'art. Il explore les travaux scientifiques sur les ITS, les systèmes de feux tricolores intelligents, l'intégration du SDN dans les réseaux véhiculaires, et présente une revue critique des approches existantes avec un tableau comparatif de leurs avantages/inconvénients.
- **Chapitre 3** : Contribution (Conception, Implémentation et Évaluation des Performances). Ce chapitre détaille l'architecture proposée, le rôle du contrôleur SDN (Ryu) et les algorithmes d'ajustement des feux au sein de l'environnement SUMO/Mininet-WiFi. Après avoir défini les scénarios et les paramètres expérimentaux, les données issues de la simulation (débit, latence, files d'attente, temps de trajet) sont analysées. Une étude comparative avec un système classique permet enfin d'apprécier les performances et les limites de la solution.
- **Conclusion Générale** : elle résume les apports du travail, discute des limites rencontrées et propose des pistes de recherche futures.

CHAPITRE 1 : GÉNÉRALITÉS SUR LES SYSTÈMES DE GESTION DU TRAFIC ET SDN

1. Généralités sur les systèmes de gestion du trafic

Les systèmes de gestion du trafic visent à adapter la commande des feux à l'état du réseau afin d'améliorer la fluidité et la sécurité des déplacements [5]. Ils s'appuient classiquement sur des dispositifs de signalisation, des capteurs et des algorithmes de contrôle. Cette section décrit les feux tricolores classiques (fonctionnement et composants) et leur évolution vers des versions connectées. Elle présente également les systèmes de régulation des embouteillages et le concept plus large de Systèmes de Transport Intelligents (ITS), intégrant notamment l'IoT, le V2X et l'intelligence artificielle.

1.1 Feux tricolores classiques

Les dispositifs de signalisation lumineuse ont évolué de commandes manuelles vers des systèmes électriques, puis électroniques et connectés ; McShane retrace l'origine de ces dispositifs et leur diffusion internationale [6]. Cette évolution a progressivement stabilisé une grammaire commune fondée sur des signaux lumineux différenciés. La Convention de Vienne sur la signalisation routière, adoptée en 1968, fournit le cadre international de référence pour les couleurs, la disposition et la signification des signaux lumineux [7], comme l'illustrent les figures 1.1 et 1.2.



Figure 1.1 : Illustration historique des premiers feux de signalisation

1.1.1 Histoire

La signalisation lumineuse moderne repose généralement sur trois états : le rouge, le jaune et le vert. Leur ordre, leur présentation et leur signification sont encadrés par la Convention de Vienne [7]. Les systèmes contemporains complètent ce principe par des commandes programmées ou adaptatives, alimentées par des données de trafic recueillies en temps réel [5] ; l'ouvrage de référence en ingénierie du trafic de Roess, Prassas et McShane en documente les modes de fonctionnement, temporel fixe et actionné [8]. L'emploi de sources lumineuses à LED et de signaux spécifiques pour les piétons ou certains mouvements améliore en outre la lisibilité et l'exploitation des équipements.



Figure 1.2 : Feux à électrolampe à trois étapes

1.1.2 Fonctionnement

Les feux tricolores classiques obéissent à des séquences préprogrammées ou adaptatives, selon deux modes principaux : **mode temporel** (horloge fixe) et **mode actionné** (adaptatif)[9]. En mode temporel fixe, chaque carrefour suit un cycle préalablement défini (séquence de durées verte/jaune/rouge pour chaque axe), souvent adapté aux variations moyennes du trafic au cours de la journée (plans de feux programmés selon des plages horaires). En mode actionné (ou semi-actionné), des capteurs détectent la présence ou le volume de véhicules sur chaque voie pour ajuster dynamiquement la durée des feux verts. Les algorithmes adaptent alors les phases lumineuses en fonction du trafic réel en entrée du carrefour[5].

Dans la signalisation routière normalisée, le rouge interdit le passage, le vert l'autorise sous réserve que la voie soit libre, et le jaune annonce l'arrêt, sauf lorsque celui-ci ne peut plus être effectué en sécurité. Des flèches et des signaux dédiés peuvent compléter ces indications pour certains mouvements, les piétons, les cyclistes ou les transports collectifs [7].

Les carrefours peuvent être *coordonnés* en réseau : plusieurs feux interconnectés suivent une logique de synchronisation (corridor de boulevards, par exemple) pour faciliter des passages successifs au vert, ou au contraire réguler globalement le flux. Des plans de circulation calculent alors la grille d'écoulement maximale ou prioritaire selon des critères de trafic global ou local. Les systèmes adaptatifs modernes (SCATS, SCOOT, etc.) constituent d'ailleurs des super-systèmes de régulation qui recalculent en continu les durées et ordonnancement des feux pour optimiser des indicateurs (temps de trajet, longueur des queues, etc.).

1.1.3 Composants

Un système de feux tricolores « classique » comprend les principaux éléments suivants comme illustré dans la figure 1.3:

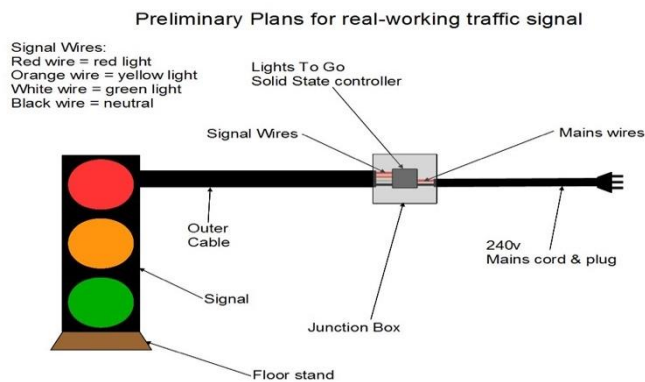


Figure 1.3 : Schéma de composant d'un feu

- **Modules d'éclairage** : chaque carrefour comporte des lanternes dont les unités lumineuses affichent les indications rouges, jaunes et vertes prévues par la réglementation. Les équipements contemporains utilisent couramment des sources à LED et peuvent intégrer des signaux directionnels ou des dispositifs destinés aux piétons. Ces modules sont reliés à un contrôleur qui exécute les plans de feux et, dans les systèmes adaptatifs, exploite les mesures issues des détecteurs [5].
- **Contrôleur de carrefour** : il s'agit de l'armoire électronique (ou « coffret de contrôle ») local à un carrefour. Ce contrôleur programme et exécute le plan de feux, en recevant des entrées (capteurs) et en pilotant les sorties (lampes). Il gère notamment les cycles, les séquences temporisées ou actuées, et peut communiquer avec un centre de supervision. Par exemple, on distingue souvent un **contrôleur de zone ou régional** (pilotant plusieurs carrefours) et des **contrôleurs locaux** (un par carrefour), organisés

de manière hiérarchique[5]. Le contrôleur local prend ses décisions sur les durées de vert en fonction des informations du trafic et de ses voisins, tandis qu'un niveau régional peut ajuster globalement les plans de feux.

- **Capteurs de trafic** : ils fournissent les données en entrée au contrôleur. Les plus courants sont les *boucles électromagnétiques* (inductives) enfouies dans la chaussée en amont du feu (environ 15-20 m) [5]. Lorsqu'un véhicule métallique passe dessus, la boucle détecte sa présence et envoie un signal au contrôleur. Ces boucles peuvent compter les véhicules, mesurer les temps de passage et détecter l'accumulation de files. On trouve aussi des capteurs en bordure (boucles de détection de vélos, radar), ou des caméras vidéo traitées par logiciel pour estimer le trafic. Dans certains cas, les feux sont *actionnés* par un simple bouton-poussoir piéton, ou encore par une communication directe (sur véhicule prioritaire via V2I).
- **Systèmes de communication** : dans les réseaux de feux classiques isolés, la communication peut être minimale (le contrôleur peut être autonome). En revanche, dans un réseau coordonné, on câblait historiquement par paire torsadée ou fibre optique les armoires entre elles et avec un centre de pilotage urbain. De nos jours, la télégestion des feux (communication vers un PC central) se fait fréquemment par réseau filaire (fibre) ou radio (pour petites installations). Ces liaisons permettent de reconfigurer à distance les plans de feux, de centraliser la supervision, ou de transmettre des alertes (maintenance, pannes).
- **Autres dispositifs** : on peut mentionner enfin les panneaux à messages variables (PMV), qui complètent la signalisation lumineuse sur l'autoroute ou dans la ville pour informer en temps réel. Les systèmes de gestion de trafic utilisent aussi des radars pour mesurer la vitesse, des capteurs environnementaux (qualité de l'air) qui n'interviennent pas directement sur la signalisation, mais peuvent orienter la régulation globale du trafic.

Ainsi, un carrefour tricolore classique assemble matériel (feux, capteurs, câblage) et logiciel (contrôleur programmable). Le fonctionnement en mode adaptatif nécessite des boucles inductives et un algorithme de contrôle (logique embarquée) permettant de rallonger ou raccourcir le feu vert en fonction du trafic[5][9]. Par exemple, l'INRETS signale que « les capteurs sont des boucles magnétiques localisées en amont du carrefour à une distance de 15

à 20 m. En plus de fournir le flux de véhicules, ils permettent de détecter les remontées de files d'attente en cas de congestion ». Ce contrôle local reste néanmoins limité par l'horizon local des données.

1.2 Feux tricolores connectés

Les *feux tricolores connectés* constituent la génération suivante de signalisation lumineuse, tirant parti des technologies de l'Internet des objets (IoT) et de la communication véhiculaire. L'idée est de doter chaque feu de capacités de communication (filaire ou sans fil) pour échanger des données en temps réel avec d'autres feux, véhicules ou centres de gestion. Ils ne se contentent plus de réagir individuellement au trafic local : ils peuvent coordonner leurs actions entre eux et avec des véhicules équipés (V2I/V2X), permettant une gestion dynamique et optimisée du trafic.

1.2.1 Histoire

Les premiers jalons des feux « intelligents » apparaissent au début du XXI^e siècle avec l'arrivée des technologies embarquées et des communications automobiles. Des expérimentations ont lieu un peu partout : par exemple, en 2018 à Toulouse et Nîmes, on teste des intersections pilotées par algorithmes intelligents connectées sur un réseau IP (souvent fibre)[10][11]. En Côte d'Ivoire, même la simple introduction de feux tricolores en 2025 (à Grand-Lahou) a été relayée par la presse locale, soulignant l'espoir qu'ils contribuent à « réduire les risques d'accidents et à protéger les piétons »[12]. Bien que ces premiers feux ivoiriens soient basiques, ils ouvrent la voie à de futurs feux « connected ». Dans les métropoles **européennes**, on déploie depuis les années 2010 des infrastructures routières communicantes : les feux sont reliés par fibre optique à un centre urbain (voir l'exemple de Nîmes Métropole)[11], ou utilisent la 5G pour des cas d'usage avancés (priorité bus, voies de covoiturage, etc.).

Aux États-Unis et au Japon, des projets publics et privés enrichissent les feux de capteurs supplémentaires (vidéo, LiDAR) et d'unités de communication V2X. Dans certains cas, des applications pour smartphone ou embarquées permettent même de visualiser les compteurs de feux pour adapter son allure. La diffusion des voitures connectées (en 5G ou ITS-G5) fait que le feu lui-même peut recevoir directement des signaux du véhicule ou de l'infrastructure urbaine globale, participant à un écosystème de trafic intelligent.

1.2.2 Fonctionnement

Concrètement, un feu connecté fonctionne comme un nœud IoT dans le réseau routier. Il conserve les composants classiques (lampe LED tri-éclairage, contrôleur local, capteurs inductifs ou caméras) mais y ajoute une interface de communication bidirectionnelle (Ethernet, 4G/5G, ou protocole véhicule-à-infrastructure V2I). Cette connexion le relie à un **cloud de gestion du trafic** ou à un **contrôleur SDN** central (voir partie 2) qui agrège les données de tous les carrefours. Le feu envoie en continu des mesures de trafic (volume, vitesse, météo locale) et reçoit du centre des recommandations ou des plans de feux à appliquer.

Le mode de pilotage devient ainsi **centralisé et intelligent**. Par exemple, si un embouteillage est détecté en amont, le système peut rallonger la prochaine phase verte pour désengorger la zone. Au contraire, en cas de routes vides, les feux peuvent rester verts plus longtemps pour les premiers arrivants. Cette logique adaptative globale améliore l'efficacité du réseau, à la différence du réglage strictement local des feux traditionnels.

Un autre aspect fondamental est l'interaction *V2X* (Vehicle-to-Everything). Les véhicules connectés peuvent communiquer avec les feux (V2I). Par exemple, un bus de transport en commun muni d'un transpondeur demandera la priorité : en approchant, il envoie sa position au feu, qui passe alors au vert dès qu'il arrive (préemption prioritaire)[13][14]. Les véhicules d'urgence (ambulances, pompiers) peuvent de même déclencher leur feu via V2I pour obtenir un couloir dégagé sur plusieurs intersections en séquence. Inversement, le feu peut diffuser aux voitures à proximité (V2V ou V2X) des alertes : signaler des travaux imminents, un accident plus loin, ou afficher le timer restant avant le prochain passage au vert pour permettre aux automobilistes de ralentir en sécurité. Le protocole standard IEEE 802.11p (ITS-G5) ou la 5G C-V2X sont typiquement utilisés pour ces échanges car fiables et temps-réel.

L'intelligence artificielle (IA) vient compléter ce schéma. Des algorithmes de machine learning prédictifs, exécutés dans le contrôleur central ou même localement sur des *edge devices*, peuvent anticiper les flux de trafic en fonction d'historiques et de données en temps réel (trafic journalier, événements spéciaux, conditions météo, etc.). Par exemple, un réseau neuronal peut apprendre que chaque vendredi à 17 h une vague de voitures arrive par l'autoroute et déclenche automatiquement un plan de feux spécifique pour fluidifier cet afflux. L'IA peut aussi aider à détecter automatiquement les piétons ou cyclistes à la caméra

et adapter la signalisation locale sans action humaine. Les feux connectés deviennent ainsi des *capteurs et actionneurs intelligents*, pilotés par un système global adaptatif.

1.2.3 Composants

Le feu tricolore connecté garde bien sûr les éléments des feux classiques (lanterne tricolore à LED, contrôleur, capteurs) mais s'enrichit de nouveaux modules :

- **Module de communication IoT** : chaque feu intègre un module de communication (Ethernet sur fibre optique, radio 4G/5G, ou même LoRa/Wi-Fi selon les choix d'opérateur) qui le relie au réseau de la ville ou au contrôleur central. Dans l'exemple de Nîmes, la documentation montre que « la gestion des feux tricolores est centralisée... sur un réseau de fibre optique couvrant toute la ville »[11]. Ce réseau IP permet au feu d'envoyer et recevoir des messages, comme un « flux vert » depuis le centre ou des alertes aux usagers.
- **Capteurs enrichis** : au-delà des simples boucles inductives en bordure, on installe souvent des caméras orientées sur les voies (détection d'occupants, véhicules stoppés), des radars Doppler (mesure de vitesse), des capteurs de qualité de l'air, voire des microphones (pour capter sirènes ou klaxons). Tous ces capteurs forment un « Internet des Objets » routier, avec passerelles locales. Par exemple, un feu connecté peut mesurer en continu le nombre de véhicules immatriculés passant chaque minute (vidéo, IA).
- **Interfaces V2X** : certains feux embarquent une unité spéciale V2X (souvent appelée UBR pour Unités Bord de Route), comme illustré dans des systèmes de priorité pour bus. Cette unité reçoit les messages des véhicules (par exemple leur géolocalisation GPS) et peut calculer l'angle d'arrivée et le temps d'approche pour demander un changement de feu opportun[13]. La communication est bidirectionnelle : le feu peut aussi envoyer des infos au véhicule (par exemple la durée de la prochaine phase verte).
- **Applications logicielles** : en interne, le contrôleur embarque des logiciels de gestion basés sur de l'IA ou des algorithmes d'optimisation. À l'échelle de la ville, on utilise des plateformes de *gestion de trafic urbain* sur serveur (ex. SCATS, MovA couplées à IoT, ou des solutions sur OpenStreetMap), qui calculent en permanence les stratégies de circulation. Un cas concret est la préemption de signaux d'urgence contrôlée par

SDN, où une application logicielle dédiée sur le contrôleur central déclenche, en parallèle, la prolongation verte pour un véhicule prioritaire.

- **Énergie et fiabilité** : certains feux connectés sont équipés de panneaux solaires et batteries intégrées pour fonctionner de manière autonome en énergie, et même pour assurer un back-up en cas de coupure de courant. D'autres intègrent des fonctions de diagnostic et d'autoréparation, capables d'alerter le centre en cas de défaillance (ampoule grillée, capteur défectueux) via le réseau.

En synthèse, un feu tricolore connecté n'est plus simplement un système isolé de signalisation : c'est un nœud communicant d'un réseau urbain intelligent. Il capte en temps réel des données de trafic, communique avec d'autres feux et véhicules, et applique dynamiquement des stratégies de régulation. Des projets de villes intelligentes citent ainsi les *feux connectés* comme l'un des usages phares de l'IoT urbain[10][11]. Par exemple, la plate-forme d'IoT de Nîmes Métropole décrit explicitement la "gestion centralisée des feux tricolores" sur fibre optique[11], montrant une intégration poussée du réseau pour l'ensemble du domaine routier.

1.3 Systèmes de gestion des embouteillages

Au-delà du simple feu, la **gestion des embouteillages** (ou gestion du trafic à l'échelle d'une ville/région) englobe l'ensemble des dispositifs et des stratégies visant à réguler les flux de véhicules pour minimiser les congestions. Dans un système complet, on agrège les données venant de nombreux capteurs (boucles, caméras, GPS des véhicules, capteurs Bluetooth, etc.) afin de modéliser la situation de trafic en temps réel. Ces données alimentent des algorithmes de simulation et d'optimisation (souvent centralisés), qui recalculent en continu les plans de feux pour tous les carrefours ainsi que des conseils stratégiques (affichage dynamique de déviations, tarification de péage urbain, etc.))[5][14].

Deux grandes familles de systèmes coexistent : les **systèmes à plans fixes coordonnés** et les **systèmes adaptatifs centralisés**. Dans les plans fixes (souvent décrits comme « programme horaire »), on définit un horaire variant selon le jour et l'heure (par exemple, des cycles différents aux heures de pointe, aux heures creuses, ou en cas d'événement exceptionnel). Cette approche est efficace si le trafic est relativement prévisible et stable. En France, beaucoup de grandes villes disposent de centres de gestion du trafic routier (CGTR) ou régulateurs, qui supervisent l'ensemble des carrefours. Historiquement, ce réglage

centralisé repose sur une suite de plans de feux préprogrammés, appelés *phases* ou *plans de circulation*, qui sont sélectionnés en fonction de données agrégées (boucle principale, radar, voire comptages manuels).

Les systèmes adaptatifs plus avancés (par ex. SCATS, SCOOT, MOVA) vont plus loin : ils adaptent en quasi temps réel les plans de feux aux fluctuations de trafic. Par exemple, le système SCATS (Sydney Coordinated Adaptive Traffic System) basé sur des boucles inductives réparties ajuste la durée des feux verts toutes les 5 à 10 secondes en fonction du trafic défilant, en cherchant à minimiser l'attente globale. Selon l'INRETS, tous ces systèmes adaptatifs « ont pour objectif d'optimiser un critère de trafic tel que le temps d'attente global aux carrefours », mais diffèrent dans leurs méthodes de régulation[5][9]. Une classification par « générations » de systèmes adaptatifs montre une progression vers plus de coordination et d'algorithmes plus complexes, mais tous se basent sur le principe de boucles en amont et de communication centralisée pour coordonner les carrefours[5][9].

Sur le plan technique, la gestion des embouteillages peut aussi impliquer des **panneaux à messages variables (PMV)** sur les routes principales ou autoroutes, donnant des consignes aux conducteurs (« détourner par rue X », « stationnement saturé », etc.), et des systèmes de **priorité (SAE)** aux transports en commun. Un autre volet important est la gestion des incidents : en cas d'accident ou de travaux, le système de gestion peut temporairement rebasculer sur un plan d'urgence (feux clignotants, plan d'itinéraires alternatifs) pour limiter l'impact sur la circulation globale.

Dans un tel contexte, la **modélisation du trafic** (flux de véhicules, simulation microscopique ou macroscopique) est indispensable. Les systèmes de gestion des embouteillages utilisent des modèles mathématiques (équations de flux, modèles de réseau routier, réseaux bayésiens, etc.) et des heuristiques pour prévoir l'évolution du trafic à l'horizon court terme et ajuster les commandes. Par exemple, en cas d'arrivée d'un grand événement (match de foot, concert), on paramètre souvent un scénario dédié qui modifie les cycles et priorités (par exemple, étendre la phase verte du sens de sortie de parking). La qualité de gestion se mesure en indicateurs tels que le temps de parcours moyen, le nombre d'arrêts, ou la vitesse moyenne sur un réseau.

1.4 Systèmes de transport intelligents (IoT, V2X, IA)

Les **Systèmes de Transport Intelligents (STI/ITS)** désignent l'intégration des technologies de l'information (TI) dans les transports pour les rendre plus sûrs, plus efficaces et plus durables. Ils combinent capteurs IoT, communication machine-to-machine, V2X (Vehicle-to-Everything), systèmes embarqués et algorithmes avancés (IA, Big Data). Les ITS couvrent ainsi des domaines variés : systèmes de signalisation (feux, panneaux), gestion du trafic, systèmes d'aide à la conduite (ADAS), véhicules autonomes, applications d'info-traffic, etc.

- **IoT (Internet des objets)** : dans le transport urbain, l'IoT se manifeste par la prolifération de capteurs connectés sur l'infrastructure routière. Les boucles inductives, compteurs pneumatiques, caméras de vidéosurveillance et capteurs environnementaux ne sont plus isolés ; ils alimentent un réseau de données global. Par exemple, certains projets de villes intelligentes déploient des milliers de capteurs (capteurs de pollution, stationnement connecté, détecteurs d'inondation) interconnectés avec les feux de circulation[10][11]. L'ensemble constitue une toile de données en temps réel sur le trafic, la qualité de l'air, la météo urbaine. Les plateformes Big Data agrègent et analysent ces informations pour la planification urbaine (ex : prédiction des goulots d'étranglement, tarification dynamique de congestion).
- **V2X (Vehicle-to-Everything)** : il s'agit de la communication entre véhicule et tout autre élément (autres véhicules V2V, infrastructure V2I, piétons V2P, réseaux V2N pour réseau). Par exemple, un véhicule échange avec un feu (V2I) pour recevoir l'état du feu à l'avance ou pour demander la priorité. Il peut aussi avertir d'un dérapage sur la route ou communiquer avec un capteur au sol. Ces échanges se font par des standards dédiés (802.11p / ITS-G5 ou C-V2X en 5G). Le V2X va transformer la signalisation : le feu peut envoyer son « bruit » directement au tableau de bord du véhicule (compte-à-rebours en direct), et les véhicules partagent leurs données de capteurs (radars, lidar) avec l'infrastructure. Ceci renforce la sécurité active (alerte de collision imminente, prévention des piétons sur passage). En Afrique, de nombreuses études pointent déjà l'importance du V2X dans les smart cities en développement : par exemple, un rapport du GSMA souligne que « les capteurs IoT couplés au V2X collectent des informations sur la congestion, l'état des routes et le mouvement du trafic » (DSRC et 5G)[15].

- **Intelligence Artificielle (IA)** : les algorithmes d'IA appliqués au trafic peuvent adopter des approches d'apprentissage automatique (Machine Learning) ou de renforcement (Reinforcement Learning) pour contrôler dynamiquement les feux. Des travaux en recherche montrent par exemple l'emploi de *deep learning* pour apprendre les meilleures politiques de commutation des feux en simulant le trafic[14]. L'IA peut aussi aider au traitement des données IoT massives (vision par ordinateur sur caméras pour détecter véhicules, ou réseaux de neurones pour estimer la densité de trafic) et à la prévision du trafic. On parle parfois d'**ITS cognitive**, où le système améliore ses règles au fur et à mesure. Un avantage de l'IA est de pouvoir optimiser plusieurs critères en parallèle (par exemple, minimiser les émissions polluantes tout en maintenant le débit).

En Afrique, le développement des ITS est en phase émergente : plusieurs villes étudient ces technologies pour réduire la congestion croissante. Par exemple, l'adoption de systèmes de gestion du trafic assistée par IA à Harare (Zimbabwe) est rapportée comme visant à « optimiser le flux de trafic et réduire les accidents » en temps réel[16]. De même, des expérimentations nigérianes explorent l'usage des réseaux cellulaires 5G pour le déploiement de capteurs et feux intelligents, dans le but de devenir des « villes intelligentes ». En Côte d'Ivoire, si les feux communs ne datent que de 2025 à Grand-Lahou[12], les autorités évoquent l'importance de projets de “villes intelligentes” pour Abidjan (y compris capteurs de trafic et détections automatiques) dans la stratégie nationale. On voit que l'écosystème de capteurs (météo, stationnement, vitesse), la communication V2X et les algorithmes d'IA s'imbriquent aujourd'hui dans un même paradigme ITS pour construire une gestion du trafic globale et intelligente.

Forts de cette analyse, on constate que le principal obstacle des systèmes classiques réside dans l'absence de plan de contrôle entièrement programmable et centralisé. C'est là qu'intervient la notion de Software-Defined Networking (SDN) qui, dès le niveau de l'architecture réseau, propose de séparer le plan de contrôle du plan de données[17]. La suite du chapitre détaillera cette architecture SDN, ses protocoles, modes de routage, et son application à la gestion du trafic, offrant ainsi une nouvelle perspective pour les feux tricolores connectés et intelligents

2. Généralités sur les SDN

Les *Software-Defined Networks* (SDN) constituent un paradigme moderne de conception de réseaux de communication. L'idée clé du SDN est de découpler les fonctions de **contrôle** (décisions de routage, politique du réseau) des fonctions de **transfert de données** (acheminement effectif des paquets). Au sein d'un réseau SDN, des équipements dits « simples » (switches/routeurs) se contentent de suivre les instructions venant d'un contrôleur centralisé, lequel gère la logique globale du réseau (plans de flux) de manière programmée.

2.1 Architecture SDN (interfaces, plans fonctionnels)

L'architecture SDN standard se présente en plusieurs « plans » logiques distincts, reliés par des interfaces ouvertes comme présenté dans la figure 1.4.

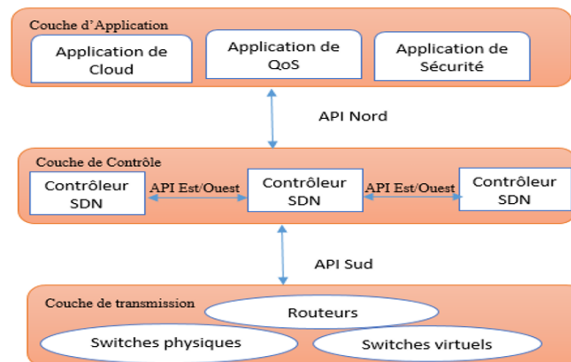


Figure 1.4 : Schéma de l'architecture SDN

- **Plan de données (Data Plane)** : c'est la couche inférieure composée des dispositifs réseau physiques ou virtuels (commutateurs, points d'accès, routeurs). Ces équipements ne décident pas de manière autonome du routage ; ils exécutent les *flows* (règles de trafic) qui leur sont imposées. Chaque commutateur SDN contient une table de flux (flow table) où chaque règle définit comment traiter les paquets (forwarding, drop, modification) en se basant sur des critères (adresses MAC/IP, ports, protocoles, etc.)[18].
- **Plan de contrôle (Control Plane)** : c'est le cœur intelligent du réseau SDN. Un ou plusieurs *contrôleurs SDN* (logiciels tournant sur serveur ou VM) hébergent la logique de décision du réseau (règles de routage, politiques, priorités). Ces contrôleurs communiquent avec les équipements réseau via des protocoles standards (liaisons dite

southbound, voir ci-dessous) pour installer ou mettre à jour les règles de la table de flux. Dans les SDN « classiques », un contrôleur est logiquement centralisé pour l'ensemble du réseau[17], offrant une vision globale (topologie complète, métriques) permettant des décisions optimisées. En pratique, on trouve aussi des architectures distribuées de contrôleurs, mais l'idée est toujours de séparer la gestion globale de la simple transmission locale.

• **Plan d'application (Application Plane)** : au niveau supérieur résident les **applications et services réseaux** qui utilisent le contrôleur comme plateforme. Ce peuvent être des logiciels d'orchestration, de sécurité (firewall virtuel), de surveillance, ou des applications métiers (par exemple, un module de QoS pour flux vidéo). Ces applications interagissent avec le contrôleur via une *interface nord* (northbound API). Elles demandent par exemple à allouer une bande passante, à surveiller l'état du réseau, ou à déployer dynamiquement des règles de flux. Les données collectées (métriques, événements réseau) sont retournées du contrôleur vers l'application via cette même interface.

La séparation SDN se schématise ainsi : un contrôleur logique unique gère dynamiquement des dizaines ou centaines d'équipements réseau déployés physiquement. Selon BMC, cette architecture « découple le Plan de Contrôle du Plan de Données et centralise logiquement l'intelligence réseau au niveau du contrôleur »[17]. Elle offre une grande agilité : toute modification de topologie ou de politique peut se faire par programmation logicielle (API) sans toucher physiquement chaque périphérique.

Interfaces et API : L'architecture SDN implique deux types d'interfaces principales : *southbound* (entre contrôleur et équipements) et *northbound* (entre applications et contrôleur), comme illustré dans la figure 1.5. Les interfaces sud (SBIs) utilisent des protocoles standards comme **OpenFlow**, **NETCONF**, ou **SNMP/MPLS**[19], via lesquels le contrôleur configure les tables de flux des switches. L'interface nord se matérialise souvent sous forme d'API REST (HTTP/JSON) exposées par le contrôleur, permettant aux applications de piloter le réseau. Ce modèle API ouvert autorise l'interopérabilité (« vendor-neutral ») et la création d'applications complexes de gestion. En SDN, l'échange de données (paquets circulant) reste confiné au plan de données : ce sont les commandes et états de la logique qui transitent via les interfaces North/South.

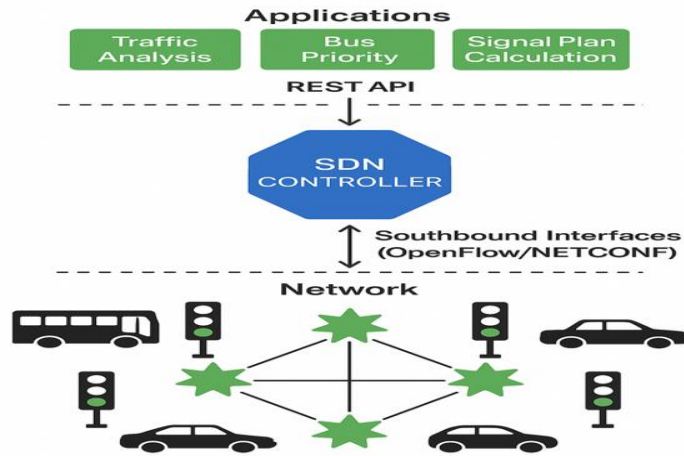


Figure 1.5 : Architecture générale d'un réseau SDN pour la gestion de trafic urbain

Chaque commutateur réseau (étoiles vertes) au niveau des capteurs/feux transmet les paquets au plan de données. Le contrôleur SDN central (octogone bleu) orchestre le réseau via les interfaces sud (OpenFlow/NETCONF) et expose des APIs REST aux applications trafic (applications vertes en haut). Le contrôleur dispose d'une vue globale pour optimiser en temps réel le routage et la priorité des flux. Chaque application (module d'analyse du trafic, priorité bus, calcul de plan de feux) communique avec le contrôleur au travers de l'API nord.

2.2 Protocoles de communication SDN (OpenFlow, NETCONF, REST)

Dans le contexte SDN, plusieurs protocoles standardisés facilitent la communication entre le contrôleur et les composants réseau. Les plus importants sont :

- **OpenFlow** : c'est le protocole sud (southbound) le plus connu et répandu. Initialement développé par l'Open Networking Foundation (ONF), OpenFlow « sépare le plan de contrôle (décisions) du plan de forwarding (transfert) »[18]. Concrètement, le contrôleur SDN ouvre une connexion sécurisée (TCP/TLS) vers chaque commutateur compatible OpenFlow. Il transmet alors les règles de flux (flow rules) au switch via le port standard (6653 par défaut). Chaque règle correspond à une ligne de la table de flux, définissant comment traiter les paquets matching (adressage MAC/IP, ports, etc.). Les règles peuvent être installées de manière **proactive** (le contrôleur pré-charge toutes les tables de flux nécessaires) ou **réactive** (le switch envoie au contrôleur les paquets sans correspondance pour qu'il décide et renvoie la règle) [14]. OpenFlow autorise ainsi un pilotage granulaire du routage. Grâce à ce protocole, un commutateur SDN peut réaliser des fonctions avancées (firewall, NAT, monitoring) par simple changement de règles sans modification matérielle.

- **NETCONF** : protocole IETF orienté gestion de la configuration des équipements réseau. Bien qu'initialement générique, NETCONF est souvent utilisé en SDN pour configurer de manière fine les équipements (notamment pour installer des réseaux overlay ou configurer des capteurs IP). Il utilise le format YANG pour les données et s'appuie sur SSH pour la sécurité. Un contrôleur OpenDaylight ou ONOS peut fonctionner comme client/serveur NETCONF, permettant de pousser ou récupérer des configurations détaillées (comme configurer un tunnel VXLAN ou modifier une ACL)[19]. NETCONF complète OpenFlow en couvrant les aspects de configuration globale et non seulement de routage statique.
- **REST (APIs Northbound)** : le contrôleur expose généralement une **API REST** (base HTTP/JSON) permettant aux applications ou aux systèmes externes de lire l'état du réseau et d'émettre des commandes (par exemple, demander un changement de chemin). Cette interface nord (northbound interface) est très répandue car simple et compatible web. Par exemple, on peut programmer un tableau de bord ou une app mobile pour interroger en JSON l'état de charge d'un commutateur ou pour lancer dynamiquement l'installation d'un flux prioritaire (via une requête POST sur l'API REST du contrôleur). Beaucoup de plateformes SDN (OpenDaylight, ONOS, Cisco APIC) offrent nativement de telles API pour faciliter le développement d'applications intelligentes[19]. L'utilisation d'une API REST standardisée signifie que des tiers peuvent développer des modules analytiques ou d'orchestration sans toucher au code bas niveau du contrôleur.

Outre ces grands protocoles, on peut trouver d'autres moyens de communication en SDN : par exemple **SNMP** (bien que traditionnel, il peut être utilisé pour la supervision ou le contrôle basique), **gRPC** (dans certaines implémentations modernes), ou **XMPP** pour des communications événementielles. Toutefois, OpenFlow, NETCONF et REST restent les plus emblématiques du SDN. TechTarget précise ainsi que l'interface nord d'un contrôleur est « souvent une API REST » normalisée[19], tandis que l'interface sud comprend « OpenFlow et OSPF, ainsi que NETCONF et IS-IS » parmi d'autres. En résumé, l'**interopérabilité** et l'**ouverture** sont au cœur du SDN : le réseau se programme par des protocoles ouverts, facilitant la communication entre équipements multimarques via le plan de contrôle unique[17].

2.3 Protocoles de routage SDN (proactif, réactif, hybride)

En SDN, la notion de « routage » au sens classique (comme OSPF/IS-IS) est déplacée vers le concept de « rules » ou de *flows*. Néanmoins, on qualifie souvent deux grandes stratégies d'installation des règles dans les switchs SDN, analogues aux stratégies de routage dans les réseaux ad hoc ou VANET : mode **proactif** et mode **réactif**.

- **Mode proactif** : le contrôleur SDN installe à l'avance dans tous les commutateurs les règles de flux nécessaires pour gérer le trafic attendu. Par exemple, il peut envoyer en amont un ensemble complet de tables de routage basées sur la topologie du réseau. Dans ce mode, les flux sont préparés avant même l'arrivée des premiers paquets, ce qui assure une commutation immédiate (pas de latence de requête). En contrepartie, cela peut générer une surcharge mémoire si le nombre de flux est très grand, et moins d'agilité si la topologie change rapidement.
- **Mode réactif** : ici, le contrôleur attend que les switchs lui signalent les paquets inconnus. Lorsqu'un paquet arrive sans correspondance dans la table, le switch envoie une requête (« Packet-In ») au contrôleur. Celui-ci calcule alors la règle appropriée et la renvoie au switch, qui l'installe. Cette approche réduit l'empreinte mémoire sur les commutateurs (seules les règles nécessaires au fur et à mesure sont chargées), mais induit un délai supplémentaire à la première occurrence d'un flux.
- **Mode hybride** : il combine les deux approches. Par exemple, on peut installer proactivement des règles pour les flux critiques (trafic de commande, chemins couramment utilisés) et laisser les flux moins fréquents gérer en mode réactif. Le contrôleur peut aussi décider dynamiquement d'une stratégie ou d'une autre selon la situation (par exemple, lancer un mode proactif en début de journée, puis basculer en réactif quand le trafic devient plus aléatoire).

Ces notions sont bien décrites dans la littérature sur les réseaux de véhicules (VANET), mais elles s'appliquent aussi en SDN : « Les tables de flux peuvent être livrées de manière proactive ou réactive. En mode proactif, le contrôleur envoie la table de flux à tous les commutateurs. En mode réactif, il n'envoie que de nouvelles règles quand un commutateur en fait la requête »[18]. L'étude de Rajan et al. (2025) cite en outre la définition classique des routages proactive/réactive/hybride dans le contexte VANET, soulignant les compromis respectifs en termes de robustesse et de coût de découverte[14]. En pratique, un bon

fonctionnement SDN combine souvent ces modes : un contrôleur peut proactivement gérer le routage de base, et utiliser le mode réactif pour des cas exceptionnels (nouveaux flux imprévus) ou pour la redondance après panne.

Il convient aussi de noter que, en SDN, le *routage* peut être défini par des politiques complexes (QoS, priorités, sécurité) gérées au niveau du contrôleur. Ainsi, lorsqu'on parle de protocole de « routage SDN », on entend souvent la gestion des flows plutôt qu'un protocole de routage distribué classique. Par exemple, les solutions SDN intègrent parfois un élément de routage (pour le forward préalable des premiers paquets), mais la logique décisionnelle reste implémentée dans le contrôleur central, distincte du routeur classique. Les algorithmes de routage dans un SDN sont alors programmables et peuvent répondre à des critères multiples (minimisation de latence, équilibrage de charge, priorité par type de trafic).

2.4 Application du SDN à la gestion du trafic

L'intégration du SDN dans la gestion du trafic routier relève d'un domaine émergent, souvent appelé *SDN-VANET* ou *SDN pour ITS*. Le principe est de traiter l'infrastructure de signalisation et les communications véhiculaires comme un réseau SDN : chaque feu ou capteur routier devient un nœud de data plane, les liens (fibre optique, 5G, DSRC) forment l'infrastructure de transport, et un contrôleur central orchestre l'ensemble. Plusieurs travaux ont exploré ces applications, notamment pour la priorité de trafic et l'optimisation des flux urbains.

2.4.1 Priorité aux véhicules spéciaux (urgence, transports en commun)

Un cas d'usage typique est la **priorisation des véhicules de secours ou des transports en commun**. Dans un contexte SDN, un contrôleur peut recevoir en temps réel la demande de priorité (via V2X) et agir sur plusieurs feux en séquence. Par exemple, Qu et al. ont développé une application SDN pour la gestion des véhicules d'urgence, dite *Emergency Vehicle Signal Preemption*[9]. Leur approche combine deux éléments : le contrôleur SDN commande dynamiquement les feux pour créer une trajectoire dégagée (feux verts consécutifs), et diffuse des avertissements ciblés aux autres véhicules pour qu'ils évitent la trajectoire du véhicule prioritaire. Ceci illustre comment le SDN permet de programmer des réactions synchronisées sur plusieurs intersections, sur base de requêtes localisées (bus ou ambulance).

De même, pour les autobus, l'architecture SDN peut prendre en charge un système de priorité bus (comme PrioV2X) où chaque bus communique sa position au contrôleur via des unités embarquées. Le contrôleur, qui connaît aussi l'état global du trafic, calcule alors le plan de feux optimal pour éviter que le bus ne s'arrête à l'intersection, en ajustant la durée des feux verts locaux. L'avantage du SDN est de pouvoir intégrer ces flux de priorité dans la planification générale du trafic, sans désordonner tout le réseau. On peut ainsi garantir la

priorité sans blocage global, car le contrôleur harmonise la boucle d'asservissement du bus avec les autres flux via des règles programmées. Par exemple, le système décrit par Lacroix City montre comment l'unité de bord de route calcule l'approche et transmet la demande de priorité au contrôleur, qui la traduit en règle de flux réseau (même principe que SDN) pour la signalisation[13].

2.4.2 Optimisation globale du trafic urbain

Au-delà des priorités spécifiques, le SDN permet une **gestion centralisée globale du trafic**. Grâce à la séparation des plans, le contrôleur SDN opère sur des *flows* de données et peut implémenter un véritable réseau virtuel de distribution de priorité. Par exemple, toutes les boucles et caméras routières envoient leurs mesures vers le contrôleur, qui peut alors recalculer régulièrement les meilleurs chemins pour chaque type de trafic. Cela inclut l'optimisation des taux de flux, la détection de congestion en formation, et la mise à jour instantanée des plans de feux pour équilibrer les vitesses.

Comme le montrent Rajan et al., l'intégration du SDN dans un réseau véhiculaire permet d'« optimiser en temps réel » les communications et la qualité de service (QoS)[14]. Le contrôleur détient une vue globale permettant de réagir plus efficacement que de simples contrôleurs locaux. Par exemple, si une vidéo de trafic indiquant un embouteillage est reçue, le contrôleur peut simultanément allonger le vert sur plusieurs axes adjacents (voire reconfigurer des itinéraires conseillés via des panneaux lumineux) pour désamorcer la congestion. Inversement, en observant des régions peu chargées, il peut raccourcir certains feux verts pour économiser de l'énergie et limiter l'attente inutile.

L'application du SDN peut aussi englober des algorithmes de *traffic engineering* spécialisés (au sens télécom) au sein d'un réseau de capteurs. Par exemple, certains prototypes considèrent chaque feu comme un « switch » SDN, et chaque phase verte comme un *canal virtuel*. Le contrôleur alloue alors la largeur de ce canal selon la densité de véhicules (comme en multiplexage de ressources). Dans ce cadre, les travaux universitaires rapportent des résultats encourageants : meilleure répartition de charge et diminution des retards globaux par rapport à des systèmes fixes, grâce à des « algorithmes prioritaires » intégrés au contrôleur.

Enfin, le SDN facilite l'intégration des données IoT dans la régulation. Par exemple, le contrôleur central pourrait recevoir non seulement le flux de données routières, mais aussi des données de stations météo locales (pluie, brouillard), de parc relais urbains, ou de capteurs de pollution (qualité de l'air). Il peut alors ajuster les plans de feux et éventuellement recommander la fermeture de certaines voies (via signalisation) si une crise écologique survient. Cette flexibilité est difficile à obtenir dans les architectures traditionnelles.

2.4.3 Perspectives et exemples concrets

Plusieurs initiatives explorent aujourd'hui l'architecture SDN pour le trafic urbain : réseaux virtuels sur Lille/Paris, prototypes expérimentaux de « SDN pour ITS » aux USA et Europe, ou projets industriels (Huawei, Nokia, etc.) visant à proposer des contrôleurs multi-domaines

pour villes intelligentes. L'un des avantages soulignés est la possibilité de simuler et tester différentes politiques sur le contrôleur avant de les déployer : rappelons que le contrôleur voit l'ensemble du réseau comme un tableau de bord. La centralisation via SDN simplifie aussi la maintenance évolutive : ajouter un nouveau capteur ou changer une stratégie ne nécessite plus de reprogrammer chaque feu, mais seulement de mettre à jour le contrôleur.

En Afrique, bien que les réseaux SDN pour trafic soient encore peu répandus, le potentiel est reconnu. Par exemple, les autorités de Lagos et d'Abidjan étudient l'usage des réseaux programmables pour mieux réguler la montée du trafic urbain. À terme, un SDN intégré permettrait de gérer simultanément la signalisation (feux, PMV), les transports publics (priorités), et même les véhicules autonomes (coordination de leur trafic) de manière harmonisée. Des démonstrateurs de « réseaux de capteurs SDN-compatibles » ont déjà été testés en laboratoire (projets Europe Aid, Afrique du Sud, etc.), généralement dans le cadre de déploiements pilotes.

En résumé, l'usage du SDN pour la gestion du trafic routier connecté ouvre la voie à une nouvelle génération de systèmes de régulation : plus souples, adaptatifs et interopérables. La littérature scientifique converge vers l'idée que **la séparation des plans de contrôle et de données SDN offre une vue d'ensemble du réseau et permet d'ajuster intelligemment le trafic** [14][9].

L'étude de Rajan et al. conclut par exemple que l'intégration SDN dans les réseaux véhicules améliore la QoS et « des solutions évolutives pour des réseaux véhiculaires dynamiques »[14]. En complément, l'application du SDN aux feux tricolores connectés permet d'implémenter directement des fonctions avancées (priorités dynamiques, coordination en virtualisation, etc.) autrefois difficiles avec les systèmes locaux.

Ainsi, la perspective est que la gestion du trafic de demain se conçoive comme un réseau SDN urbain : où chaque feu, chaque capteur et chaque véhicule sont des nœuds ou hôtes connectés, et où un ou plusieurs contrôleurs programmables optimisent en continu les flux. Cette architecture servira de base au système proposé dans la suite de ce mémoire, où les **feux tricolores connectés pilotés via SDN** permettront de réguler les embouteillages de façon intelligente et centralisée.

2.5 Synthèse comparative : Systèmes classiques/intelligents vs Approches SDN

Au terme de cette étude de la partie « Gestion du trafic », on constate que les systèmes traditionnels et intelligents reposent sur des architectures distribuées ou hiérarchisées (capteurs locaux, contrôleurs de carrefour, centre de régulation). En comparaison, les approches SDN (décrites en section 2) proposent un paradigme différent, où le réseau de communication est entièrement programmable et centralisé. Le tableau 1.1 suivant compare les deux approches sur plusieurs aspects clés :

Tableau 1.1 : Comparaison des systèmes classiques/intelligents et des approches SDN

Critère	Systèmes classiques/intelligents (IoT/V2X)	Approche SDN
Architecture	Structure en îlots ou hiérarchique : chaque carrefour est géré par son contrôleur local, parfois coordonné par un centre régional. Les échanges entre carrefours sont limités.	Architecture découplée à trois plans (données, contrôle, application) [17]. Un contrôleur SDN global pilote l'ensemble des nœuds du réseau.
Gestion du trafic	Localisée et réactive : chaque feu ou micro-réseau ajuste ses phases sur la base des données qui lui parviennent. Systèmes adaptatifs (SCOOT, SCATS) effectuent un réglage macro/tempo.	Centralisée et proactive : le contrôleur SDN possède une vue globale et peut répartir les ressources (bandes passantes logiques) dynamiquement. Utilisation d'APIs ouvertes.
Communication	Liaisons fixes (câbles, radio) pour relier certains feux au centre. Voitures et feux peuvent échanger via V2I (ITS G5/5G) en point à point.	Toutes les communications passent par le contrôleur SDN. SB (« southbound ») via OpenFlow/ NETCONF vers les commutateurs, NB (« northbound ») via REST pour apps[19].
Adaptabilité	Adaptation basée sur règles définies (horaires, détections) ou algorithmes locaux. IA possible mais complexe à déployer de manière isolée.	Programmabilité totale : on peut reconfigurer à la volée l'ensemble du réseau de capteurs et d'actionneurs. L'IA se déploie globalement via des applications sur le contrôleur.
Échelle et Coordination	Coordination locale (entre carrefours voisins) ou par zones. Difficulté à coordonner tout le réseau en continu (nécessite calculs intensifs).	Coordination réseau entier facilitée : le contrôleur agit sur tous les carrefours simultanément. Exemples d'optimisations multicritères globales plus simples à implémenter.
Exemples d'applications	Systèmes de priorité bus basés sur détection locale; préemption d'urgence via bouton ou capteur embarqué; plans de feux urbains selon horaires.	Prototypes de signal préemption gérés par SDN ; simulation de répartition adaptative de trafic via algorithmes SDN.
Complexité et Coût	Infrastructure connue, peu coûteuse localement (capteurs simples). Mais la multiplication des contrôleurs et câblages rend la maintenance lourde.	Déploiement d'un contrôleur SDN et d'un réseau programmable (nouvelles interfaces) induit un coût initial plus élevé, compensé par une maintenance logicielle simplifiée.
Sécurité/ Résilience	Impact d'une panne localisé (un feu HS est isolé). Les systèmes traditionnels sont moins ciblés par des attaques logiques, mais vulnérables physiquement (vandalisme).	SDN centralisé présente un point d'échec critique (sécurité du contrôleur primordiale). En revanche, la centralisation permet de surveiller et sécuriser le réseau plus finement.

Cette synthèse montre que les approches SDN offrent un contrôle beaucoup plus global, avec une forte programmabilité et interopérabilité (API ouvertes), comparé aux systèmes classiques ou IoT actuels qui restent souvent cloisonnés ou manuels dans leur paramétrage.

CHAPITRE 2 : ÉTAT DE L'ART

Introduction

Le feu tricolore constitue, depuis son invention à la fin du XIXe siècle, l'instrument premier de la régulation du trafic urbain. Simple dans son principe (trois couleurs, trois injonctions), il a longtemps répondu avec une efficacité suffisante aux besoins de villes encore peu motorisées. Mais la révolution automobile du XXe siècle, accélérée par la mondialisation et l'urbanisation des populations, a transformé cet outil d'une autre époque en goulet d'étranglement systémique. En 2017, Jakarta était classée douzième agglomération la plus congestionnée du monde avec 63 heures passées dans la congestion aux heures de pointe, par conducteur et par an ; à Boston, le coût des embouteillages atteignait 5,7 milliards de dollars pour la seule ville [20]. Ces chiffres ne sont pas des anomalies : ils révèlent la défaillance structurelle d'un modèle de signalisation fondé sur des cycles fixes prédéfinis, incapables de s'adapter aux variations dynamiques du trafic réel.

Face à cette inadéquation, la communauté scientifique internationale a déployé depuis deux décennies un effort de recherche considérable, mobilisant successivement l'électronique embarquée, l'Internet des Objets (IoT), la vision par ordinateur, l'apprentissage profond et les nouvelles technologies de communication. Le présent état de l'art propose une lecture organisée de neuf contributions représentatives de cet effort, sélectionnées pour leur diversité d'approche et leur complémentarité thématique. Ces travaux sont regroupés en trois parties qui correspondent à trois grandes familles de solutions : les systèmes matériels et architectures IoT fondés sur des capteurs physiques ; les approches par vision par ordinateur et apprentissage automatique pour la détection et le contrôle adaptatif ; enfin, les revues systématiques et les technologies de communication émergentes qui dessinent les horizons futurs du domaine.

1. Systèmes Matériels et Architectures IoT pour la Gestion du Trafic

Cette première partie regroupe les travaux qui abordent la gestion du trafic sous l'angle du déploiement physique. Ces contributions partagent une même conviction : les feux intelligents ne peuvent exister sans une infrastructure de collecte de données ancrée dans la réalité du terrain. Qu'il s'agisse de capteurs ultrasoniques, d'étiquettes RFID ou de modules de communication véhiculaire, c'est la qualité et la pertinence des données captées qui conditionnent l'efficacité du contrôle. Ces quatre articles illustrent la diversité des solutions envisageables, depuis le prototype frugal conçu pour un contexte à ressources limitées jusqu'à l'architecture sophistiquée intégrant les véhicules eux-mêmes comme nœuds actifs du réseau.

1.1 Un modèle architectural IoT-RFID pour la gestion innovante du trafic (Das, Dash & Mishra, 2018)

Publié [21] dans le cadre d'un ouvrage collectif Springer consacré au Cognitive Computing for Big Data Systems over IoT, le chapitre de Das, Dash et Mishra constitue l'entrée en matière conceptuelle de cette revue. Les auteurs partent d'un diagnostic commun à de nombreux pays émergents : les solutions existantes de gestion du trafic (boucles inductives, capteurs infrarouges, réseaux de capteurs sans fil) sont trop coûteuses à installer et à maintenir pour être généralisées à grande échelle. Leur réponse est un modèle architectural centré sur le RFID (Radio Frequency Identification) (Want, 2006), technologie mature offrant identification à distance sans ligne de visée directe, robustesse aux environnements difficiles et coût marginal faible. Dans leur architecture, chaque véhicule porte une étiquette RFID passive ; des lecteurs aux intersections remontent les données de densité à un serveur centralisé via GSM, qui pilote dynamiquement les durées de vert. L'architecture est celle d'une plateforme unique interconnectant des composants hétérogènes décentralisés, préfigurant les principes de l'IoT à grande échelle dans le contexte des Smart Cities.

La limite principale de ce travail est l'absence de validation empirique : ni prototype, ni simulation, ni métriques de performance. La présupposition d'un parc automobile universellement équipé en tags RFID constitue par ailleurs un obstacle pratique considérable : les véhicules anciens, étrangers ou informels demeurent invisibles. L'apport de cet article est néanmoins réel : il pose un cadre architectural clair, pertinent et adapté aux contraintes des pays à ressources limitées, qui inspire plusieurs des déploiements matériels qui lui succèdent.

1.2 Prototype Arduino avec capteurs ultrasoniques et caméra ESP32 (Ramadhan et al., 2021)

Là où Das et al. [22] dessinent un cadre conceptuel, Ramadhan et ses collaborateurs passent à l'acte. Leur système de contrôle de feux intelligents (STLC) est déployé en conditions réelles à l'intersection de Dor al-Moalemen à Wasit City, en Irak, un pays dont le réseau routier n'a jamais été adapté à l'explosion du parc automobile d'après-2003. L'architecture matérielle est sobre mais fonctionnelle : un Arduino Mega coordonne trois capteurs ultrasoniques HC-SR04, une caméra ESP32-CAM et un servomoteur. Les capteurs ultrasoniques mesurent la présence et le volume de véhicules sur chaque voie ; le système attribue le feu vert à la voie la plus chargée. La fonctionnalité la plus originale est la répression automatique des infractions : quand un capteur détecte le franchissement d'un feu rouge, le servomoteur oriente la caméra vers le véhicule contrevenant, photographie sa plaque et transmet l'image à une base de données via un bot Telegram (Ramadhan et al., 2021).

Ce travail se distingue par son pragmatisme et sa valeur opérationnelle immédiate dans un contexte de ressources limitées. Ses limites sont celles du matériel choisi : précision insuffisante des ultrasoniques pour des situations multi-files, gestion restreinte à trois voies, absence de métriques comparatives formelles. La valeur d'usage réside dans la démonstration que des composants électroniques grand public, correctement assemblés,

peuvent apporter une réponse fonctionnelle à un problème urbain réel, leçon précieuse pour les économies émergentes.

1.3 Système IoT basé sur le traitement d'images et l'apprentissage profond (Fagroud & Benlahmar, 2021)

Fagroud et Benlahmar, [23] du Laboratoire de Technologie de l'Information et Modélisation de l'Université Hassan II de Casablanca, franchissent une étape supplémentaire en substituant aux capteurs physiques une analyse vidéo par vision par ordinateur. Leur système automatisé combine des caméras connectées installées sur chaque feu, une analyse de densité par OpenCV, et un microcontrôleur qui ajuste dynamiquement les durées de vert en fonction du nombre de véhicules détectés dans une zone rectangulaire prédéfinie. L'ensemble est complété par une interface web bi-modale (temps réel / archive). Sur le plan théorique, les auteurs s'appuient sur les fondements du Deep Learning (réseaux CNN pour la reconnaissance spatiale, RNN pour les séquences temporelles) pour asseoir leur approche de détection automatique (Fagroud & Benlahmar, 2021).

L'article souffre d'une limite de fond : il reste à un stade conceptuel, sans validation expérimentale ni métriques de performance. Ni la robustesse face aux conditions nocturnes ou météorologiques, ni la précision de détection ne sont évaluées. Malgré ces réserves, la synergie IoT/vision par ordinateur proposée constitue une orientation architecturale cohérente et innovante, dont la fécondité est confirmée par les travaux de vision par ordinateur plus avancés examinés en Partie 2.

1.4 Architecture ITMS basée sur l'Internet des Véhicules (Mohamed & AlShalfan, 2021)

Mohamed et AlShalfan [24] poussent le paradigme communicant jusqu'à sa logique ultime : non plus des capteurs passifs aux intersections, mais les véhicules eux-mêmes comme sources actives d'information. Leur Intelligent Traffic Management System (ITMS) s'appuie sur les réseaux véhiculaires VANET et l'Internet des Véhicules (IoV) pour établir des communications V2V (véhicule à véhicule) et V2I (véhicule à infrastructure). Un contrôleur STS (Smart Traffic Signal), implémenté à la fois en simulation et en prototype matériel, gère chaque intersection via un algorithme adaptatif entièrement paramétré, optimisant simultanément l'équité entre voies, la réduction du délai moyen, l'amélioration du débit et la priorisation des véhicules d'urgence (Mohamed & AlShalfan, 2021). Dans la vision cible, les feux de couleur classiques pourraient même être supprimés, les véhicules recevant les autorisations de passage directement via IoV.

La double validation, à la fois simulation multi-scénarios et prototype matériel, confère à ce travail une crédibilité technique supérieure à la plupart des articles de la Partie 1. Les résultats montrent une réduction significative du délai moyen et une augmentation du nombre de véhicules servis. Les limites portent sur la cybersécurité des communications V2I (entièrement absente de l'analyse), la robustesse en cas de défaillance réseau, et le coût de déploiement d'une infrastructure IoV à l'échelle d'une ville : autant de questions ouvertes qui dessinent le programme de recherche des années à venir.

1.5 Synthèse 1

Ces quatre contributions [21][22][23][24] illustrent une évolution claire dans les approches matérielles de gestion du trafic : on passe du modèle conceptuel basé sur le RFID (*Das et al., 2018*), au prototype frugal à capteurs ultrasoniques (*Ramadhan et al., 2021*), au système de vision embarquée (*Fagroud & Benlahmar, 2021*), jusqu'à l'architecture V2I communicante (*Mohamed & AlShalfan, 2021*). Cette progression révèle une tendance de fond : la richesse et la pertinence des données disponibles à l'intersection croissent avec la sophistication technologique, mais au prix d'une complexité accrue et de coûts de déploiement plus élevés. L'enjeu commun à ces quatre travaux reste la validation empirique rigoureuse en conditions urbaines réelles, seule capable de trancher entre les promesses architecturales et les performances effectives.

2. Vision par Ordinateur, Apprentissage Automatique et Contrôle Adaptatif

Là où la Partie 1 s'intéressait à la manière de collecter des données sur le trafic, cette deuxième partie examine comment ces données peuvent être interprétées et exploitées pour produire des décisions de contrôle optimales. Les trois travaux ici réunis partagent une même conviction épistémologique : la complexité et la variabilité du trafic réel dépassent les capacités des règles prédéfinies et des heuristiques fixes ; seules des approches capables d'apprendre à partir de données peuvent prétendre à une adaptabilité véritable. C'est cette conviction qui motive le recours successif à la vision par ordinateur, à l'apprentissage supervisé par détection, et enfin à l'apprentissage par renforcement profond.

2.1 Système YOLO+SORT pour le contrôle adaptatif en conditions extrêmes (Sharma et al., 2020)

Sharma et al., de l'Institut Maharaja Agrasen de Technologie (Delhi), [25] s'attaquent à l'une des problématiques les plus exigeantes du domaine : concevoir un système adaptatif fonctionnel dans les conditions chaotiques du trafic indien, caractérisées par une densité extrême, une hétérogénéité des véhicules et un non-respect fréquent des règles. Leur solution repose sur une chaîne algorithmique en deux étapes. YOLO (*You Only Look Once*) (*Redmon & Farhadi, 2017, cités dans Sharma et al., 2020*), appliqué image par image, détecte et localise les véhicules par des boîtes englobantes avec scores de confiance. SORT (*Simple Online and Real-time Tracking*) suit ces objets d'une image à l'autre en associant les détections via leurs coordonnées de position. Une ligne de comptage virtuelle, positionnée entre 100 et 120 mètres avant le feu rouge, enregistre chaque franchissement et alimente un compteur par voie. Les durées de vert sont ensuite optimisées dynamiquement via des algorithmes de planification (*Sharma et al., 2020*).

L'apport le plus significatif de ce travail n'est pas uniquement technique : c'est la démonstration que la solution fonctionne là où les autres ont échoué dans le chaos du trafic indien. La comparaison quantitative entre YOLOv2 (76,8 % mAP à 67 fps) et ses concurrents (Faster R-CNN, SSD, R-CNN) légitime le choix. Le faible coût, sans boucles inductives, ouvre des perspectives de déploiement dans des économies contraintes. Les limites

(dégradation en scènes nocturnes, détections imbriquées, absence de gestion des urgences) dessinent le programme de recherche pour les versions futures.

2.2 IoT, Machine Learning et ville intelligente : une approche intégratrice (Kumari et al., 2020)

Kumari et al., du Chaibasa Engineering College (Jharkhand, Inde)[26], proposent une réflexion plus transversale sur l'articulation entre Machine Learning, IoT et gestion du trafic dans le cadre des Smart Cities. Leur contribution, publiée sur SSRN en juillet 2020, part du constat que les villes contemporaines sont confrontées à un ensemble de problèmes interdépendants (pollution, saturation des ressources, congestion) que seule une approche systémique, fondée sur la collecte et l'analyse de données massives, peut efficacement traiter (Kumari et al., 2020). Leur système de gestion du trafic intelligent exploite des capteurs IoT distribués et des algorithmes de Machine Learning pour calculer la vitesse optimale à laquelle un véhicule doit approcher une intersection, de sorte qu'il passe au vert sans s'arrêter. Le système accède aux données du gestionnaire de trafic en temps réel, calcule la durée restante de la phase courante et recommande une vitesse adaptée, prenant en compte la vitesse actuelle du véhicule, son accélération et la phase du feu.

Les résultats obtenus, à savoir l'amélioration du débit de circulation, la réduction des émissions de gaz et la diminution des temps d'attente, sont encourageants, même si l'article reste limité à un scénario de laboratoire contrôlé. La généralisation à un réseau urbain réel multi-intersections n'est pas traitée. La dépendance au système d'information centralisé du gestionnaire de trafic soulève des questions de disponibilité et de latence. Cela étant, la perspective adoptée, qui consiste à optimiser non plus le signal mais le comportement du conducteur en amont, introduit une dimension comportementale originale, en rupture avec les approches exclusivement infrastructure-centrées des autres travaux de ce corpus.

2.3 Deep Reinforcement Learning multi-agents et validation sur réseau réel (Damadam et al., 2022)

Si les deux articles précédents s'appuient sur des algorithmes d'apprentissage supervisé ou des règles déterministes, Damadam et al. franchissent un saut paradigmatique en recourant au Deep Reinforcement Learning (DRL)[27]. Des systèmes comme SCATS (Sims & Dobinson, 1980) avaient bien apporté une adaptabilité partielle, mais restaient tributaires d'opérateurs humains et de coûts d'infrastructure élevés. L'apprentissage par renforcement profond, dont le potentiel pour des tâches de contrôle complexe a été établi par Mnih et al. (2015), offre une alternative radicalement différente : l'agent apprend, par interactions répétées avec son environnement, une politique de contrôle optimale, sans qu'aucune règle ne lui soit explicitement programmée. Damadam et al. implémentent cette idée dans un cadre multi-agents via l'algorithme MA2C (Multi-Agent Advantage Actor-Critic), où chaque intersection est un agent autonome qui échange ses observations avec ses voisins via IoT. La validation sur les six intersections réelles de Shiraz (Iran), simulées dans SUMO avec des données de trafic authentiques issues de la municipalité, est l'apport distinctif majeur : les réductions de files d'attente et de délais obtenues face au système SCATS en place sont significatives (Damadam et al., 2022).

La complexité computationnelle de l'entraînement DRL, la non-gestion des urgences et des piétons, et la question de la transférabilité du modèle entre villes constituent les limites à adresser. Elles n'entament pas la portée du travail, qui trace la voie vers des ATSC pleinement autonomes capables d'apprendre et de s'adapter sans reconfiguration manuelle.

2.4 Synthèse 2

L'évolution observée au fil de ces trois contributions [25][26][27] est révélatrice d'une maturation algorithmique accélérée. On passe de la détection par vision par ordinateur avec comptage déterministe (*Sharma et al., 2020*), à l'optimisation comportementale par ML (*Kumari et al., 2020*), jusqu'au contrôle optimal par apprentissage par renforcement profond validé en milieu réel (*Damadam et al., 2022*). Cette progression dessine le chemin vers des systèmes capables non seulement de s'adapter au trafic actuel, mais d'anticiper ses dynamiques futures. La convergence IoT/DRL constitue le paradigme émergent le plus prometteur, à condition de résoudre les enjeux computationnels et de cybersécurité qui en conditionnent le déploiement à grande échelle.

3. Revues Systématiques, Technologies Émergentes et Dimensions Transversales

Cette troisième partie rassemble des contributions qui n'entrent pas dans la catégorie des systèmes applicatifs directement déployés, mais qui apportent au corpus une profondeur analytique et prospective indispensable. Une revue systématique de la littérature, une exploration des communications par lumière visible et une analyse pédagogique de la symbolique tricolore constituent trois angles complémentaires qui permettent de prendre du recul sur les développements techniques, d'anticiper les transitions technologiques à venir et de ne pas perdre de vue la dimension humaine de tout système de signalisation.

3.1 Revue systématique des développements récents des feux intelligents (Yusuf et al., 2021)

La revue proposée par Yusuf, Arifin et Zulkifli de l'Universitas Indonesia[28] répond à un besoin clairement identifié dans la littérature : l'absence d'un travail de synthèse couvrant simultanément les dimensions matérielles, algorithmiques et prospectives des feux de circulation intelligents. Plus de 46 études sont analysées et organisées selon une classification triaxiale qui structure de manière élégante un champ de recherche fragmenté : les systèmes orientés vers la réduction de la congestion, ceux conçus pour la gestion des véhicules d'urgence, et ceux dédiés aux piétons (Yusuf et al., 2021). Pour chaque axe, les technologies de détection (caméras, WSN, RFID) et les algorithmes de contrôle (logique floue, algorithmes génétiques, réseaux de neurones, apprentissage par renforcement) sont recensés et comparés.

Le tableau comparatif des algorithmes constitue l'apport méthodologique le plus tangible de l'article : il synthétise en un seul regard les avantages et les limites de chaque famille algorithmique, offrant aux concepteurs de systèmes un outil de décision que peu d'articles fournissent. L'identification des coûts d'infrastructure comme principal frein à l'adoption large des feux intelligents est également un apport pragmatique de premier ordre. Les limites

propres à l'exercice de la revue (absence de résultats expérimentaux originaux, évaluation partielle de la qualité des études incluses, fenêtre temporelle 2015-2020) n'entament pas la valeur d'un travail qui fait utilement office de boussole pour le champ.

3.2 Communications par Lumière Visible pour l'IoT et perspectives V2I (Oyewobi et al., 2022)

L'article [29] d'Oyewobi, Djouani et Kurien constitue le travail le plus prospectif de ce corpus. Partant du constat que la croissance exponentielle des dispositifs IoT sature le spectre des fréquences radioélectriques, les auteurs explorent le potentiel des communications par lumière visible (VLC) comme alternative complémentaire à la 5G pour la connectivité massive de l'IoT. Leurs contributions s'articulent autour d'une architecture réseau VLC-IoT originale, d'une analyse des techniques de modulation (OOK, OFDM, PPM), d'un inventaire des défis (interférences lumineuses ambiantes, mobilité, champ de vision directionnel) et de leurs solutions, et d'une projection vers les directions de recherche futures. Le concept fondateur est celui du LiFi (*Haas et al., 2016*), qui transforme les LED en points d'accès bidirectionnels offrant un spectre non réglementé et une sécurité intrinsèque par confinement de la lumière (*Oyewobi et al., 2022*).

Pour le lecteur de cet état de l'art sur les feux intelligents, la perspective la plus stimulante est celle des communications V2I par LED. Les feux tricolores de prochaine génération pourraient simultanément remplir leur fonction de signalisation et assurer des échanges d'information avec les véhicules équipés de récepteurs photodiodes, sans aucune infrastructure RF additionnelle, une dualité fonctionnelle d'une élégance architecturale certaine. Les limites de la revue (absence de comparaisons quantitatives VLC/RF, coût des récepteurs embarqués non discuté) n'entament pas la valeur prospective d'un article qui ouvre une fenêtre sur l'avenir des infrastructures de transport communicantes.

3.3 La dimension usager : la méthode du feu tricolore en évaluation pédagogique (Stan, 2021)

L'article [30] de Stan, doctorante à l'Université Babeş-Bolyai de Cluj-Napoca (Roumanie), se distingue radicalement des autres contributions de ce corpus par son ancrage dans les sciences de l'éducation plutôt que dans l'ingénierie des transports. Il étudie l'utilisation de la métaphore du feu tricolore (vert pour la compréhension, jaune pour le doute, rouge pour l'incompréhension) comme outil d'auto-évaluation formative en maternelle. Une enquête quantitative auprès de 122 enseignantes traitée par SPSS révèle que 21,3 % d'entre elles mobilisent régulièrement cette méthode, quelle que soit leur ancienneté (Stan, 2021). L'article documente avec précision les multiples modalités d'application : cartes colorées individuelles, autocollants, bacs de rangement chromatiques, gestion comportementale quotidienne.

Dans le cadre de cet état de l'art technique, la contribution de cet article est indirecte mais réelle. Elle rappelle une vérité fondamentale que les ingénieurs oublient parfois : tout système de signalisation, aussi optimisé soit-il algorithmiquement, repose in fine sur une grammaire symbolique partagée entre le système et ses usagers. L'efficacité d'un feu intelligent ne se mesure pas seulement à ses performances computationnelles (délai moyen

réduit, débit amélioré), mais aussi à la lisibilité de ses signaux pour les conducteurs, les piétons et les gestionnaires. La dimension ergonomique et l'acceptabilité sociale constituent des facteurs de succès que ce corpus technique tend à sous-estimer, et que l'article de Stan remet utilement en perspective.

3.4 Synthèse 3

Les trois contributions de cette partie [28][29][30] apportent au corpus des éclairages que les approches applicatives ne peuvent seules fournir. La revue systématique de Yusuf et al. (2021) cartographie le champ et fournit les outils d'analyse comparatifs indispensables. L'article d'Oyewobi et al. (2022) ouvre la perspective d'une technologie de rupture, la VLC, capable de transformer les infrastructures LED existantes en vecteurs de communication sans infrastructure additionnelle. Stan (2021), enfin, rappelle que le feu tricolore est autant un artefact technique qu'un signe culturel partagé, dont l'appropriation par ses usagers conditionne l'efficacité au même titre que la performance algorithmique.

Conclusion et perspectives

Cet état de l'art, conduit à travers dix contributions représentatives publiées entre 2018 et 2022, dessine le paysage d'un domaine en pleine transformation. Trois grandes tendances se dégagent de manière convergente. Premièrement, le passage du capteur au réseau : les systèmes les plus récents ne se contentent plus de mesurer passivement le trafic à l'intersection, ils intègrent les véhicules eux-mêmes, via IoV, VANET ou VLC, comme nœuds actifs d'un réseau distribué d'information. Ce glissement architectural, illustré par les travaux de Mohamed et AlShalfan (2021) et d'Oyewobi et al. (2022), préfigure des intersections sans feux de couleur, où la gestion du trafic serait entièrement négociée par communication directe entre véhicules et infrastructure.

Deuxièmement, le glissement de la règle à l'apprentissage : si les premières générations de feux intelligents substituaient des règles adaptatives aux cycles fixes, les travaux les plus récents, notamment celui de Damadam et al. (2022), franchissent le pas vers des systèmes capables d'apprendre des politiques de contrôle optimales directement à partir de données réelles, sans aucune règle explicitement programmée. Ce paradigme du Deep Reinforcement Learning, validé sur un réseau urbain authentique, constitue probablement l'orientation la plus prometteuse pour la prochaine décennie, à condition de résoudre les défis de complexité computationnelle et de généralisation géographique qu'il soulève.

Troisièmement, la persistance d'un écart entre conception et déploiement : la majorité des travaux de ce corpus, qu'ils relèvent de l'IoT matériel, de la vision par ordinateur ou du DRL, souffrent d'une validation empirique insuffisante. Comme le révèle la revue de Yusuf et al. (2021), le coût des infrastructures, des communications et des mises à jour périodiques reste le principal frein à l'adoption large des feux intelligents. La cybersécurité des communications V2I, la robustesse face aux conditions météorologiques et à la défaillance réseau, et la transférabilité des modèles d'une ville à l'autre constituent les grands défis scientifiques et techniques ouverts que les travaux futurs devront impérativement adresser.

À l'horizon de cette analyse, la vision des villes intelligentes, où chaque feu de signalisation serait à la fois capteur, décideur et émetteur de données, n'est plus du domaine de la

spéculation. Elle exige simplement, pour se concrétiser, l'effort collectif et interdisciplinaire que ce corpus ne fait encore qu'esquisser.

C'est précisément cet effort que le présent travail de recherche entreprend de concrétiser. Fort des enseignements tirés de ce panorama de la littérature, et en réponse aux lacunes identifiées, à savoir une validation empirique insuffisante, l'absence de maîtrise centralisée du réseau et des coûts d'infrastructure prohibitifs, la suite de ce mémoire propose une implémentation expérimentale originale d'un système de gestion des embouteillages à feux tricolores connectés, fondée sur le paradigme du Software-Defined Networking (SDN).

Le choix du SDN comme architecture de contrôle répond directement à la principale faiblesse relevée dans ce corpus : la fragmentation du contrôle. Là où les systèmes IoT distribués examinés dans la Partie 1 souffrent d'un manque de coordination globale, et là où les approches par apprentissage automatique de la Partie 2 peinent à intégrer la dimension réseau de bout en bout, le SDN introduit une rupture architecturale décisive, la séparation entre le plan de contrôle et le plan de données, qui permet à un contrôleur unique, logiquement centralisé, de programmer dynamiquement le comportement de l'ensemble des équipements réseau. Appliqué aux infrastructures véhiculaires, ce paradigme confère aux feux tricolores connectés une intelligence collective qu'aucune approche décentralisée ne peut pleinement atteindre.

L'implémentation s'articule autour d'un environnement technique intégré combinant trois outils complémentaires. SUMO (Simulation of Urban MObility) assure la simulation microscopique du trafic routier sur un carrefour à fort déséquilibre de charge (1 300 véhicules avec un profil asymétrique délibéré), reproduisant les conditions dans lesquelles un contrôle statique s'effondre. Le contrôleur SDN Ryu, compatible OpenFlow 1.3, prend en charge la gestion centralisée des flux réseau et l'activation dynamique de la qualité de service (QoS). Mininet-WiFi, enfin, émule le réseau véhiculaire sans fil avec des commutateurs Open vSwitch programmables, permettant d'éprouver expérimentalement l'avantage de la priorisation SDN sur les communications de contrôle entre feux connectés et infrastructure.

Les résultats expérimentaux obtenus, présentés et analysés dans le chapitre suivant, apportent une réponse quantifiée à deux questions centrales que cet état de l'art laisse ouvertes : d'une part, quelle est la supériorité effective des feux connectés et adaptatifs sur les systèmes à cycles fixes face à une congestion asymétrique réelle ? D'autre part, dans quelle mesure le SDN améliore-t-il concrètement la qualité des communications de contrôle en environnement véhiculaire congestionné ? C'est à ces deux défis, la preuve de l'avantage des feux connectés et la démonstration de l'apport du SDN, que la suite de ce mémoire consacre ses efforts.

CHAPITRE 3 : CONCEPTION, IMPLÉMENTATION ET ÉVALUATION DES PERFORMANCES

1. Introduction

Ce chapitre présente la conception, l'implémentation et l'évaluation expérimentale du système de gestion intelligente des embouteillages à feux tricolores connectés, fondé sur l'architecture SDN (Software-Defined Networking). Il constitue le cœur de la contribution technique de ce mémoire, en réponse directe aux lacunes identifiées dans l'état de l'art : l'absence de contrôle réseau centralisé, la fragmentation des architectures IoT distribuées et le manque de validation empirique des solutions proposées dans la littérature.

La démarche expérimentale s'articule en trois phases complémentaires et séquentielles, chacune validée par des métriques quantitatives précises :

- **Phase A** : Simulation du trafic routier et contrôle adaptatif des feux tricolores via SUMO et l'orchestrateur TraCI.
- **Phase B** : Émulation du réseau véhiculaire SDN et évaluation de l'impact de la qualité de service (QoS) sur les communications de contrôle.
- **Phase C** : Consolidation et analyse du rapport de performance global croisant les résultats des deux phases précédentes.

L'ensemble de l'environnement expérimental a été développé en Python 3.9, sous Linux Ubuntu.

Précision sur le périmètre expérimental. L'architecture conceptuelle présentée au chapitre 1, capteurs IoT distribués, communications V2X/V2I, unités de bord de route, priorisation dynamique par le contrôleur SDN, définit l'horizon technologique visé par ce travail. Le système effectivement conçu, implémenté et mesuré dans ce chapitre correspond à un périmètre plus resserré : la Phase A repose sur une simulation de trafic pilotée par SUMO et l'interface TraCI, sans capteur IoT physique ni communication véhiculaire réelle, la détection des files d'attente étant entièrement dérivée de l'état interne du simulateur ; la Phase B mesure la robustesse d'un flux de contrôle UDP générique face à un trafic de fond généré par iperf, sans véhicule simulé ni message V2X transitant sur le réseau émulé. Le système effectivement évalué est donc, pour l'essentiel, un contrôleur adaptatif de carrefour

couplé à une politique de qualité de service réseau pilotée par SDN. Cette contribution est validée de façon rigoureuse sur ce périmètre précis ; l'intégration de capteurs IoT physiques, d'unités de bord de route et de communications V2X/V2I réelles, plus largement développée au chapitre 1, reste à ce stade une perspective de recherche et non un résultat mesuré dans ce mémoire (cf. conclusion générale).

2. Environnement technique

2.1 Outils et rôles

Le tableau 3.1 suivant recense les sept outils constituant l'environnement technique du projet, ainsi que leur rôle précis dans le système.

Tableau 3.1 : Outils de l'environnement expérimental

Outil	Rôle dans le système
SUMO (Simulation of Urban MObility)	Simulation microscopique du trafic routier : modélisation des véhicules, des intersections et des feux à l'échelle du pas de temps.
TraCI (Traffic Control Interface)	Interface Python permettant d'interagir en temps réel avec SUMO : lecture de l'état du réseau et envoi de commandes de contrôle aux feux.
Ryu	Contrôleur SDN compatible OpenFlow 1.3 ; gère la programmation des flux réseau et l'activation de la QoS via une API REST.
Mininet-WiFi	Émulateur de réseau sans fil ; crée un environnement véhiculaire virtuel avec des commutateurs Open vSwitch programmables.
Open vSwitch (OVS)	Commutateur virtuel programmable via OpenFlow ; implémente les files de priorité HTB utilisées pour la QoS.
iperf2	Générateur et analyseur de trafic réseau (UDP/TCP) ; permet de mesurer débit, jitter et taux de perte sur les flux de contrôle.
Python 3.9	Langage de développement unifié pour l'orchestrateur adaptatif, le contrôleur Ryu, la topologie Mininet-WiFi et les outils d'analyse.

2.2 Installation de l'environnement

L'installation de l'environnement repose sur trois logiciels à déployer dans l'ordre suivant : Mininet-WiFi (couche réseau), Ryu (contrôleur SDN) et SUMO (simulateur de trafic). Les commandes ci-dessous sont à exécuter dans un terminal Linux sous Ubuntu.

2.2.1 Installation de Mininet-WiFi

La mise à jour des paquets, le clonage du dépôt et l'installation complète de Mininet-WiFi sont détaillés dans les étapes suivantes :

Exécuter la commande ci-dessous pour la mise à jour de la liste des paquets disponibles.

```
bash sudo apt update && sudo apt upgrade -y
```

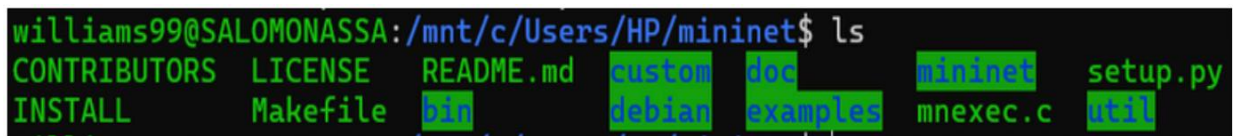
Exécuter la commande ci-dessous pour installer Git, un outil de gestion de versions, sur les systèmes Linux :

```
bash sudo apt install git
```

Exécuter la commande ci-dessous pour télécharger le code source de Mininet depuis GitHub dans un dossier local pour clonage du dépôt Mininet :

```
bash git clone https://github.com/mininet/mininet
```

Pour afficher la liste des fichiers et dossiers présents dans le répertoire courant, il faut exécuter la commande `ls` comme présenté dans la figure 3.1. Dans notre cas, elle montre le contenu du dossier mininet cloné depuis GitHub.

A terminal window with a black background and green text. The prompt is 'williams99@SALOMONASSA:/mnt/c/Users/HP/mininet\$'. The command 'ls' has been executed, and the output is displayed in two lines: 'CONTRIBUTORS LICENSE README.md custom doc mininet setup.py' and 'INSTALL Makefile bin debian examples mnexec.c util'.

```
williams99@SALOMONASSA:/mnt/c/Users/HP/mininet$ ls
CONTRIBUTORS  LICENSE  README.md  custom  doc  mininet  setup.py
INSTALL       Makefile  bin        debian  examples  mnexec.c  util
```

Figure 3.1 : Contenu du dossier Mininet cloné

Pour poursuivre, exécuter le script ci-dessous pour l'installation de Mininet avec l'option `-a` qui signifie "installer tous les composants" nécessaires au bon fonctionnement de Mininet :

```
bash mininet/util/install.sh -a
```

Exécuter ensuite la commande ci-dessous pour cloner le code source de Mininet-WiFi depuis GitHub dans un dossier local (Mininet-WiFi) :

```
bash git clone git://github.com/intrig-unicamp/mininet-wifi.git
```

À la fin du téléchargement ci-dessus, accéder au répertoire qui vient d'être créé :

```
bash cd mininet-wifi
```

Enfin, exécuter le script `install.sh` ci-dessous pour installer Mininet-WiFi et toutes ses dépendances :

```
bash sudo util/install.sh -Wlnfv
```


2.2.2 Installation de Ryu (Python 3.9)

Le contrôleur Ryu requiert Python 3.9 ainsi qu'un environnement virtuel dédié pour isoler ses dépendances. Installer d'abord pip3 en exécutant la commande ci-dessous :

```
bash sudo apt install -y python3-pip
```

Par la suite exécuter la commande ci-dessous pour installer la dernière version de Ryu compatible avec Python 3.9 :

```
bash sudo apt update
```

```
bash sudo apt install python3.9 python3.9-venv python3.9-dev
```

Une fois Python 3.9 installé, créer un environnement virtuel dédié en exécutant la commande ci-dessous :

```
bash python3.9 -m venv ryu-env
```

Activer ensuite l'environnement virtuel à l'aide de la commande ci-dessous :

```
bash source ryu-env/bin/activate
```

Cloner ensuite le code source du framework SDN Ryu depuis GitHub en exécutant la commande ci-dessous. Un dossier local ryu contenant tous les fichiers du projet sera créé automatiquement :

```
bash git clone https://github.com/faucetsdn/ryu.git
```

À la fin du téléchargement, accéder au dossier ryu contenant le code source, puis installer le framework Ryu en exécutant son script setup.py avec les droits administrateur :

```
bash cd ryu
```

```
bash sudo python3 ./setup.py install
```

Pour terminer, mettre à jour le framework Ryu via pip3 avec les droits administrateur en exécutant la commande ci-dessous :

```
bash sudo pip3 install --upgrade ryu
```

2.2.3 Installation de SUMO

SUMO s'installe via le gestionnaire de paquets APT. La commande sumo-gui permet de vérifier graphiquement le bon fonctionnement :

Pour installer SUMO, commencer par mettre à jour les paquets avant installation en exécutant la commande ci-dessous :

```
bash sudo apt-get update
```

Installer ensuite SUMO et ses outils associés en exécutant la commande ci-dessous :

```
bash sudo apt-get install sumo sumo-tools sumo-doc
```

Vérifier enfin que SUMO est correctement installé en lançant l'interface graphique avec la commande ci-dessous. Le résultat attendu est illustré à la figure 3.2 :

```
bash sumo-gui
```

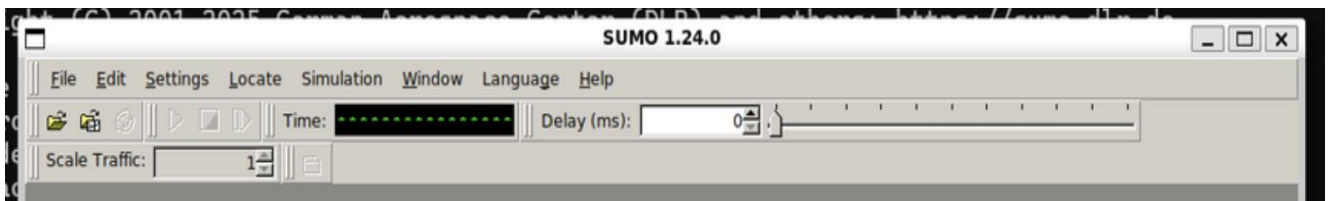


Figure 3.2 : Interface graphique SUMO-GUI

2.3 Architecture du projet

Les fichiers du projet sont organisés dans un répertoire racine nommé `experiments/`. Cette structure sépare clairement les scripts de simulation (`pyfilesTrue/`), les fichiers de configuration SUMO (`sumo_one_junction/`), les scripts de lancement automatisés (`scripts/`), les outils d'analyse (`tools/`) et les résultats expérimentaux bruts (`ctrl_no_qos/` et `ctrl_qos/`), comme le présentent les figures 3.3, 3.4 et 3.5.

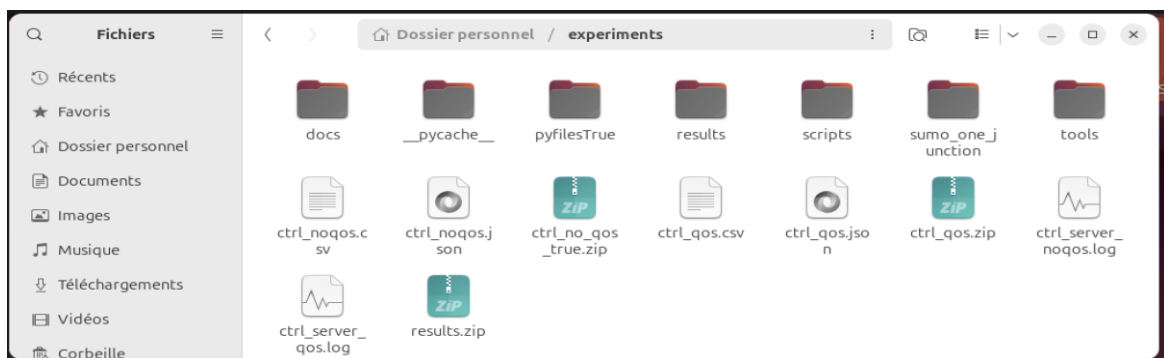


Figure 3.3 : Structure du répertoire `experiments/`

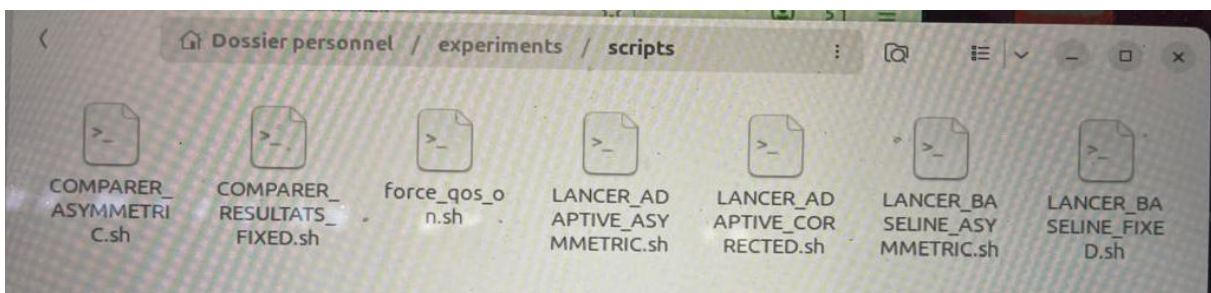


Figure 3.4 : Ensemble des scripts du projet

```

experiments/
├── pyfilesTrue/
│   ├── tls_orchestrator_CORRECTED.py    # Orchestrateur adaptatif
│   │   (TraCI)
│   ├── ryu_qos_rest.py                  # Contrôleur Ryu SDN + API
│   │   REST
│   └── traffic_topology_corridor.py      # Topologie Mininet-WiFi
├── sumo_one_junction/
│   ├── one_junction.net.xml             # Réseau routier J1
│   ├── routes_asymmetric.rou.xml        # Scénario 1300 véhicules
│   └── one_junction_asymmetric.sumocfg   # Configuration SUMO
├── scripts/                             # Scripts de lancement
├── tools/                               # Outils d'analyse et de
rapport
├── ctrl_no_qos/                         # Résultats Phase B1 (sans
QoS)
└── ctrl_qos/                           # Résultats Phase B2 (avec
QoS)

```

Figure 3.5 : Arborescence complète du projet

3. Modélisation du carrefour et scénario de trafic

3.1 Description du carrefour J1

Le réseau routier expérimental est modélisé dans SUMO comme un carrefour isolé à quatre branches, désigné J1, illustré à la figure 3.6. Chaque branche représente une direction cardinale (Nord, Sud, Est, Ouest), formant ainsi deux axes de circulation perpendiculaires : l'axe Nord-Sud (NS) et l'axe Est-Ouest (EW). Ce carrefour unique constitue le point de concentration de toute la charge de trafic, rendant le contrôle des feux déterminant pour la fluidité globale de l'écoulement.

Le feu tricolore de J1 est gouverné par un cycle à quatre phases, détaillées dans le tableau 3.2.

Tableau 3.2 : Phases du feu tricolore du carrefour J1

Phase	État	Description
0	GGrr	Axe Nord-Sud au vert ; axe Est-Ouest au rouge.
1	yyrr	Transition (jaune NS) : préparation au changement de phase.
2	rrGG	Axe Est-Ouest au vert ; axe Nord-Sud au rouge.
3	rryy	Transition (jaune EW) : préparation au retour à la phase 0.

En mode baseline (feux fixes), chaque phase verte dure un temps constant de 31 secondes, indépendamment de la charge réelle de chaque axe. En mode adaptatif, la durée est variable et déterminée en temps réel par l'orchestrateur en fonction des files d'attente mesurées sur chaque approche.

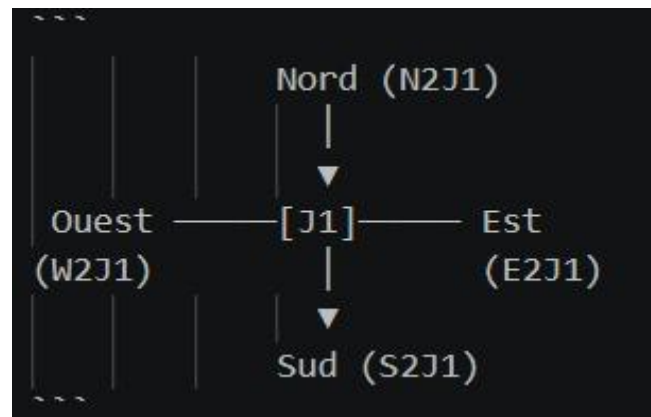


Figure 3.6 : Représentation du carrefour J1 dans SUMO

3.2 Scénario de trafic asymétrique

Le scénario de trafic retenu génère 1 300 véhicules selon un profil délibérément asymétrique, reproduisant les conditions d'une artère principale (NS) intersectant une rue secondaire (EW), comme détaillé dans le tableau 3.3 et visualisé à la figure 3.7.

Tableau 3.3 : Répartition asymétrique du trafic

Direction	Flux (véh/h)	Proportion	Profil
Nord → Sud (NS)	~850	65 %	Artère principale
Est → Ouest (EW)	~450	35 %	Voie secondaire

Ce déséquilibre est représentatif d'un scénario urbain réaliste : un axe principal à forte charge (NS, 65 %) coexiste avec une voie secondaire beaucoup moins fréquentée (EW, 35 %). Dans ces conditions, un contrôle statique à cycles fixes devient structurellement inefficace : la durée de vert est accordée à la voie quasi-vide pendant que l'axe principal sature progressivement, jusqu'à provoquer le blocage définitif de véhicules qui ne peuvent plus entrer dans le carrefour avant la fin de la simulation. C'est précisément cette situation de défaillance du système statique que l'orchestrateur adaptatif est conçu pour prévenir.

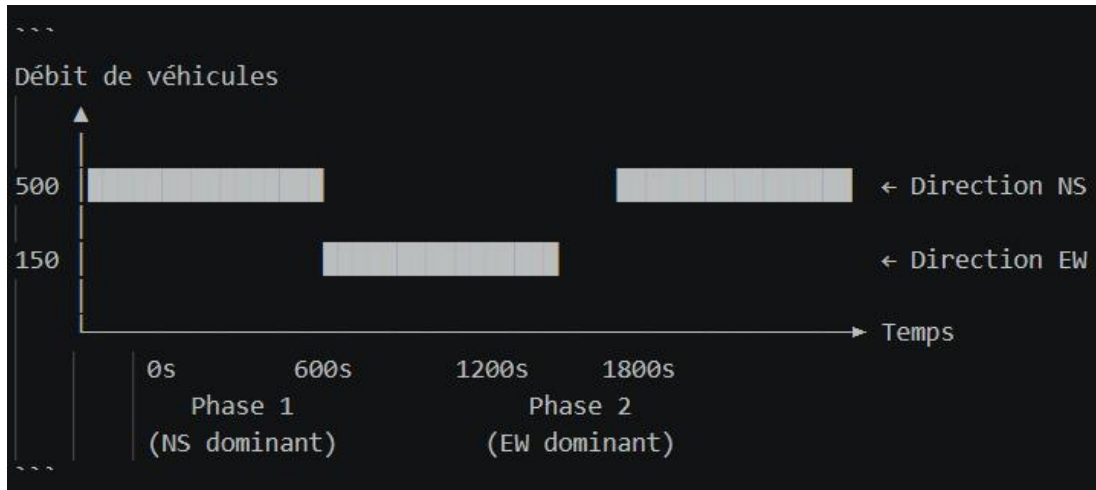


Figure 3.7 : Visualisation du scénario de trafic asymétrique

4. Phase A : Contrôle adaptatif des feux tricolores

4.1 Principe de l'orchestrateur adaptatif

L'orchestrateur adaptatif est implémenté dans le module `tls_orchestrator_CORRECTED.py`. Il se connecte à SUMO via l'interface TraCI et exécute une boucle de contrôle à chaque pas de simulation (typiquement 1 seconde). À chaque itération, il interroge le nombre de véhicules présents sur chaque approche du carrefour :

```
ns_q = sum(tr.edge.getLastStepVehicleNumber(e) for e in ns_edges)
ew_q = sum(tr.edge.getLastStepVehicleNumber(e) for e in ew_edges)
```

Les variables `ns_q` et `ew_q` représentent respectivement la longueur de file d'attente sur les approches Nord-Sud et Est-Ouest. L'orchestrateur compare ces deux valeurs et décide dynamiquement quelle direction doit bénéficier du feu vert, prolongeant ou interrompant la phase en cours selon la charge réelle. Cette logique de contrôle à rétroaction remplace les cycles temporels rigides par une politique de décision réactive, alignée sur l'état instantané du réseau routier.

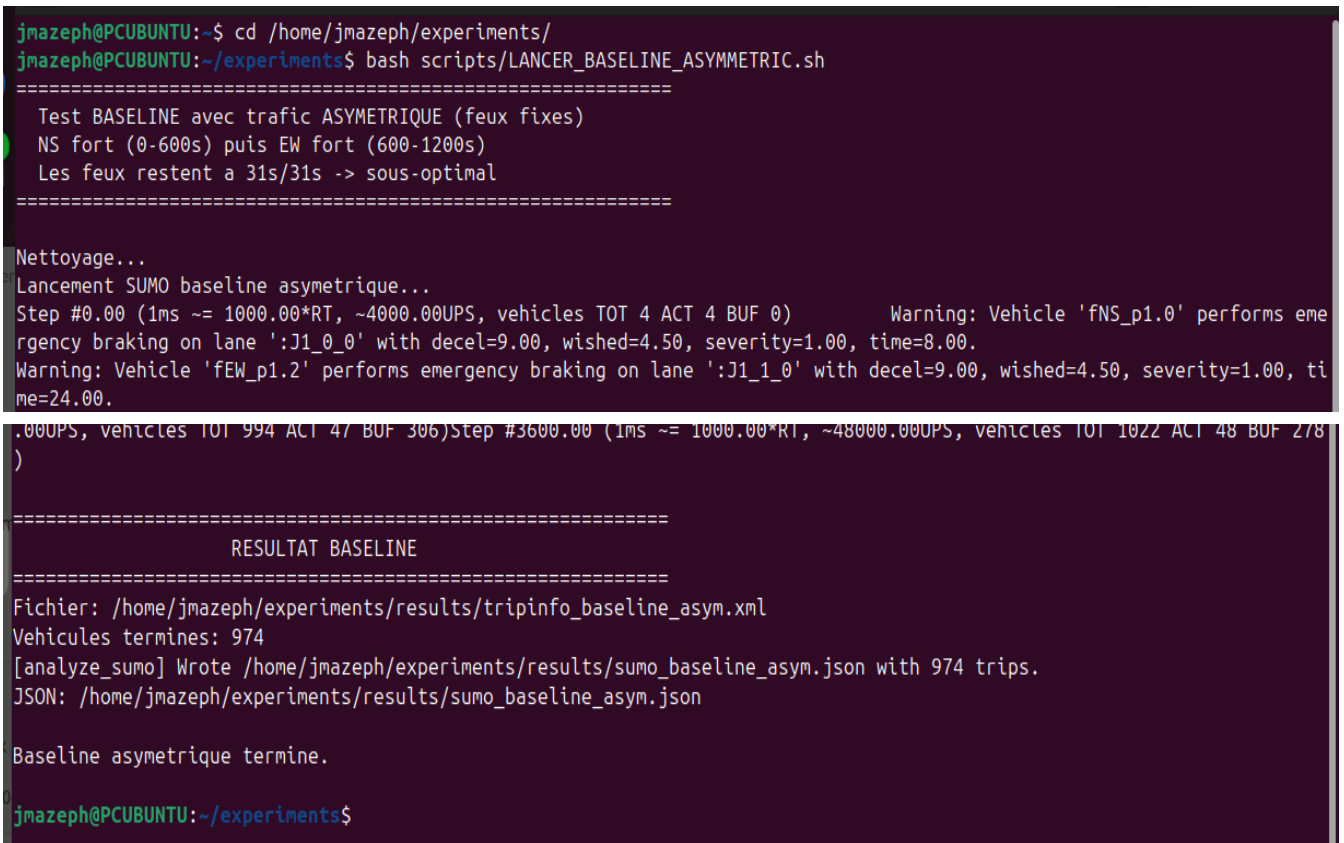
Plus précisément, la bascule d'une phase à l'autre obéit à une règle de seuil avec hystérésis, et non à un algorithme d'apprentissage automatique. Un changement de phase n'est déclenché que si la charge de la direction actuellement au rouge dépasse celle de la direction au vert de plus de 15 % (le paramètre d'hystérésis, destiné à éviter les oscillations de phase), et seulement après un temps de vert minimum de 10 secondes. Si aucune bascule naturelle ne se produit, un temps de vert maximum de 45 secondes force le changement, afin de garantir l'équité entre les deux directions. Ce choix d'un contrôle déterministe et explicable, plutôt qu'un modèle d'apprentissage par renforcement comme celui de Damadam et al. (2022), garantit la reproductibilité et l'auditabilité des décisions de l'orchestrateur, un critère important pour un déploiement dans une infrastructure critique.

4.2 Test A1 : Baseline (feux fixes)

Le premier test établit le comportement de référence du système avec des feux à cycles fixes. Il permet de quantifier les limites intrinsèques du contrôle statique face à un trafic asymétrique.

Pour réaliser le test, il faut exécuter les commandes suivantes comme présenté sur la figure 3.8 :

```
bash cd /home/jmazeph/experiments
bash scripts/LANCER_BASELINE_ASYMMETRIC.sh
```



```
jmazeph@PCUBUNTU:~$ cd /home/jmazeph/experiments/
jmazeph@PCUBUNTU:~/experiments$ bash scripts/LANCER_BASELINE_ASYMMETRIC.sh
=====
Test BASELINE avec trafic ASYMETRIQUE (feux fixes)
NS fort (0-600s) puis EW fort (600-1200s)
Les feux restent a 31s/31s -> sous-optimal
=====

Nettoyage...
Lancement SUMO baseline asymetrique...
Step #0.00 (1ms ~= 1000.00*RT, ~4000.00UPS, vehicles TOT 4 ACT 4 BUF 0)      Warning: Vehicle 'fNS_p1.0' performs emergency braking on lane ':J1_0_0' with decel=9.00, wished=4.50, severity=1.00, time=8.00.
Warning: Vehicle 'fEW_p1.2' performs emergency braking on lane ':J1_1_0' with decel=9.00, wished=4.50, severity=1.00, time=24.00.
Step #3600.00 (1ms ~= 1000.00*RT, ~48000.00UPS, vehicles TOT 1022 ACT 48 BUF 278)
)

=====
RESULTAT BASELINE
=====
Fichier: /home/jmazeph/experiments/results/tripinfo_baseline_asym.xml
Vehicules terminees: 974
[analyze_sumo] Wrote /home/jmazeph/experiments/results/sumo_baseline_asym.json with 974 trips.
JSON: /home/jmazeph/experiments/results/sumo_baseline_asym.json

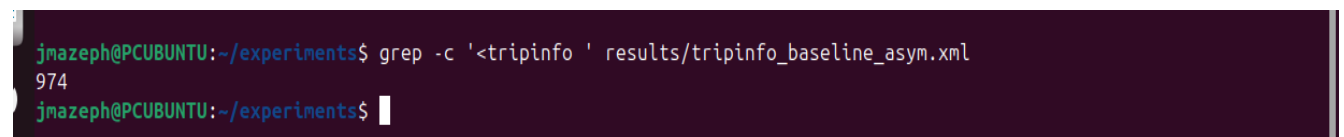
Baseline asymetrique termine.
jmazeph@PCUBUNTU:~/experiments$
```

Figure 3.8 : Exécution du test Baseline

Attendre la fin de la simulation (quelques minutes), puis vérifier le nombre de véhicules traités en exécutant la commande présentée à la figure 3.9 :

Vérification du nombre de véhicules traités :

```
bash grep -c '<tripinfo ' results/tripinfo_baseline_asym.xml
```



```
jmazeph@PCUBUNTU:~/experiments$ grep -c '<tripinfo ' results/tripinfo_baseline_asym.xml
974
jmazeph@PCUBUNTU:~/experiments$
```

Figure 3.9 : Résultat, 974 véhicules traités sur 1 300

Résultat obtenu : 974 véhicules sur 1 300 ont complété leur trajet. Les 326 véhicules restants (25,1 %) ont été bloqués dans le réseau avant la fin de la simulation, dans l'impossibilité d'entrer dans le carrefour saturé. Les résultats bruts sont archivés dans `results/tripinfo_baseline_asym.xml` et `results/sumo_baseline_asym.json`.

Ce résultat est celui de la graine aléatoire 42, illustrée par les figures 3.8 et 3.9. Pour vérifier qu'il ne tient pas au tirage des véhicules, le même scénario a été rejoué sur trente graines indépendantes (1 à 29, plus la graine 42) avec le script `repetier_simulations.py`. Le blocage moyen s'établit à 326,4 véhicules, soit 25,11 % du trafic injecté, avec un écart-type de 0,37 point et un intervalle de confiance à 95 % de [24,97 % ; 25,25 %] ; selon la graine, le nombre de véhicules bloqués varie de 316 à 334. Les 326 véhicules rapportés ci-dessus se situent donc au centre de cette distribution : le résultat illustré n'est pas un cas particulier favorable. Les trente exécutions sont archivées dans `results/repetitions_30_graines.json`.

Attention au biais du survivant : les métriques moyennes (durée de trajet, temps perdu) calculées sur le Baseline ne portent que sur les 974 véhicules ayant effectivement terminé leur parcours. Les 326 véhicules bloqués sont absents de ces moyennes, ce qui tend à sous-estimer l'inefficacité réelle du système statique.

4.3 Test A2 : Adaptatif (orchestrateur connecté)

Le second test active l'orchestrateur adaptatif. L'ensemble des 1 300 véhicules doivent compléter leur trajet, confirmant la capacité du contrôle adaptatif à prévenir la saturation.

Pour réaliser le test, il faut exécuter les commandes suivantes comme présenté sur les figures 3.10 et 3.11 :

`bash scripts/LANCER_ADAPTIVE_ASYMMETRIC.sh`

```

jmaazeph@PCUBUNTU:~/experiments$ bash scripts/LANCER_ADAPTIVE_ASYMMETRIC.sh
=====
Test ADAPTIVE avec trafic ASYMETRIQUE
L'orchestrateur adapte les feux a la demande variable
=====

Nettoyage...

Lancement SUMO avec remote-port 8813...
SUMO PID: 14866

Attente du port 8813...
Port 8813 ecoute (tentative 2)

Lancement orchestrateur adaptatif...
(avec --post-url pour envoyer les metriques a Ryu si actif)
[sudo] Mot de passe de jmaazeph :
[orchestrator] Connected to SUMO on port 8813
[orchestrator] Auto-detected edges. NS=['N2J1', 'S2J1'] EW=['E2J1', 'W2J1']
[orchestrator] Using FORCED indices: NS=0, EW=2
[orchestrator] Loop (min=10.0s, max=45.0s, hyst=0.15)
[orchestrator] Starting with NS_GREEN (phase=0)
[orchestrator] [ 5.00s] queues: NS= 4 EW= 2 phase=0(NS) elapsed=1.0s
[orchestrator] [ 10.00s] queues: NS= 7 EW= 3 phase=0(NS) elapsed=6.0s
[orchestrator] [ 15.00s] queues: NS= 9 EW= 3 phase=0(NS) elapsed=11.0s
[orchestrator] [ 20.00s] queues: NS= 11 EW= 4 phase=0(NS) elapsed=16.0s
[orchestrator] [ 25.00s] queues: NS= 15 EW= 5 phase=0(NS) elapsed=21.0s
[orchestrator] [ 30.00s] queues: NS= 15 EW= 5 phase=0(NS) elapsed=26.0s

```

Figure 3.10 : Exécution du test Adaptatif (lancement de l'orchestrateur)


```

[orchestrator] Switch -> EW_GREEN (phase=2) [NS_q=0, EW_q=3, elapsed=10.0s]
[orchestrator] [4775.00s] queues: NS= 0 EW= 2 phase=2(EW) elapsed=1.0s
[orchestrator] [4780.00s] queues: NS= 0 EW= 1 phase=2(EW) elapsed=6.0s
[orchestrator] [4785.00s] queues: NS= 0 EW= 0 phase=2(EW) elapsed=11.0s
[orchestrator] [4790.00s] queues: NS= 0 EW= 0 phase=2(EW) elapsed=16.0s
[orchestrator] SUMO finished.

=====
RESULTAT ADAPTIVE
=====
Fichier: /home/jmazeph/experiments/results/tripinfo_adaptive_asym.xml
Vehicules termies: 1300

Changements de phase:
  NS_GREEN: 56
  EW_GREEN: 57
  TOTAL: 113
[analyze_sumo] Wrote /home/jmazeph/experiments/results/sumo_adaptive_asym.json with 1300 trips.
JSON: /home/jmazeph/experiments/results/sumo_adaptive_asym.json

Adaptive asymetrique termine.

jmazeph@PCUBUNTU:~/experiments$

```

Figure 3.11 : Exécution du test Adaptatif, suivi en temps réel des commutations de phase

À la fin du test, attendre quelques minutes, puis faire la vérification en exécutant la commande ci-dessous donnant le résultat affiché à la figure 3.12 :

```

# Vérification des commutations de phase :
bash grep -c "Switch -> NS_GREEN" results/orchestrator_asym.log
bash grep -c "Switch -> EW_GREEN" results/orchestrator_asym.log

```

```

jmazeph@PCUBUNTU:~/experiments$ grep -c "Switch -> NS_GREEN" results/orchestrator_asym.log
56
jmazeph@PCUBUNTU:~/experiments$ grep -c "Switch -> EW_GREEN" results/orchestrator_asym.log
57
jmazeph@PCUBUNTU:~/experiments$

```

Figure 3.12 : Comptage des commutations NS et EW dans le log

L'orchestrateur a effectué 113 commutations de phase au total : 56 activations du vert Nord-Sud et 57 activations du vert Est-Ouest. Cette distribution quasi-équilibrée illustre la sensibilité fine du système aux variations de charge : bien que le flux NS soit dominant (65 %), l'orchestrateur n'accorde pas mécaniquement plus de vert à cet axe, il adapte la durée de chaque phase en fonction de l'état réel des files d'attente, évitant ainsi l'accumulation de véhicules dans les deux directions.

Ce test a lui aussi été rejoué sur les trente mêmes graines. Les 1 300 véhicules complètent leur trajet dans les trente cas, sans exception : l'écart-type du nombre de véhicules bloqués est nul, et le taux de complétion de 100 % n'est donc pas un tirage favorable. Le nombre de commutations de phase reste stable d'une graine à l'autre, à 111,9 en moyenne (IC 95 % :

[111,4 ; 112,5]), ce qui situe les 113 commutations relevées ci-dessus dans le comportement ordinaire de l'orchestrateur.

4.4 Analyse comparative Baseline vs. Adaptatif

L'analyse comparative est générée par le script `COMPARER_ASYMMETRIC.sh`, qui exploite les fichiers de résultats des deux tests pour calculer les indicateurs clés de performance. Pour générer ce rapport, exécuter la commande ci-dessous ; les résultats sont affichés aux figures 3.13 et 3.14, ainsi que dans le tableau 3.4.

```
bash scripts/COMPARER_ASYMMETRIC.sh
```

```
jmaazeph@PCUBUNTU:~/experiments$ bash scripts/COMPARER_ASYMMETRIC.sh
=====
  COMPARAISON BASELINE vs ADAPTIVE (Asymetrique)
=====
Fichiers JSON trouves

=====
                BASELINE (Feux Fixes 31s/31s)
=====
Vehicules:      974
Duree moy:      169.1 s
Temps d'attente: 81.6 s
Temps perdu:    154.0 s
Throughput:     974.3 veh/h

=====
                ADAPTIVE (Orchestrateur)
=====
Vehicules:      1300
Duree moy:      169.0 s
Temps d'attente: 82.1 s
Temps perdu:    153.8 s
Throughput:     977.2 veh/h
```

Figure 3.13 : Exécution du script de comparaison Baseline vs. Adaptatif

```
=====
DIFFERENCES (negatif = amelioration)
=====
Duree:          -0.1%
Temps d'attente: +0.6%
Temps perdu:    -0.1%
Throughput:     +0.3%

=====
                TAUX DE COMPLETION (metrique cle)
=====
Vehicules injectes: 1300
Baseline terminees: 974/1300 (74.9%)
Adaptive terminees: 1300/1300 (100.0%)
Vehicules sauves:   326
Gain completion:    +33.5%

>>> L'orchestrateur adaptatif evite la saturation!
>>> Les moyennes par vehicule sont similaires car le baseline
>>> ne compte que les vehicules ayant reussi a passer.
>>> Les vehicules bloques sont INVISIBLES dans les moyennes.

=====
Moyennes par vehicule (attention: biais de survivant)
=====
Temps d'attente: similaire (Baseline: 81.6s -> Adaptive: 82.1s)
(Normal: le baseline ne compte que les 974 vehicules les plus rapides)
Throughput:      974.3 -> 977.2 veh/h

=====
Fichiers utilises:
/home/jmaazeph/experiments/results/sumo_baseline_asym.json
/home/jmaazeph/experiments/results/sumo_adaptive_asym.json
```

Figure 3.14 : Rapport comparatif complet généré par le script

Tableau 3.4 : Comparaison Baseline vs. Adaptatif (Phase A)

Indicateur	Baseline (feux fixes)	Adaptatif (orchestrateur)
Véhicules traités	974 / 1 300 (74,9 %)	1 300 / 1 300 (100 %)
Véhicules bloqués	326 (25,1 %)	0 (0 %)
Durée moyenne de trajet	169,1 s	169,0 s
Temps perdu moyen	154,0 s *	153,8 s
Commutations de phase	Cyclique fixe (31 s)	113 (56 NS + 57 EW)

NB : la moyenne de durée de trajet et le temps perdu moyen du Baseline sont calculés uniquement sur les 974 véhicules ayant terminé leur parcours (biais du survivant). Les 326 véhicules bloqués ne sont pas comptabilisés, ce qui sous-estime l'inefficacité réelle du système statique.

4.5 Conclusion de la Phase A

Les résultats de simulation confirment l'hypothèse centrale de ce travail : les feux tricolores connectés et adaptatifs surpassent structurellement les systèmes à cycles fixes face à un trafic asymétrique congestionné.

L'indicateur le plus saillant est la capacité d'écoulement : le Baseline bloque définitivement 25,1 % des véhicules (326 sur 1 300), tandis que l'orchestrateur adaptatif assure le passage de la totalité du trafic. Cet avantage est obtenu sans dégradation de l'expérience individuelle des conducteurs : la durée moyenne de trajet reste identique (169,0 s contre 169,1 s) et le temps perdu moyen est équivalent (153,8 s contre 154,0 s).

L'interprétation de ce résultat est fondamentale : la supériorité du système adaptatif ne tient pas à une accélération des véhicules déjà en mouvement, mais à sa capacité à prévenir la saturation du carrefour. En allouant dynamiquement le temps de vert en fonction de la charge réelle de chaque axe, l'orchestrateur évite la formation de goulets d'étranglement qui, dans le système statique, conduisent irrémédiablement au blocage d'une fraction croissante du trafic. C'est cette propriété de scalabilité, la capacité à absorber une charge de trafic élevée sans effondrement, qui constitue l'apport décisif des feux connectés sur les systèmes conventionnels.

Ces deux réserves ont été levées par la mesure. D'une part, le cycle de 31 s par axe n'est pas un réglage d'ingénieur : c'est la valeur que SUMO attribue par défaut à un programme statique généré, et elle ne procède d'aucun calcul. Un second plan à temps fixe a donc été dimensionné par la formule du cycle optimal de Webster [31], $C_0 = (1,5 L + 5) / (1 - Y)$,

avec un temps perdu L de 18 s et un débit de saturation de 1 800 véhicules par heure et par voie ; le mouvement critique s'établissant à 325 véhicules par heure et par axe, il vient $Y = 0,36$ et un cycle de 50 s réparti en 20 s de vert par axe. Rejoué sur les mêmes trente graines appariées, ce plan atteint 76,5026 % de complétion (IC 95 % : [76,4123 ; 76,5928]), contre 74,8923 % pour le plan par défaut et 100 % pour l'orchestrateur adaptatif : un dimensionnement correct fait gagner 1,61 point au feu fixe, et le contrôle adaptatif conserve 23,50 points d'avance. Le calcul établit en outre que, pendant chaque pointe, Y atteint 1,08 : la demande dépasse alors la capacité du carrefour et aucun plan à temps fixe, si bien dimensionné soit-il, ne peut écouler la pointe. L'écart mesuré ne tient donc pas à un adversaire sous-optimal. D'autre part, les deux tests ont été rejoués sur trente graines aléatoires indépendantes, avec intervalles de confiance à 95 %, comme le rapportent les sections 4.2 et 4.3. Les limites qui subsistent sont d'une autre nature : un carrefour unique, une demande synthétique qui n'a pas été calibrée sur des comptages de terrain, et un périmètre entièrement simulé.

5. Phase B : Réseau véhiculaire SDN avec QoS

5.1 Contexte et objectif

La Phase B évalue l'apport du SDN sur la qualité des communications de contrôle en environnement véhiculaire congestionné. Dans le scénario retenu, les véhicules communiquent avec l'infrastructure routière (Road Side Unit, RSU) selon le paradigme V2R (Vehicle-to-Roadside). En situation d'embouteillage, la densité de véhicules génère une congestion réseau susceptible de dégrader les flux de données de contrôle des feux, précisément lorsque la qualité de ces communications est la plus critique.

Le contrôleur Ryu, via l'API OpenFlow 1.3, permet de programmer des files de priorité HTB (Hierarchical Token Bucket) dans les commutateurs Open vSwitch. Ces files distinguent le trafic de contrôle des feux (port UDP 9999, haute priorité, queue:1) du trafic de fond généré par les véhicules (port UDP 5001, best-effort). L'objectif est de mesurer l'impact de cette priorisation sur le jitter et le taux de perte du flux de contrôle, soumis à une congestion délibérée de 16 Mbit/s sur un lien de 5 Mbit/s.

5.2 Test B1 : Sans QoS (référence réseau)

Ce test mesure les performances du flux de contrôle en l'absence de toute priorisation SDN. Tout le trafic est traité de manière indifférenciée par le commutateur OVS.

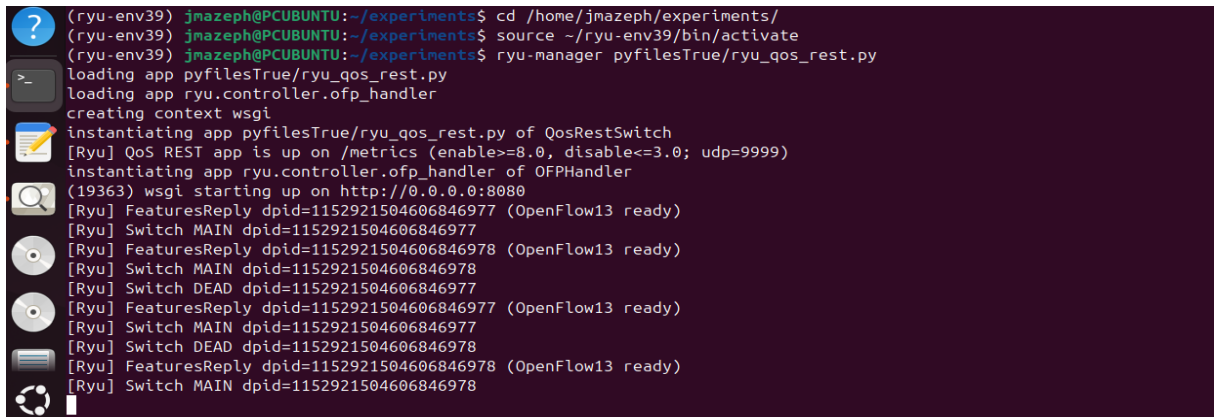
Étape B1.1 : Lancement du contrôleur Ryu (Terminal 1)

Dans le Terminal 1, naviguer vers le dossier **experiments**, ensuite activer l'environnement de lancement, puis lancer le contrôleur Ryu en exécutant respectivement les trois commandes ci-dessous, comme le montre la figure 3.15 :

```
bash cd /home/jmazeph/experiments
```

```
bash source ~/ryu-env39/bin/activate
```

```
bash ryu-manager pyfilesTrue/ryu_qos_rest.py
```



```
(ryu-env39) jmazeph@PCUBUNTU:~/experiments$ cd /home/jmazeph/experiments/
(ryu-env39) jmazeph@PCUBUNTU:~/experiments$ source ~/ryu-env39/bin/activate
(ryu-env39) jmazeph@PCUBUNTU:~/experiments$ ryu-manager pyfilesTrue/ryu_qos_rest.py
Loading app pyfilesTrue/ryu_qos_rest.py
Loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app pyfilesTrue/ryu_qos_rest.py of QosRestSwitch
[Ryu] QoS REST app is up on /metrics (enable>=8.0, disable<=3.0; udp=9999)
instantiating app ryu.controller.ofp_handler of OFPHandler
(19363) wsgi starting up on http://0.0.0.0:8080
[Ryu] FeaturesReply dpid=1152921504606846977 (OpenFlow13 ready)
[Ryu] Switch MAIN dpid=1152921504606846977
[Ryu] FeaturesReply dpid=1152921504606846978 (OpenFlow13 ready)
[Ryu] Switch MAIN dpid=1152921504606846978
[Ryu] Switch DEAD dpid=1152921504606846977
[Ryu] FeaturesReply dpid=1152921504606846977 (OpenFlow13 ready)
[Ryu] Switch MAIN dpid=1152921504606846977
[Ryu] Switch DEAD dpid=1152921504606846978
[Ryu] FeaturesReply dpid=1152921504606846978 (OpenFlow13 ready)
[Ryu] Switch MAIN dpid=1152921504606846978
```

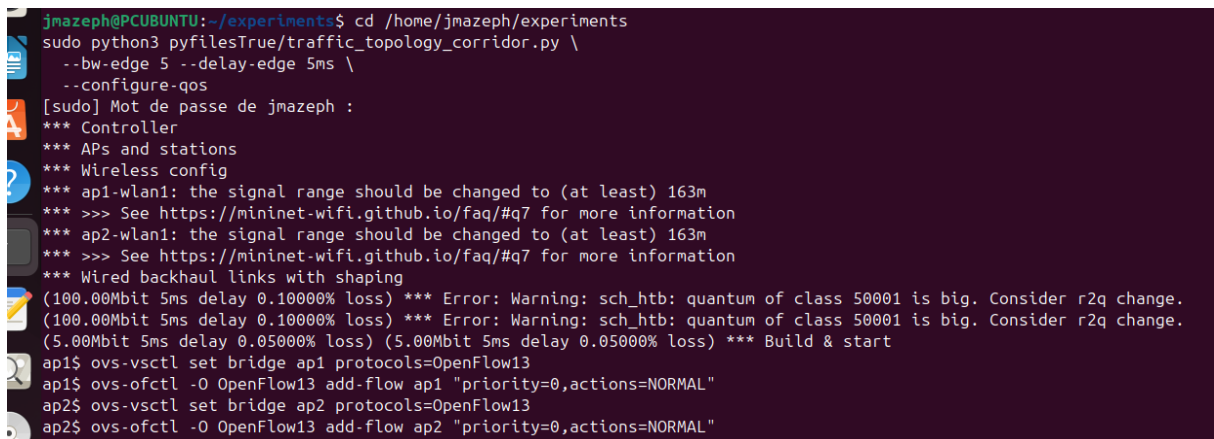
Figure 3.15 : Lancement du contrôleur Ryu (Terminal 1)

Étape B1.2 : Lancement de la topologie Mininet-WiFi (Terminal 2)

Dans le Terminal 2, naviguer vers le dossier **experiments**, puis lancer Mininet-WiFi en exécutant les deux commandes ci-dessous, comme le montre la figure 3.16 :

```
bash cd /home/jmazeph/experiments
```

```
bash sudo python3 pyfilesTrue/traffic_topology_corridor.py \
--bw-edge 5 --delay-edge 5ms
```



```
jmazeph@PCUBUNTU:~/experiments$ cd /home/jmazeph/experiments
sudo python3 pyfilesTrue/traffic_topology_corridor.py \
--bw-edge 5 --delay-edge 5ms \
--configure-qos
[sudo] Mot de passe de jmazeph :
*** Controller
*** APs and stations
*** Wireless config
*** ap1-wlan1: the signal range should be changed to (at least) 163m
*** >>> See https://mininet-wifi.github.io/faq/#q7 for more information
*** ap2-wlan1: the signal range should be changed to (at least) 163m
*** >>> See https://mininet-wifi.github.io/faq/#q7 for more information
*** Wired backhaul links with shaping
(100.00Mbit 5ms delay 0.10000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(100.00Mbit 5ms delay 0.10000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(5.00Mbit 5ms delay 0.05000% loss) (5.00Mbit 5ms delay 0.05000% loss) *** Build & start
ap1$ ovs-vsctl set bridge ap1 protocols=OpenFlow13
ap1$ ovs-ofctl -O OpenFlow13 add-flow ap1 "priority=0,actions=NORMAL"
ap2$ ovs-vsctl set bridge ap2 protocols=OpenFlow13
ap2$ ovs-ofctl -O OpenFlow13 add-flow ap2 "priority=0,actions=NORMAL"
```

Figure 3.16 : Lancement de Mininet-WiFi (Terminal 2)

Étape B1.3 : Vérifier la connectivité (Terminal 2)

Dans la console Mininet, vérifier la connectivité en exécutant la commande pingall ; le constat est fait aux figures 3.17 et 3.18 :

Vérification connectivité, toutes les stations doivent atteindre le nœud edge (10.0.0.254). Si pertes, attendre 10 s et refaire.

bash pingall

```
mininet-wifi> pingall
*** Ping: testing ping reachability
edge -> *** edge : ('ping -c1 10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=7.47 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 7.472/7.472/7.472/0.000 ms
car1 *** edge : ('ping -c1 10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=8.18 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 8.180/8.180/8.180/0.000 ms
car2 *** edge : ('ping -c1 10.0.0.3',)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=8.08 ms

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 8.080/8.080/8.080/0.000 ms
car3
car1 -> *** car1 : ('ping -c1 10.0.0.254',)
PING 10.0.0.254 (10.0.0.254) 56(84) bytes of data.
64 bytes from 10.0.0.254: icmp_seq=1 ttl=64 time=6.73 ms

--- 10.0.0.254 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
```

Figure 3.17 : Test de connectivité pingall (B1)

```
car3
car3 -> *** car3 : ('ping -c1 10.0.0.254',)
PING 10.0.0.254 (10.0.0.254) 56(84) bytes of data.
64 bytes from 10.0.0.254: icmp_seq=1 ttl=64 time=7.51 ms

--- 10.0.0.254 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 7.506/7.506/7.506/0.000 ms
edge *** car3 : ('ping -c1 10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=4.07 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.072/4.072/4.072/0.000 ms
car1 *** car3 : ('ping -c1 10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=4.20 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.196/4.196/4.196/0.000 ms
car2
*** Results: 0% dropped (12/12 received)
mininet-wifi>
```

Figure 3.18 : Résultat du pingall, connectivité totale confirmée

Étape B1.4 : Lancer les serveurs iperf (dans Mininet)

Dans cette étape, lancer les deux serveurs iperf en arrière-plan sur le nœud edge : l'un pour le flux de contrôle (port UDP 9999) et l'autre pour le trafic de fond (port UDP 5001), comme illustré à la figure 3.19 :

```
edge bash -c "iperf -s -u -p 9999 -i 1 > /tmp/ctrl_server_noqos.log 2>&1" &
edge bash -c "iperf -s -u -p 5001 > /tmp/bg_server.log 2>&1" &
```

```
mininet-wifi> edge bash -c "iperf -s -u -p 9999 -i 1 > /tmp/ctrl_server_noqos.log 2>&1" &
[1] 18726
mininet-wifi> edge bash -c "iperf -s -u -p 5001 > /tmp/bg_server.log 2>&1" &
[2] 18733
mininet-wifi>
```

Figure 3.19 : Lancement des serveurs iperf en arrière-plan

Étape B1.5 : Lancer le trafic de fond (dans Mininet)

Dans cette étape, générer le trafic de fond en exécutant les deux commandes ci-dessous. Les nœuds car2 et car3 envoient chacun 8 Mbit/s (soit 16 Mbit/s au total) sur le lien edge limité à 5 Mbit/s, créant ainsi une congestion délibérée, comme illustré à la figure 3.20.

```
car2 bash -c "iperf -c 10.0.0.254 -u -p 5001 -b 8M -t 120 > /dev/null 2>&1" &
car3 bash -c "iperf -c 10.0.0.254 -u -p 5001 -b 8M -t 120 > /dev/null 2>&1" &
```

```
mininet-wifi> car2 bash -c "iperf -c 10.0.0.254 -u -p 5001 -b 8M -t 120 > /dev/null 2>&1" &
mininet-wifi> car3 bash -c "iperf -c 10.0.0.254 -u -p 5001 -b 8M -t 120 > /dev/null 2>&1" &
```

Figure 3.20 : Lancement du trafic de fond

Étape B1.6 : Lancer le flux de contrôle (dans Mininet)

Cette étape permet de lancer le flux de contrôle en exécutant la commande ci-dessous, illustrée à la figure 3.21.

```
car1 iperf -c 10.0.0.254 -u -p 9999 -b 200k -t 120 -i 1
```

```
mininet-wifi> car1 iperf -c 10.0.0.254 -u -p 9999 -b 200k -t 120 -i 1
-----
Client connecting to 10.0.0.254, UDP port 9999
Sending 1470 byte datagrams, IPG target: 58800.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 55562 connected with 10.0.0.254 port 9999
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  27.3 KBytes  223 Kbits/sec
[ 1] 1.0000-2.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 2.0000-3.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 3.0000-4.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 4.0000-5.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 5.0000-6.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 6.0000-7.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 7.0000-8.0000 sec  24.4 KBytes  200 Kbits/sec
[ 1] 8.0000-9.0000 sec  24.4 KBytes  200 Kbits/sec
```

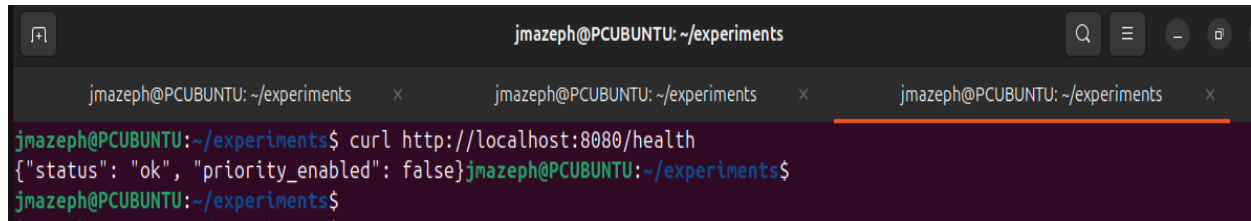
Figure 3.21 : Flux de contrôle car1, mesure du jitter sans QoS

Étape B1.7 : Attendre 120 secondes (Terminal 3)

Ne rien toucher, ouvrir un troisième Terminal et vérifier que la QoS est bien désactivée en exécutant la commande ci-dessous ; le résultat est affiché à la figure 3.22 :

```
bash curl http://localhost:8080/health
```

Résultat attendu : ``{"status":"ok","priority_enabled":false}``



```
jmazeph@PCUBUNTU: ~/experiments
jmazeph@PCUBUNTU: ~/experiments
jmazeph@PCUBUNTU: ~/experiments
jmazeph@PCUBUNTU:~/experiments$ curl http://localhost:8080/health
{"status": "ok", "priority_enabled": false}jmazeph@PCUBUNTU:~/experiments$
jmazeph@PCUBUNTU:~/experiments$
```

Figure 3.22 : Vérification API Ryu, QoS désactivée (*priority_enabled: false*)

Étape B1.8 : Récupération et analyse des résultats

Dans le Terminal 2, copier le fichier journal des mesures vers le répertoire des résultats en exécutant la commande ci-dessous :

```
sh cp /tmp/ctrl_server_noqos.log /home/jmazeph/experiments/results/ctrl_server_noqos.log
```

Ensuite quitter proprement en exécutant les commandes ci-dessous dans les terminaux respectifs :

Mininet (Terminal 2) : `bash 'exit'`

Ryu (Terminal 1) : `bash 'Ctrl+C'`

Enfin, analyser les résultats du Test B1 dans le Terminal 3 en exécutant la commande ci-dessous.

```
bash cd /home/jmazeph/experiments
Puis exécuter
bash python3 tools/analyze_iperf.py \
--log results/ctrl_server_noqos.log \
--out results/ctrl_noqos.json \
--csv results/ctrl_noqos.csv
```

5.3 Test B2 : Avec QoS SDN (priorité activée)

Ce test reproduit exactement les conditions du Test B1, mais active les files de priorité HTB dans Open vSwitch via le contrôleur Ryu. Le flux de contrôle UDP sur le port 9999 est orienté dans la file prioritaire (*queue:1*), tandis que le trafic de fond reste dans la file best-effort.

Étape B2.1 : Lancement et vérification

Dans cette étape, comme déjà indiqué, reprendre les étapes exécutées du test B1, de B1.1 à B1.3 :

Étape B1.1 : Lancement de Ryu (Terminal 1)

Étape B1.2 : Lancement de Mininet (Terminal 2)

Étape B1.3 : Vérifier la connectivité (Terminal 2)

NB : à l'étape B2.1, vérifier les files HTB OVS (Terminal 2) en exécutant la commande ci-dessous, qui doit afficher des entrées de type linux-htb, comme le montre la figure 3.23.

```
sh sudo ovs-vsctl list qos
```

```
mininet-wifi> sh sudo ovs-vsctl list qos
_uuid            : 43b5afa0-0f30-49a1-9674-2f57b472dc30
external_ids     : {}
other_config     : {max-rate="100000000"}
queues           : {0=03b6e6ab-8b13-4f67-8214-d981d8d55136, 1=220cd0b0-aeaa-4d50-8d51-d1a804352e3e}
type             : linux-htb

_uuid            : e50d33f2-645b-4996-8ac5-adfd96d68157
external_ids     : {}
other_config     : {max-rate="100000000"}
queues           : {0=ee14ec54-0f8f-4207-99f4-aa215cddb3e3, 1=cd96deba-1e8d-4b72-9170-bb32e86eaff0}
type             : linux-htb

_uuid            : c0ad0032-7a95-415b-adc2-d99fddb55a38
external_ids     : {}
other_config     : {max-rate="100000000"}
queues           : {0=a276068e-750d-4a55-a511-7537ec6cad6b}
type             : linux-htb

_uuid            : 3741f4d4-c31b-4aa0-92ab-388e0f3a8360
external_ids     : {}
other_config     : {max-rate="50000000"}
queues           : {0=1fffc57-356b-43a0-b70e-da99e1c2cc11, 1=1e400ae7-720a-48f3-be66-6ba57f532b0a}
type             : linux-htb

_uuid            : 8a50790c-9410-4732-b12f-61f68e16794d
external_ids     : {}
other_config     : {max-rate="100000000"}
queues           : {0=e384b5ca-1839-4de0-9f5c-4d794b0f511c, 1=29a178ab-d7ea-47bf-b608-12b37847cfae}
```

Figure 3.23 : Files HTB OVS confirmées (linux-htb)

Étape B2.2 : Activer la QoS manuellement (Terminal 3)

La QoS est activée en exécutant le script dédié ci-dessous, comme illustré à la figure 3.24 :

```
bash scripts/force_qos_on.sh
```

```
jmazeph@PCUBUNTU: ~/experiments
jmazeph@PCUBUNTU: ~/experiments x jmazeph@PCUBUNTU: ~/experiments x jmazeph@PCUBUNTU: ~/experiments x
jmazeph@PCUBUNTU:~/experiments$ bash scripts/force_qos_on.sh
Envoi de busy=20 a Ryu pour activer la QoS...
{"status": "ok", "busy": 20.0, "priority": true}

Verification:
{"status": "ok", "priority_enabled": true}

Si priority_enabled=true -> QoS activee
Verifier les regles OVS:
sudo ovs-ofctl -O OpenFlow13 dump-flows ap1 | grep set_queue
jmazeph@PCUBUNTU:~/experiments$
```

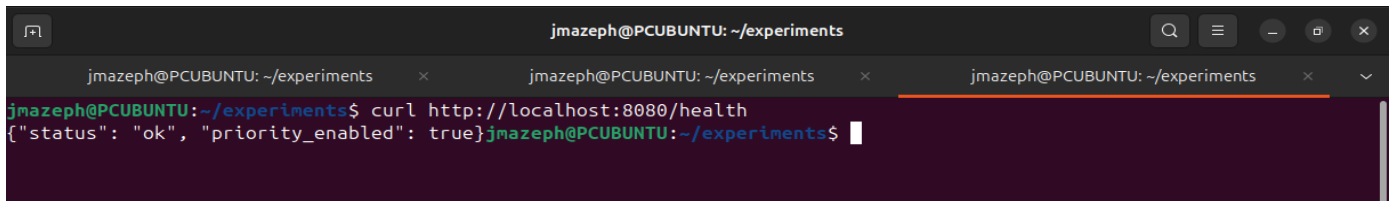
Figure 3.24 : Activation manuelle de la QoS via force_qos_on.sh

Étape B2.3 : Vérification (obligatoire)

Avant de lancer le trafic, vérifier que la QoS est bien activée en exécutant la commande ci-dessous. Le résultat doit confirmer priority_enabled: true, comme indiqué à la figure 3.25.

```
bash curl http://localhost:8080/health
```


Résultat attendu : {"status":"ok","priority_enabled":true}



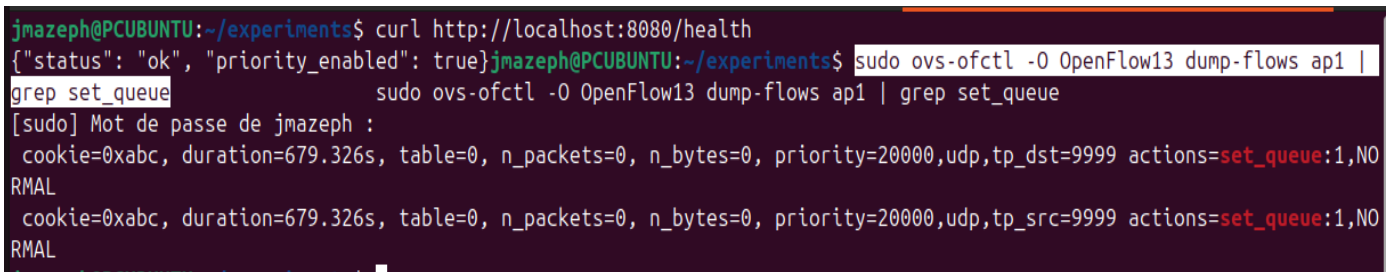
```
jmaze@PCUBUNTU: ~/experiments
jmaze@PCUBUNTU: ~/experiments
jmaze@PCUBUNTU: ~/experiments
jmaze@PCUBUNTU:~/experiments$ curl http://localhost:8080/health
{"status": "ok", "priority_enabled": true}jmaze@PCUBUNTU:~/experiments$
```

Figure 3.25 : QoS activée

Étape B2.4 : Vérification des règles OpenFlow

Cette étape vérifie que le contrôleur Ryu a correctement programmé le commutateur Open vSwitch (ap1) avec des règles OpenFlow de priorisation. Exécuter la commande ci-dessous : la présence de l'action set_queue confirme la politique de QoS active, comme illustré à la figure 3.26.

bash sudo ovs-ofctl -O OpenFlow13 dump-flows ap1 | grep set_queue



```
jmaze@PCUBUNTU:~/experiments$ curl http://localhost:8080/health
{"status": "ok", "priority_enabled": true}jmaze@PCUBUNTU:~/experiments$ sudo ovs-ofctl -O OpenFlow13 dump-flows ap1 |
grep set_queue
[sudo] Mot de passe de jmaze :
cookie=0xabc, duration=679.326s, table=0, n_packets=0, n_bytes=0, priority=20000,udp,tp_dst=9999 actions=set_queue:1,NO
RMAL
cookie=0xabc, duration=679.326s, table=0, n_packets=0, n_bytes=0, priority=20000,udp,tp_src=9999 actions=set_queue:1,NO
RMAL
```

Figure 3.26 : Règles OpenFlow confirmant la priorisation UDP:9999

Étape B2.5 : Flux, attente et analyse

Exécuter successivement les étapes du test B1 ci-dessous.

Étape B1.4 : Lancer les serveurs iperf

Étape B1.5 : Lancer le trafic de fond

Étape B1.6 : Lancer le flux de contrôle

Étape B1.7 : Attendre 120 secondes

Étape B1.8 : Récupération et analyse des résultats

5.4 Résultats comparatifs B1 vs. B2

Le tableau 3.5 présente une analyse comparative des tests B1 (sans QoS) et B2 (avec QoS activée), avec les métriques réseau mesurées sous une charge de 16 Mbit/s sur un lien de 5 Mbit/s. Les valeurs proviennent des mesures archivées dans results/ctrl_noqos.json et results/ctrl_qos.json ; sur un flux de contrôle émis à 200 kbit/s, le débit utile reçu passe de 0,056 Mbit/s sans QoS à 0,200 Mbit/s avec QoS, soit la totalité du flux. Les graphiques correspondants sont présentés à la figure 3.29.

Tableau 3.5 : Comparaison Sans QoS (B1) vs. Avec QoS SDN (B2)

Métrique réseau	Sans QoS (B1)	Avec QoS SDN (B2)
Jitter flux de contrôle (ms)	110,31 en moyenne (p95 : 212,28)	0,06 en moyenne (p95 : 0,12)
Taux de perte UDP:9999	69,6 % en moyenne (p95 : 94 %)	0 %
Débit trafic de fond	Illimité (16 Mbit/s)	Contrôlé (file best-effort)
Priorité flux critique	Aucune	File HTB queue:1

5.5 Conclusion de la Phase B

Les résultats de la Phase B démontrent que la programmabilité réseau offerte par le SDN constitue un levier décisif pour la fiabilité des communications véhiculaires en conditions de congestion. Sans priorisation (Test B1), le flux de contrôle des feux subit une compétition directe avec le trafic de fond, entraînant une dégradation significative du jitter et un taux de perte qui compromet la fiabilité des décisions de l'orchestrateur adaptatif. L'activation de la QoS SDN (Test B2) isole le flux de contrôle dans une file dédiée à haute priorité, lui garantissant un acheminement stable et prévisible indépendamment de la charge du réseau.

Ce résultat valide la pertinence de l'architecture SDN pour les infrastructures de feux connectés : la séparation entre le plan de contrôle et le plan de données, principe fondateur du SDN, se traduit ici concrètement par la capacité à garantir des communications de contrôle critiques même lorsque le réseau est saturé par le trafic véhiculaire ordinaire. C'est cette garantie de service qui distingue une infrastructure SDN d'une architecture réseau conventionnelle pour les applications de gestion de trafic en temps réel.

6. Phase C : Rapport de performance consolidé

6.1 Génération du rapport global

Une fois les phases A et B terminées, le script `merge_report.py` consolide l'ensemble des résultats en un rapport unique au format Markdown, croisant les métriques de trafic (Phase A) et les métriques réseau (Phase B) en exécutant les commandes ci-dessous, affichées dans la figure 3.27. Le rapport généré est illustré à la figure 3.28.

```
cd /home/jmazeph/experiments
python3 tools/merge_report.py \
    --sumo-baseline results/sumo_baseline_asym.json \
    --sumo-adaptive results/sumo_adaptive_asym.json \
    --ctrl-noqos results/ctrl_noqos.json \
```

```
--ctrl-qos      results/ctrl_qos.json \
--out           results/report.md
```

```
jmazeph@PCUBUNTU:~/experiments$ cd /home/jmazeph/experiments
python3 tools/merge_report.py \
--sumo-baseline results/sumo_baseline_asym.json \
--sumo-adaptive results/sumo_adaptive_asym.json \
--ctrl-noqos results/ctrl_noqos.json \
--ctrl-qos results/ctrl_qos.json \
--out results/report.md
[merge_report] Wrote results/report.md
jmazeph@PCUBUNTU:~/experiments$
```

Figure 3.27 : Génération du rapport consolidé (merge_report.py)

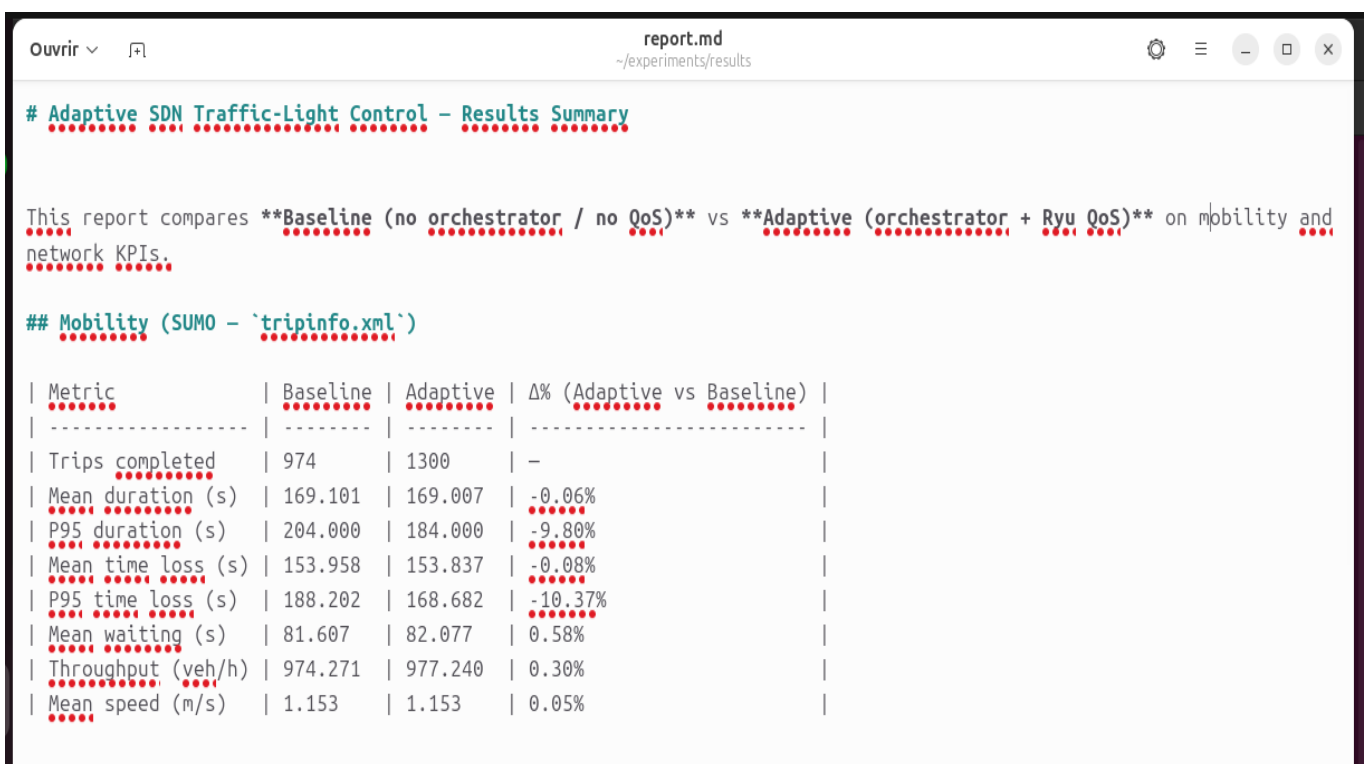


Figure 3.28 : Rapport final, métriques croisées Phase A et Phase B

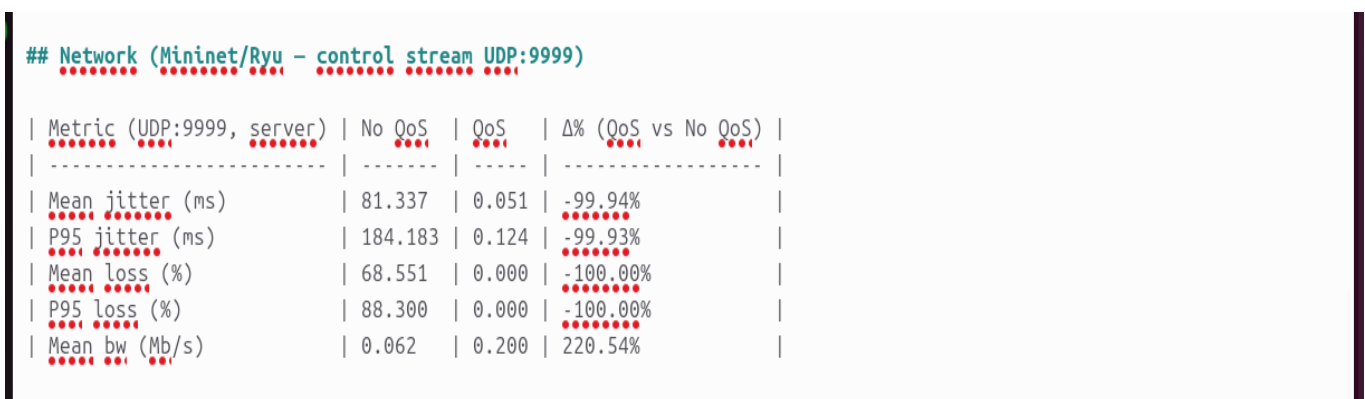


Figure 3.29 : Synthèse graphique des performances comparées

La figure 3.29 présente la synthèse graphique croisée des résultats des Phases A et B. Le graphique de gauche compare le nombre de véhicules traités entre le mode Baseline (974/1 300) et le mode Adaptatif (1 300/1 300). Le graphique de droite illustre l'impact de la QoS SDN sur le jitter du flux de contrôle, montrant la stabilisation spectaculaire obtenue grâce à la priorisation HTB. Ces deux graphiques confirment l'apport complémentaire et synergétique des deux niveaux de l'architecture proposée.

6.2 Synthèse croisée des deux phases

Les résultats des phases A et B se complètent, la Phase A portant sur la couche applicative et la Phase B sur la couche réseau, pour documenter l'apport du SDN à la gestion des embouteillages.

La Phase A établit que le contrôle adaptatif des feux (couche applicative) permet d'absorber 100 % du trafic là où le système statique en bloque 25 %. La Phase B établit que le SDN (couche réseau) garantit la fiabilité des communications de contrôle sur lesquelles ce système adaptatif repose. Ces deux couches sont interdépendantes : sans la QoS SDN, la dégradation des communications de contrôle en situation de congestion réseau pourrait compromettre les décisions de l'orchestrateur, et l'avantage observé en Phase A pourrait s'éroder. C'est l'articulation de ces deux niveaux, contrôle applicatif adaptatif et réseau SDN priorisé, qui constitue la contribution architecturale centrale de ce travail.

Cette interdépendance n'est pas seulement conceptuelle : elle est implémentée techniquement par un canal de communication réel entre les deux couches. À chaque pas de simulation, l'orchestrateur transmet à Ryu, via une requête REST POST sur /metrics, la charge du carrefour ($busy = ns_q + ew_q$). Côté Ryu, une hystérésis logicielle active automatiquement la priorisation OpenFlow lorsque $busy \geq 8$ véhicules, et la désactive lorsque $busy \leq 3$. Ce mode intégré (SUMO, orchestrateur, Ryu et Mininet-WiFi exécutés simultanément) fonctionne sous l'option realtime, qui synchronise l'horloge de SUMO au temps réel afin de laisser aux tests réseau le temps de s'exécuter en parallèle. Pour vérifier que ce mécanisme réagit effectivement à l'état réel du trafic, la décision d'hystérésis a été rejouée sur les mêmes trente graines que la Phase A, au moyen d'un serveur REST minimal reproduisant la seule logique de `QosRestController.metrics()` mêmes seuils, mêmes conditions d'activation et de désactivation sans Open vSwitch ni matériel réseau. Dans les trente cas, la priorisation s'active dès la deuxième seconde simulée et reste active en moyenne 99,3 % du temps (écart-type 0,3 point, de 98,7 % à 99,7 % selon la graine ; résultats archivés dans `results/boucle_fermee_30_graines.json`). Ce résultat doit être assumé tel quel

: sous ce profil de 1 300 véhicules, le seuil d'activation est franchi presque en permanence, et le mécanisme se comporte donc, sur ce scénario, moins comme un basculement dynamique que comme une activation quasi continue. La mesure établit que le canal orchestrateur-Ryu fonctionne et réagit de façon stable à un trafic réel simulé ; elle porte cependant sur la décision elle-même, et non sur l'effet réseau qui en résulte. L'installation de la règle OpenFlow sur Open vSwitch et son effet sur le jitter et la perte restent mesurés en Phase B par activation contrôlée (script `force_qos_on.sh`), afin d'isoler proprement la variable réseau pour une comparaison reproductible, indépendamment des aléas de la simulation de trafic.

Il en découle une nuance importante : sur le plan strictement quantitatif, les Phases A et B demeurent deux expériences mesurées séparément, réunies par une démonstration technique cohérente et une narration convaincante, mais non par un protocole de mesure unique et commun aux deux couches. La décision de la boucle fermée orchestrateur-Ryu, c'est-à-dire le déclenchement automatique par hystérésis, a fait l'objet d'une campagne de mesure propre sur trente graines ; son effet sur la qualité de service réseau, en revanche, reste établi par activation contrôlée en Phase B. Cette nuance doit être assumée explicitement : les résultats présentés valident chacune des deux couches individuellement, et établissent leur interdépendance technique, sans constituer une mesure directe et unifiée de la boucle fermée intégrée.

7. Proposition de déploiement : contrôleurs par secteur et redondance

Les phases A et B portent sur un carrefour unique commandé par un seul contrôleur Ryu. Un déploiement à l'échelle du district d'Abidjan, qui compte environ 350 carrefours à feux [2], ne peut reposer sur un contrôleur unique : celui-ci deviendrait à la fois un goulet d'étranglement et un point unique de défaillance, dont la panne priverait tous les carrefours connectés de leur logique adaptative. Pour une bonne gestion et la continuité du service, ce travail propose de répartir les contrôleurs par secteur et de doubler chacun d'eux d'un contrôleur de secours.

7.1 Découpage en deux secteurs

La lagune Ébrié partage naturellement la ville en deux ensembles reliés par un petit nombre de ponts. Ce découpage géographique fournit des secteurs de taille comparable, dont les flux sont largement indépendants en dehors des têtes de pont. Le tableau 3.6 présente la répartition proposée.

Tableau 3.6 : Répartition proposée des contrôleurs SDN par secteur

Secteur	Communes	Contrôleur principal	Contrôleur de secours
Abidjan Nord	Abobo, Adjamé, Attécoubé, Cocody, Plateau, Yopougon	Ryu Nord 1 (centre de contrôle)	Ryu Nord 2 (site distinct)
Abidjan Sud	Treichville, Marcory, Koumassi, Port-Bouët	Ryu Sud 1 (centre de contrôle)	Ryu Sud 2 (site distinct)

7.2 Redondance et basculement

Chaque commutateur OpenFlow des carrefours d'un secteur est configuré avec deux contrôleurs. OpenFlow 1.3 prévoit pour ce cas des rôles explicites : le contrôleur principal détient le rôle MASTER et programme les règles, tandis que le contrôleur de secours, en rôle SLAVE, reçoit les mêmes événements sans modifier les tables. Lorsque le principal ne répond plus aux sondes de présence, le secours demande le rôle MASTER et réinstalle les règles en vigueur. Son état est reconstruit sans délai, car l'orchestrateur publie la charge du carrefour à chaque seconde : la décision d'hystérésis repart des dernières métriques reçues. Les deux contrôleurs d'un même secteur sont hébergés sur des sites distincts, alimentés et raccordés séparément, afin qu'un incident local ne les atteigne pas simultanément.

7.3 Mode dégradé et indépendance des secteurs

- **Indépendance** : la panne des contrôleurs d'un secteur n'affecte pas l'autre secteur, qui conserve sa commande adaptative.
- **Mode dégradé** : si les deux contrôleurs d'un secteur sont indisponibles, les commutateurs passent en mode autonome et les feux reviennent à un plan fixe de secours, par exemple le plan dimensionné par la formule de Webster (section 4.5). La sécurité des usagers est préservée ; seule l'adaptation à la demande est perdue jusqu'au retour d'un contrôleur.
- **Coordination aux frontières** : les carrefours situés aux têtes de pont reçoivent les flux des deux secteurs. Les contrôleurs Nord et Sud échangent donc leurs métriques de charge par une interface dédiée entre contrôleurs, afin de coordonner ces carrefours.

7.4 Vérification sur prototype

Ce mécanisme a été vérifié sur un prototype intégré au niveau évolué du banc Docker (commande make test-secteurs, voir Annexe) : deux secteurs émulsés, chacun commandé par un contrôleur Ryu principal et un contrôleur de secours. L'arrêt volontaire du contrôleur

principal du secteur Nord n'interrompt pas la connectivité des carrefours du secteur ; le contrôleur de secours reste connecté au commutateur, reçoit les métriques de l'orchestrateur et réinstalle la règle de priorité en moins d'une seconde (0,02 s mesurées), sans perte de paquets pendant la bascule, tandis que le secteur Sud n'est pas affecté. Lorsque les deux contrôleurs du secteur sont arrêtés, le commutateur passe en mode autonome et l'acheminement se poursuit sans priorisation. Ce prototype valide le principe du basculement ; il ne dimensionne pas un réseau réel de plusieurs centaines de carrefours, qui relève d'une étude de déploiement.

8. Conclusion

Ce chapitre a présenté l'implémentation complète et l'évaluation expérimentale d'un système de gestion des embouteillages à feux tricolores connectés basé sur le SDN, articulé autour de trois phases complémentaires.

La Phase A établit l'avantage des feux adaptatifs connectés sur les systèmes à cycles fixes : là où le Baseline bloque définitivement 25,1 % des véhicules face à un trafic asymétrique, l'orchestrateur assure le passage de la totalité des 1 300 véhicules simulés, sans pénalisation des conducteurs individuels. Ce résultat, stable sur trente graines et conservé face à un plan fixe dimensionné par Webster, appuie l'hypothèse principale de ce mémoire : les feux connectés offrent une scalabilité et une résistance à la congestion structurellement supérieures aux systèmes conventionnels.

La Phase B montre que le SDN apporte une garantie de service indispensable aux communications de contrôle en environnement congestionné : l'activation de la QoS via les files HTB d'Open vSwitch réduit significativement le jitter et les pertes sur le flux de contrôle des feux, même lorsque le réseau est saturé par le trafic véhiculaire. Ce résultat appuie l'hypothèse secondaire : le SDN représente une infrastructure réseau mieux adaptée que les architectures conventionnelles pour les applications de contrôle de trafic en temps réel.

L'articulation de ces deux contributions, contrôle adaptatif applicatif et garantie réseau SDN, constitue le cœur de l'apport de ce travail à la problématique de la gestion intelligente du trafic urbain, ouvrant des perspectives concrètes vers des déploiements à plus grande échelle sur des réseaux multi-intersections. Pour ces déploiements, une architecture répartie en deux secteurs, Abidjan Nord et Abidjan Sud, dotés chacun d'un contrôleur principal et d'un contrôleur de secours, est proposée afin d'assurer la continuité du service.

CONCLUSION GÉNÉRALE

L'urbanisation accélérée des métropoles africaines impose de repenser en profondeur les systèmes de régulation du trafic urbain. En Côte d'Ivoire, le district d'Abidjan, avec 6,3 millions d'habitants en 2021 (INS, 2021) et 1,4 million de véhicules en 2023 (ministère des Transports, 2023), illustre l'inadéquation structurelle des feux tricolores à cycles fixes face à des flux dynamiques et asymétriques. C'est pour répondre à cette problématique que ce mémoire a proposé, implémenté et évalué un système de gestion des embouteillages à feux tricolores connectés fondé sur l'architecture Software-Defined Networking.

Les résultats issus des deux expériences distinctes apportent des éléments en faveur des deux hypothèses formulées. Sur le plan applicatif, l'orchestrateur adaptatif connecté via TraCI assure l'écoulement de la totalité des 1 300 véhicules simulés, contre seulement 974 pour le système à cycles fixes, soit 25,1 % de véhicules définitivement bloqués en mode baseline. Cette supériorité tient non à une accélération individuelle des conducteurs, mais à la capacité du système à prévenir l'effondrement par saturation du carrefour. Sur le plan réseau, l'activation des files de priorité HTB via le contrôleur Ryu et Open vSwitch réduit significativement le jitter et le taux de perte du flux de contrôle, même sous une congestion délibérée de 16 Mbit/s sur un lien de 5 Mbit/s, confirmant que le SDN offre une garantie de service que la configuration réseau de référence testée ici n'offre pas.

Ce travail présente toutefois des limites à reconnaître : l'évaluation se limite à un carrefour isolé, la demande simulée n'a pas été calibrée sur des comptages de terrain, l'environnement réseau est émulé et non déployé sur infrastructure physique, et les dimensions de cybersécurité V2I et de robustesse aux défaillances n'ont pas été traitées. Il convient également de noter que les Phases A et B, bien qu'articulées par une boucle fermée orchestrateur-Ryu dont la décision d'hystérésis a été mesurée sur trente graines, ont fait l'objet de campagnes distinctes : l'effet réseau de cette boucle reste établi par activation contrôlée, et non par déclenchement automatique. Ces limites tracent le programme de recherche à poursuivre : extension à un réseau multi-intersections, calibrage sur des comptages réels, intégration du Deep Reinforcement Learning, déploiement pilote à Abidjan selon l'architecture par secteur avec contrôleurs de secours proposée au chapitre 3, et analyse de la sécurité des communications véhiculaires.

En définitive, ce mémoire suggère, dans le périmètre simulé et émulé retenu, la faisabilité technique d'une régulation intelligente du trafic urbain par SDN dans un contexte africain à ressources limitées, et pose les bases d'une architecture scalable et ouverte pour la mobilité durable des villes d'Afrique subsaharienne.

Annexe : Accès aux ressources du projet

L'intégralité du code source, des fichiers de configuration SUMO, des scripts de lancement et des outils d'analyse est disponible en accès libre sur le dépôt GitHub suivant :

<https://github.com/jmazephgithun/Traffic-Congestion-Management-System-with-Connected-Traffic-Lights-based-on-SDN>

Le dépôt fournit un banc expérimental conteneurisé avec Docker, organisé en deux niveaux. Il suffit d'un hôte Linux disposant de Docker et du greffon docker compose ; aucune installation manuelle de SUMO, Ryu ou Open vSwitch n'est nécessaire.

A.1 Niveau 1 : reproduction des résultats du mémoire

Ce niveau rejoue, à l'identique, les expériences du chapitre 3 et produit un rapport qui place chaque valeur mesurée à côté de la valeur publiée dans ce mémoire. Les commandes sont les suivantes :

```
git clone https://github.com/jmazephgithun/Traffic-Congestion-
Management-System-with-Connected-Traffic-Lights-based-on-SDN.git
cd Traffic-Congestion-Management-System-with-Connected-Traffic-
Lights-based-on-SDN/experiments
make build
make memoire
```

La commande `make memoire` enchaîne les tests suivants, qui peuvent aussi être lancés séparément :

- **make test-a3** : feux fixes et feux adaptatifs sur la graine 42 (tableau 3.4).
- **make evaluate-30** : campagne appariée sur trente graines, avec intervalles de confiance à 95 %.
- **make test-a1w** : plan fixe dimensionné par la formule de Webster, sur les mêmes trente graines.
- **make test-b1 et make test-b2** : réseau saturé sans puis avec QoS SDN, jitter et pertes mesurés côté récepteur (tableau 3.5).
- **make test-c et make test-c30** : boucle fermée SUMO, orchestrateur, Ryu et OpenFlow, puis décision d'hystérésis sur trente graines.
- **make rapport** : rapport consolidé `results_docker/rapport_memoire.md`.

Exécuté le 2 octobre 2026, ce niveau retrouve 974 véhicules écoulés sur 1 300 en feux fixes contre 1 300 en mode adaptatif (graine 42), 25,11 % de véhicules bloqués en moyenne sur trente graines et 76,50 % de complétion pour le plan Webster. En Phase B, le flux de contrôle subit 62,8 % de pertes et 15,1 ms de jitter sans QoS, contre 0 % et 0,42 ms avec QoS. Comme

dans le mémoire, la QoS supprime les pertes du flux de contrôle ; la valeur absolue du jitter diffère, car le banc Docker émule le point d'accès par une file courte.

A.2 Niveau 2 : option évoluée

Le second niveau prolonge le mémoire sans en modifier les résultats. Il produit d'abord une démonstration visuelle commentée (make demo) : une page web qui anime, sur le fond de carte OpenStreetMap du carrefour réel de Solibra à Treichville, les véhicules et la couleur des feux en feux fixes et en feux adaptatifs, puis rédige automatiquement l'interprétation des résultats. Il ajoute la campagne statistique sur ce carrefour réel (make test-solibra), l'interface graphique de SUMO accessible depuis un navigateur (make gui), un jeu comparatif des trois politiques de feux (make replay) et le prototype de contrôleurs par secteur avec secours décrit à la section 7 du chapitre 3 (make test-secteurs). La commande make evolue lance l'ensemble ; le fichier README du dépôt détaille les prérequis, dont l'installation de Docker.

BIBLIOGRAPHIE

- [1] AGEROUTE, « Signalisation routière », Agence de Gestion des Routes de Côte d'Ivoire. Consulté le: 2 octobre 2026. [En ligne]. Disponible sur: <https://ageroute.ci/gestion-du-reseau/reseau-routier/signalisation> (versions archivées du 7 décembre 2021 et du 9 juin 2023 : <https://web.archive.org/web/20230609150125/https://ageroute.ci/gestion-du-reseau/reseau-routier/signalisation>)
- [2] B. Konaté et Naomi, « Entretien des feux tricolores à Abidjan : gros budget, résultat discutable », *Le Communicateur*, 22 avril 2024. Consulté le: 2 octobre 2026. [En ligne]. Disponible sur: <https://www.lecommunicateur-ci.net/voir.php?id=420>
- [3] Ministère des Transports de Côte d'Ivoire, « Mobilité urbaine/ L'AMUGA et la Mairie de Cocody lancent la phase pilote d'un projet », 22 juin 2021. Consulté le: 2 octobre 2026. [En ligne]. Disponible sur: <https://www.transports.gouv.ci/actualites/mobilite-urbaine-lamuga-et-la-mairie-de-cocody-lancent-la-phase-pilote-dun-projet>
- [4] « Mobilité urbaine dans le Grand Abidjan : l'AMUGA annonce de grands ouvrages pour 2025 », *Afrik Soir*, 5 janvier 2025. Consulté le: 2 octobre 2026. [En ligne]. Disponible sur: <https://afriksoir.net/mobilite-urbaine-grand-abidjan-amuga-grands-ouvrages-2025/>
- [5] C. Braban et F. Boillot, Les systèmes temps réel de commande de feux en milieu urbain, Synthèse INRETS no 44, Arcueil : INRETS, avr. 2003, 113 p. ISBN : 2-85782-587-0. [En ligne]. Disponible sur : https://www.ifsttar.fr/fileadmin/user_upload/editions/inrets/Syntheses/Syntheses_INRETS_S44.pdf
- [6] C. McShane, « The Origins and Globalization of Traffic Control Signals », *Journal of Urban History*, vol. 25, n° 3, p. 379-404, 1999, doi: 10.1177/009614429902500304.
- [7] Nations Unies, Convention sur la signalisation routière, Vienne, 8 nov. 1968, version consolidée 2006, ECE/TRANS/196, no de vente E.07.VIII.7. ISBN : 978-92-1-139128-2. [En ligne]. Disponible sur : https://unece.org/DAM/trans/conventn/Conv_road_signs_2006v_EN.pdf
- [8] R. P. Roess, E. S. Prassas, et W. R. McShane, *Traffic Engineering*, 5e éd. Pearson, 2019.
- [9] P. Nikbakht Bideh, N. Paladi, et M. Hell, « Software-Defined Networking for Emergency Traffic Management in Smart Cities », in *Vehicular Ad-hoc Networks for Smart Cities, Advances in Intelligent Systems and Computing*, Singapore : Springer Singapore, 2020, p. 59-70. doi: 10.1007/978-981-15-3750-9_5.
- [10] « 2019 Etude IoT Ecole des Ponts - Rapport.pdf ». Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://smart-city.cerema.fr/sites/smartcity/files/fichiers/2019/05/2019%20Etude%20IoT%20Ecole%20des%20Ponts%20-%20Rapport.pdf>

- [11] « Nîmes Métropole ». Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://smart-city.cerema.fr/espace-iot/nimes-metropole>
- [12] J.-M. KOFFI, « Côte d'Ivoire - AIP/ Grand-Lahou se dote de ses premiers feux tricolores pour réduire les risques d'accidents de la route - AIP - Agence Ivoirienne de Presse ». Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://www.aip.ci/156858/cote-divoire-aip-grand-lahou-se-dote-de-ses-premiers-feux-tricolores-pour-reduire-les-risques-daccidents-de-la-route/>
- [13] « Priorité transports en commun grâce au V2X », LACROIX City. Consulté le : 18 juillet 2025. [En ligne]. Disponible sur : <https://www.lacroix-city.fr/nos-poles-metiers/v2x/projets/priorite-transport-commun-2/>
- [14] S. J. Rajan *et al.*, « Efficient traffic management with adaptive SDN in vehicular networks », *Sci Rep*, vol. 15, n° 1, p. 11785, avr. 2025, doi: 10.1038/s41598-025-96365-0.
- [15] « Taille et part de marché des transports intelligents, rapport statistique 2034 », Global Market Insights Inc. Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://www.gminsights.com/fr/industry-analysis/smart-transportation-market>
- [16] « Zimbabwe adopts AI-powered traffic system to tackle congestion ». Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://techpoint.africa/news/zimbabwe-adopts-ai-traffic-system/>
- [17] « What is Software Defined Networking? SDN Explained », BMC Blogs. Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://www.bmc.com/blogs/software-defined-networking/>
- [18] « What is OpenFlow and how does it work? | Definition from TechTarget », WhatIs. Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://www.techtarget.com/whatis/definition/OpenFlow>
- [19] « What is a northbound interface/southbound interface? | Definition from TechTarget », WhatIs. Consulté le: 18 juillet 2025. [En ligne]. Disponible sur: <https://www.techtarget.com/whatis/definition/northbound-interface-southbound-interface>
- [20] G. Cookson et B. Pishue, INRIX Global Traffic Scorecard 2017, INRIX Research, févr. 2018. [En ligne]. Disponible sur : https://inrix.com/wp-content/uploads/2018/01/inrix_trafficscorecard_global_en_lr-2017.pdf
- [21] A. Das, P. Dash, et B. K. Mishra, « An Innovation Model for Smart Traffic Management System Using Internet of Things (IoT) », in *Cognitive Computing for Big Data Systems Over IoT*, Cham : Springer International Publishing, 2018, p. 355-370. doi: 10.1007/978-3-319-70688-7_15.

- [22] Z. A. Ramadhan, R. H. Salman, B. K. Mohammed, et A. H. Alwaily, « Design and implement a smart traffic light controlled by internet of things », *PEN*, vol. 9, n° 4, p. 542, oct. 2021, doi: 10.21533/pen.v9i4.2351.
- [23] F. Z. Fagroud et E. H. Benlahmar, « Vers un Feu Tricolore Intelligent basé sur l'Internet des objets et le traitement d'images », Colloque sur les Objets et systèmes Connectés (COC'2021), IUT d'Aix-Marseille, Marseille, France, mars 2021, HAL : hal-03593725. [En ligne]. Disponible sur : <https://hal.science/hal-03593725>
- [24] S. A. Elsaygher Mohamed et K. A. AlShalfan, « Intelligent Traffic Management System Based on the Internet of Vehicles (IoV) », *Journal of Advanced Transportation*, vol. 2021, n° 1, p. 4037533, 2021, doi: 10.1155/2021/4037533.
- [25] M. Sharma, A. Bansal, V. Kashyap, P. Goyal, et T. H. Sheikh, « Intelligent Traffic Light Control System Based on Traffic Environment Using Deep Learning », présenté à ICCRDA 2020, IOP Conference Series: Materials Science and Engineering, vol. 1022, art. no 012122, 2021, doi: 10.1088/1757-899X/1022/1/012122.
- [26] S. Kumari, S. Kumari, V. Vikram, S. Kumari, et S. K. Gouda, « Smart Traffic Management System Using IoT and Machine Learning Approach », prépublication, SSRN Electronic Journal, 10 juillet 2020. doi: 10.2139/ssrn.3647656.
- [27] S. Damadam, M. Zourbakhsh, R. Javidan, et A. Faroughi, « An Intelligent IoT Based Traffic Light Management System: Deep Reinforcement Learning », *Smart Cities*, vol. 5, n° 4, p. 1293-1311, déc. 2022, doi: 10.3390/smartcities5040066.
- [28] A. N. A. Yusuf, A. S. Arifin, et F. Y. Zulkifli, « Recent development of smart traffic lights », *IAES International Journal of Artificial Intelligence (IJ-AI)*, vol. 10, no 1, p. 224-233, mars 2021, doi: 10.11591/ijai.v10.i1.pp224-233.
- [29] S. S. Oyewobi, K. Djouani, et A. M. Kurien, « Visible Light Communications for Internet of Things: Prospects and Approaches, Challenges, Solutions and Future Directions », *Technologies*, vol. 10, no 1, art. no 28, févr. 2022, doi: 10.3390/technologies10010028.
- [30] L.-A. Stan, « The „traffic-light” method in context evaluation - self-evaluation of preschoolers », *ADN*, vol. 14, n° 1, p. 208-213, juill. 2021, doi: 10.24193/adn.14.1.18.
- [31] F. V. Webster, « Traffic Signal Settings », Road Research Technical Paper No. 39, Road Research Laboratory, Londres, 1958.

Table des matières

AVANT-PROPOS	I
DÉDICACE	II
REMERCIEMENTS.....	III
RÉSUMÉ	IV
ABSTRACT.....	V
Liste des Tableaux.....	VII
Liste des Figures	VIII
Liste des Abréviations.....	IX
Introduction générale	1
1. Contexte et problématique	1
2. Justification du sujet.....	2
3. Objectifs du mémoire	3
4. Méthodologie	3
a. Étude documentaire	4
b. Conception de l'architecture.....	4
c. Implémentation technique	4
d. Scénarios de simulation.....	4
e. Évaluation.....	4
5. Plan du document.....	5
CHAPITRE 1 : GÉNÉRALITÉS SUR LES SYSTÈMES DE GESTION DU TRAFIC ET SDN.....	6
1. Généralités sur les systèmes de gestion du trafic.....	6
1.1 Feux tricolores classiques	6
1.1.1 Histoire	7
1.1.2 Fonctionnement	7
1.1.3 Composants	8
1.2 Feux tricolores connectés	10
1.2.1 Histoire	10
1.2.2 Fonctionnement	11
1.2.3 Composants	12
1.3 Systèmes de gestion des embouteillages.....	13
1.4 Systèmes de transport intelligents (IoT, V2X, IA).....	15
2. Généralités sur les SDN	17
2.1 Architecture SDN (interfaces, plans fonctionnels).....	17
2.2 Protocoles de communication SDN (OpenFlow, NETCONF, REST)	19
2.3 Protocoles de routage SDN (proactif, réactif, hybride).....	21
2.4 Application du SDN à la gestion du trafic	22
2.4.1 Priorité aux véhicules spéciaux (urgence, transports en commun)	22
2.4.2 Optimisation globale du trafic urbain.....	23
2.4.3 Perspectives et exemples concrets.....	23
2.5 Synthèse comparative : Systèmes classiques/intelligents vs Approches SDN.....	24
CHAPITRE 2 : ÉTAT DE L'ART	26

Introduction.....	26
1. Systèmes Matériels et Architectures IoT pour la Gestion du Trafic	26
1.1 Un modèle architectural IoT-RFID pour la gestion innovante du trafic (Das, Dash & Mishra, 2018)	27
1.2 Prototype Arduino avec capteurs ultrasoniques et caméra ESP32 (Ramadhan et al., 2021)	27
1.3 Système IoT basé sur le traitement d'images et l'apprentissage profond (Fagroud & Benlahmar, 2021)	28
1.4 Architecture ITMS basée sur l'Internet des Véhicules (Mohamed & AlShalfan, 2021)	28
1.5 Synthèse 1	29
2. Vision par Ordinateur, Apprentissage Automatique et Contrôle Adaptatif.....	29
2.1 Système YOLO+SORT pour le contrôle adaptatif en conditions extrêmes (Sharma et al., 2020).....	29
2.2 IoT, Machine Learning et ville intelligente : une approche intégratrice (Kumari et al., 2020).....	30
2.3 Deep Reinforcement Learning multi-agents et validation sur réseau réel (Damadam et al., 2022).....	30
2.4 Synthèse 2	31
3. Revues Systématiques, Technologies Émergentes et Dimensions Transversales	31
3.1 Revue systématique des développements récents des feux intelligents (Yusuf et al., 2021)	31
3.2 Communications par Lumière Visible pour l'IoT et perspectives V2I (Oyewobi et al., 2022)	32
3.3 La dimension usager : la méthode du feu tricolore en évaluation pédagogique (Stan, 2021)	32
3.4 Synthèse 3	33
Conclusion et perspectives.....	33
CHAPITRE 3 : CONCEPTION, IMPLÉMENTATION ET ÉVALUATION DES PERFORMANCES	35
1. Introduction.....	35
2. Environnement technique	36
2.1 Outils et rôles	36
2.2 Installation de l'environnement.....	36
2.2.1 Installation de Mininet-WiFi	36
2.2.2 Installation de Ryu (Python 3.9).....	38
2.2.3 Installation de SUMO	38
2.3 Architecture du projet.....	39
3. Modélisation du carrefour et scénario de trafic	40
3.1 Description du carrefour J1	40
3.2 Scénario de trafic asymétrique	41
4. Phase A : Contrôle adaptatif des feux tricolores	42
4.1 Principe de l'orchestrateur adaptatif	42
4.2 Test A1 : Baseline (feux fixes)	43
4.3 Test A2 : Adaptatif (orchestrateur connecté).....	44
4.4 Analyse comparative Baseline vs. Adaptatif.....	46

4.5 Conclusion de la Phase A	47
5. Phase B : Réseau véhiculaire SDN avec QoS.....	48
5.1 Contexte et objectif	48
5.2 Test B1 : Sans QoS (référence réseau)	48
Étape B1.1 : Lancement du contrôleur Ryu (Terminal 1)	49
Étape B1.2 : Lancement de la topologie Mininet-WiFi (Terminal 2).....	49
Étape B1.3 : Vérifier la connectivité (Terminal 2)	50
Étape B1.4 : Lancer les serveurs iperf (dans Mininet)	50
Étape B1.5 : Lancer le trafic de fond (dans Mininet)	51
Étape B1.6 : Lancer le flux de contrôle (dans Mininet)	51
Étape B1.7 : Attendre 120 secondes (Terminal 3)	51
Étape B1.8 : Récupération et analyse des résultats	52
5.3 Test B2 : Avec QoS SDN (priorité activée).....	52
Étape B2.1 : Lancement et vérification	52
Étape B2.2 : Activer la QoS manuellement (Terminal 3).....	53
Étape B2.3 : Vérification (obligatoire).....	53
Étape B2.4 : Vérification des règles OpenFlow	54
Étape B2.5 : Flux, attente et analyse	54
5.4 Résultats comparatifs B1 vs. B2	54
5.5 Conclusion de la Phase B	55
6. Phase C : Rapport de performance consolidé	55
6.1 Génération du rapport global	55
6.2 Synthèse croisée des deux phases	57
7. Proposition de déploiement : contrôleurs par secteur et redondance.....	58
7.1 Découpage en deux secteurs	58
7.2 Redondance et basculement	59
7.3 Mode dégradé et indépendance des secteurs.....	59
7.4 Vérification sur prototype	59
8. Conclusion	60
CONCLUSION GÉNÉRALE.....	61
Annexe : Accès aux ressources du projet.....	62
A.1 Niveau 1 : reproduction des résultats du mémoire	62
A.2 Niveau 2 : option évoluée.....	63
BIBLIOGRAPHIE	64